

POWE.RS v2.0 Data Model Specification

Document Purpose: Complete specification of input/output data models for the refactored POWE.RS SDDP solver with MPI-based distributed computing.

Status: DRAFT - Hybrid MPI+OpenMP Architecture Specified

Last Updated: 2026-01-20

Review Item	Status
SDDP Specialist Review	Complete
Data Format Specialist Review	Complete
HPC Specialist Review	Complete (Two-Level Sync, NUMA, Work Distribution)
Rust Developer Review	Complete
Cut Preallocation Strategy	Full Preallocation with Dynamic Capacity (Section 5.4.3)
Parallelism Strategy	Hybrid MPI+OpenMP with dynamic dispatch (Section 6.1, 6.10-6.13)
Hot-Path Execution	PAR preprocessing, warm-start patterns, batch operations (Section 5.4.11-5.4.13)

Table of Contents

1. Design Principles
 2. Production Scale Reference
 - 2.1 Performance Expectations by Scale
 3. Input Data Model
 4. Output Data Model
 5. Internal Data Structures
 - 5.1 Core Algorithm Structures
 - 5.2 LP Subproblem Structure
 - 5.3 FCF with Replication
 - 5.4 Solver Interface Specification
 - 5.4.11 Uncertainty Observation Data (PAR Preprocessing)
 - 5.4.12 Backward Pass Warm-Start Pattern
 - 5.4.13 Hot-Path Execution Flow
 - 5.5 LP Scaling Specification
 6. MPI Communication Structures
 - 6.1 Hybrid MPI+OpenMP Architecture Overview
 - 6.2-6.8 Message Structures, Synchronization, Aggregation, etc.
 - 6.9 HPC Implementation Requirements (Critical)
 - 6.10 Dynamic Work Distribution
 - 6.11 Shared Memory Scenario Storage
 - 6.12 Two-Level Cut Aggregation
 - 6.13 Reproducibility Guarantees
 7. File Format Decisions
 8. Validation Requirements
 9. Next Steps
-

1. Design Principles

1.1 Format Selection Criteria

Data Type	Recommended Format	Rationale
Configuration & Parameters	JSON	Human-readable, easily editable, small size
Entity Registries	JSON	Structured objects with relationships
Time Series Data	Parquet	Columnar, compressed, efficient for large data
Policy Data (Cuts/States/Vertices)	FlatBuffers	Zero-copy deserialization, cache-friendly dense arrays, in-memory during training
Simulation Results	Parquet	High volume, per-entity indexing
Dictionaries/Metadata	CSV	Human-readable, small, universal

1.2 Key Design Goals

1. **Separation of Concerns:** Static system data vs. dynamic algorithm data vs. stochastic data
2. **Scalability:** File formats that scale to production sizes without memory explosion
3. **Reproducibility:** All inputs deterministically produce same outputs
4. **Warm-start Support:** Efficient serialization/deserialization of algorithm state
5. **Distributed I/O:** Rank 0 loads, broadcasts to workers (or parallel loading where beneficial)
6. **Declaration Order Invariance:** Results must not depend on the order entities are declared in input files

1.3 Declaration Order Invariance (Critical Requirement)

CRITICAL: The optimization results **MUST** be identical regardless of the order in which entities are declared in input files. This is a fundamental correctness requirement.

Principle: If a user declares hydros A and B, runs the program, then exchanges the declaration order (B before A), the numerical results must be **bit-for-bit identical** (given the same random seed and same IDs).

What determines identity: The **entity ID** is the sole identifier. Two runs are equivalent if:

- All entity IDs are the same
- All entity properties are the same
- All relationships (by ID) are the same
- The random seed is the same

What must NOT affect results:

- Order of entities in JSON arrays (**hydros, thermals, buses, lines**)
- Order of rows in Parquet tables
- Order of constraints in **generic_constraints.json**
- Order of stages in **stages.json** (sorted by ID internally)
- Order of blocks within a stage (sorted by ID internally)
- Order of correlation blocks or entities within correlation blocks

Implementation Requirements:

1. **Canonical Ordering:** After loading, all entity collections must be sorted by ID before any processing
2. **Deterministic Iteration:** All iterations over entities must use the canonical (sorted by ID) order
3. **LP Variable Ordering:** LP variables must be created in canonical order (by entity ID, then by block ID)
4. **LP Constraint Ordering:** LP constraints must be added in canonical order
5. **Random Number Generation:** Scenario generation must iterate entities in canonical order
6. **Cut Coefficients:** Cut coefficient ordering must follow the canonical state variable order

Note: During implementation, one can assume that all the inputs are mostly sorted, so use sorting algorithms that have grater performance if all the inputs are already sorted by ID, ideally returning in few cycles if the data is already sorted by ID.

Validation: The test suite must include order-invariance tests that:

1. Run the same case with entities in different declaration orders
2. Verify bit-for-bit identical results (costs, decisions, cuts)

```
// Canonical ordering example
impl System {
  /// Sort all entity collections by ID for order-invariant processing
  pub fn canonicalize(&mut self) {
    self.buses.sort_by_key(|b| b.id);
    self.lines.sort_by_key(|l| l.id);
    self.hydros.sort_by_key(|h| h.id);
    self.thermals.sort_by_key(|t| t.id);
  }
}

impl GenericConstraints {
  /// Sort constraints by ID for order-invariant processing
  pub fn canonicalize(&mut self) {
    self.constraints.sort_by_key(|c| c.id);
  }
}
```

Why This Matters:

- **Debugging:** Users can reorganize input files without worrying about result changes
- **Version Control:** Reordering entities for readability doesn't create spurious diffs in results
- **Correctness:** Non-deterministic behavior from ordering is a bug, not a feature
- **Parallelism:** MPI ranks must agree on ordering without communication

1.4 LP Subproblem Formulation Reference

Complete Formulation: See [MATHEMATICAL FORMULATIONS.md](#) for the authoritative mathematical specification of the SDDP algorithm and LP subproblem.

This data model specification focuses on **data structures and file formats**. The complete LP formulation, including:

- SDDP algorithm (forward/backward passes, convergence)
- Objective function and constraints
- Hydro production function models (constant, FPHA)
- Block formulation variants (parallel, chronological)
- Stochastic inflow modeling (PAR(p))
- Cut generation and aggregation
- Risk measures and advanced features

is documented in the mathematical formulations document.

Key Cross-References:

This Document	Mathematical Formulations	Description
hydros.json → productivity	Section 6.1	Constant productivity model
hydros.json → fpha_*	Section 6.2	FPHA coefficients
config.json → block_mode	Section 5	Block formulation variant
scenarios/inflow_models.parquet	Section 8	PAR(p) model parameters

This Document	Mathematical Formulations	Description
config.json → inflow_non_negativity	Section 9	Inflow treatment method
stages.json → transitions	Section 13	Discount rate
policy/cuts/	Section 10	Cut coefficients

Variable/Constraint Sizing: See Section 2 for production-scale LP dimensions.

2. Production Scale Reference

Based on the target production scenario:

Dimension	Value	Memory Impact
Stages	120	Graph size
Blocks per Stage	1-24 (varies), typically 3	LP structure, outputs
Hydros	160	State dimension
Max AR Order	12	State dimension, Variables/Constraints
Thermals	130	Variables
Buses	6	Variables/Constraints
Lines	10	Variables/Constraints
Forward Passes	200	Parallelism
Iterations	50	Cut pool size
Scenarios per Node	20	Branching
Simulation Scenarios	2000	Output size

2.1 State Dimension Estimates

2.1.1 State Variables and Dimension The **state dimension** determines the size of Benders cuts. State variables include:

Component	Count	Description
Storage	$\mathcal{H}$	End-of-stage reservoir volume for each hydro
AR Lags	$\sum_h P_h$	Inflow lag values for AR(P) models
Battery SOC	$\mathcal{BAT}$	Battery state of charge (if batteries exist)
GNL Committed	$\sum_{gnl} L_{gnl}$	GNL dispatch pipeline (if GNL exists)

State Dimension Formula:

$$N_{state} = N_{hydro} + \sum_{h=1}^{N_{hydro}} P_h + N_{battery} + \sum_{gnl} L_{gnl}$$

For production scale (160 hydros, AR order up to 12):

- Storage: 160
- AR lags: $160 \times 12 = 1920$ (worst case, all hydros use max order)
- Total: up to 2080

Note: The actual state dimension depends on the AR orders specified in `inflow_models.parquet`. If most hydros use AR(6), the dimension would be $160 + 160 \times 12 = 1120$.

2.2 Variable and Constraint Counts

2.2.1 Variable Count per Subproblem

Component	Formula	Typical Count
Future cost	1	1
Deficit	$N_{bus} \times N_{block} \times N_{seg}$	$6 \times 3 \times 3 = 54$
Excess	$N_{bus} \times N_{block}$	$6 \times 3 = 18$
Exchange (direct + reverse)	$2 \times N_{line} \times N_{block}$	$2 \times 10 \times 3 = 60$
Hydro storage	N_{hydro}	160
Hydro incremental inflow AR	$N_{hydro} \times P$	$160 \times 12 = 1920$
Hydro turbined flow	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Hydro spillage	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Hydro generation	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Hydro diversion	$N_{div} \times N_{block}$	$\sim 10 \times 3 = 30$
Hydro evaporation	$N_{evap} \times N_{block}$	$\sim 50 \times 3 = 150$
Hydro withdrawal	$N_{withdrawal} \times N_{block}$	$\sim 20 \times 3 = 60$
Hydro slacks	$N_{hydro} \times N_{block} \times 6$	$160 \times 3 \times 6 = 2880$
Thermal generation	$N_{thermal} \times N_{block} \times \bar{N}_{seg}$	$130 \times 3 \times 1.5 = 585$
Contracts	$(N_{imp} + N_{exp}) \times N_{block}$	$5 \times 3 = 15$
Pumping	$N_{pump} \times N_{block}$	$5 \times 3 = 15$
Total Variables		$\sim 7,500$

2.2.2 Constraint Count per Subproblem

Component	Formula	Typical Count
Load balance	$N_{bus} \times N_{block}$	$6 \times 3 = 18$
Hydro water balance	N_{hydro}	160
Incremental inflow AR dynamics	N_{hydro}	160
Lagged incremental inflow fixing	$N_{hydro} \times P$	$160 \times 12 = 1920$
Hydro generation (constant)	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Hydro generation (FPHA)	$N_{fpha} \times N_{block} \times \bar{M}_{planes}$	$50 \times 3 \times 10 = 1500$
Outflow definition	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Outflow bounds (min/max)	$2 \times N_{hydro} \times N_{block}$	$2 \times 160 \times 3 = 960$
Turbined min	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Generation min	$N_{hydro} \times N_{block}$	$160 \times 3 = 480$
Evaporation	$N_{evap} \times N_{block}$	$50 \times 3 = 150$
Water withdrawal	$N_{withdrawal} \times N_{block}$	$20 \times 3 = 60$
Generic constraints	$N_{generic}$	~ 50
Benders cuts (pre-allocated)	N_{cuts}	10,000–15,000
Total Constraints		$\sim 17,000$–$22,000$

Note: The constraint count is dominated by pre-allocated Benders cut slots. During early iterations, most cut constraints are inactive (bounds set to $[-\infty, +\infty]$).

2.2.3 Counting Formulas (Exact) For precise sizing, use the following formulas where parameters come from the configuration:

Variables:

```

N_VAR = 1
+ N_BUS × N_BLOCK × (AVG_DEF_SEGMENTS + 1)
+ 2 × N_LINE × N_BLOCK
+ N_HYDRO
+ N_HYDRO × AR_ORDER
+ N_HYDRO × N_BLOCK × 4
+ N_HYDRO_DIV × N_BLOCK
+ N_HYDRO_EVAP × N_BLOCK
+ N_HYDRO_WITHDRAWAL × N_BLOCK
+ N_HYDRO × N_BLOCK × N_SLACK_TYPES
+ N_THERMAL × N_BLOCK × AVG_COST_SEGMENTS
+ (N_CONTRACT_IMP + N_CONTRACT_EXP) × N_BLOCK
+ N_PUMP × N_BLOCK × 2
# theta
# deficit + excess
# exchange
# storage
# incremental inflow model
# q, s, g, inflow
# diversion
# evaporation
# withdrawal
# slack vars
# thermal
# contracts
# pump flow + power

```

Constraints:

```

N_CON = N_BUS × N_BLOCK
+ N_HYDRO
+ N_HYDRO
+ N_HYDRO × AR_ORDER
+ N_HYDRO × N_BLOCK
+ N_HYDRO_FPFA × N_BLOCK × AVG_FPFA_PLANES
+ N_HYDRO × N_BLOCK
+ N_HYDRO × N_BLOCK × 2
+ N_HYDRO × N_BLOCK
+ N_HYDRO × N_BLOCK
+ N_HYDRO_EVAP × N_BLOCK
+ N_HYDRO_WITHDRAWAL × N_BLOCK
+ N_GENERIC
+ N_CUT_CAPACITY
# load balance
# water balance
# inflow AR dynamics
# lagged inflow fixing
# generation (constant)
# FPFA (additional)
# outflow definition
# outflow bounds
# turbinized min
# generation min
# evaporation
# withdrawal
# generic constraints
# Benders cuts

```

State Dimension:

```

N_STATE = N_HYDRO
+ SUM(AR_ORDER[h] for h in HYDROS)
+ N_BATTERY
+ SUM(GNL_LAG[t] for t in GNL_THERMALS)
# storage
# AR lags
# SOC
# GNL pipeline

```

2.2.4 Sizing Calculator Tool A Python script is provided to calculate LP dimensions from a JSON configuration:

```

# Calculate sizes for production configuration
python scripts/lp_sizing.py scripts/lp_sizing_production.json

# Interactive mode with prompts
python scripts/lp_sizing.py --interactive

# Output as JSON for programmatic use
python scripts/lp_sizing.py scripts/lp_sizing_production.json --json

```

The script outputs:

- Variable counts by category
- Constraint counts by type
- State dimension breakdown
- Memory estimates (LP matrix, cut storage, solver workspace)

See `scripts/lp_sizing.py` for the implementation and `scripts/lp_sizing_production.json` for a production-scale configuration example.

2.3 Performance Expectations by Scale (old placeholder values, needs revision, mainly the memory requirements with the `lp_sizing` tool)

Purpose: This table provides expected timing targets for different problem scales, enabling performance validation and regression detection. Timings are per-iteration unless otherwise noted.

Hardware Assumptions:

Component	Specification
CPU	AMD EPYC 9R14 or equivalent (192 cores, 3.7 GHz base)
Memory	DDR5, 384 GB/node
Network	InfiniBand HDR (200 Gb/s) or equivalent
Storage	NVMe SSD for I/O operations

Test Systems:

Scale	Stages	Hydros	Thermals	Inflow AR Order	Scenarios	Ranks	Threads/Rank	Forward Time	Backward Time	Memory/Rank
Unit Test	3	1	2	0	2	1	1	<0.1s	<1s	<20 MB
Small	6	1	2	1	10	1	2	<0.2s	<2s	<50 MB
Medium	12	80	1	1	100	4	12	<5s	<15s	<2 GB
Large	24	160	1	1	200	16	16	<15s	<45s	<6 GB
Production	40	1	1	160	2000	200	16	<30s	<90s	<20 GB

Key Performance Indicators:

Metric	Target	Measurement Point
LP solve (warm-start)	<2 ms	Hot-path, 500-row problem
LP solve (cold-start)	<20 ms	First solve or basis invalid
RHS batch update	<100 s	500 constraint updates
Solution extraction	<50 s	Primal + basis to buffers
Cut broadcast	<5 ms	1000 cuts × 2080 coefficients
Parallel efficiency	>80%	At 128 ranks vs 1 rank
Warm-start hit rate	>70%	Forward pass consecutive stages

Scaling Expectations:

EXPECTED SCALING BEHAVIOR

Forward Pass:
Time (stages × LP_solve_time) / (ranks × threads)
Near-linear speedup expected up to scenarios/2 threads

Backward Pass:

Time $\text{stages} \times (\text{branch_solves} / \text{threads} + \text{sync_overhead})$
 Sequential stage dependency limits parallelism
 Communication overhead: ~5-10% at 64 ranks, ~15-20% at 256 ranks

Memory:

Per-rank $\text{cuts_per_stage} \times \text{state_dim} \times 8 + \text{solver_instances} \times 15\text{MB}$
 Shared memory (MPI windows) reduces replication within node

Convergence Reference:

Problem Type	Typical Iterations	Optimality Gap
Simple (few hydros, short horizon)	10-20	<0.1%
Medium (regional system)	30-50	<0.5%
Complex (full national grid)	50-100	<1.0%
With CVaR risk measure	+20-50% iterations	Same gap target

Note: Iteration counts assume reasonable initial policy (warm-start from previous study). Cold-start may require 2-3× more iterations.

3. Input Data Model

3.1 Directory Structure

Note: Scenario noise generation always uses standard normal distributions. Non-negative inflow values are enforced at runtime via the configured `inflow_non_negativity` method. Deterministic load can be achieved by setting variance to 0 in the uncertainty model.

```
case_directory/
  config.json           # Algorithm configuration
  stages.json           # Stage definitions with blocks (incl. pre-study, Markov states)
  initial_conditions.json # Initial storage, GNL pipelines
  penalties.json        # Global penalty defaults (required)
  system/
    buses.json          # Bus definitions with deficit segments
    lines.json          # Transmission line definitions
    hydros.json         # Hydro plant registry
    thermals.json       # Thermal plant registry
    hydro_geometry.parquet # Volume-height-area tables for evaporation/FPHA (optional)
    hydro_production_models.json # Production function model per stage (optional)
    hydro_production_data.parquet # Tailrace/losses data for FPHA (optional)
    pumping_stations.json # Pumped storage / elevatórias (optional)
    energy_contracts.json # Import/export energy contracts (optional)
    non_controllable_sources.json # Wind/solar sources (optional, DEFERRED)
    batteries.json      # Battery storage (optional, DEFERRED)
  scenarios/
    correlation.json     # Correlation profiles (default + named profiles)
    correlation_schedule.parquet # Stage → profile mapping (optional)
    load_factors.json    # Load distribution by block (optional)
    exchange_factors.json # Exchange limits by block (optional)
    inflow_models.parquet # PAR model parameters per hydro × stage
    load_models.parquet  # Load model parameters per bus × stage
    inflow_history.parquet # Historical inflows for AR initialization
    non_controllable_models.parquet # Wind/solar stochastic models (optional, DEFERRED)
```



```

constraints/
  bus_penalties.parquet      # Stage-varying bus penalties (sparse, optional)
  hydro_penalties.parquet    # Stage-varying hydro penalties (sparse, optional)
  thermal_bounds.parquet     # Time-varying thermal bounds (optional)
  hydro_bounds.parquet       # Time-varying hydro bounds (optional)
  line_bounds.parquet        # Time-varying line bounds (optional)
  contract_bounds.parquet    # Time-varying contract bounds (optional)
  battery_bounds.parquet     # Time-varying battery bounds (optional, DEFERRED)
  generic_constraints.json    # User-defined linear constraints
  constraint_bounds.parquet  # Time-varying constraint bounds
simulation/                  # Simulation-related data
  external_scenarios/       # External (deterministic) scenarios for simulation (optional)
    inflows.parquet         # Scenario-based inflows
    loads.parquet           # Scenario-based loads
    non_controllable.parquet # Scenario-based non-controllable sources
policy/                      # Policy data directory (input/output, auto-created)
  metadata.json             # Algorithm state, RNG, bounds (optional on input)
  state_dictionary.json      # State variable mapping (required if cuts exist)
  cuts/                    # Outer approximation (standard SDDP cuts)
    stage_000.bin
    stage_001.bin
    ...
  states/                  # Visited states for cut selection
    stage_000.bin
    ...
  vertices/                # Inner approximation (SIDP upper bounds, optional)
    stage_000.bin
    ...
  basis/                   # Solver basis for exact reproducibility (optional)
    stage_000.bin
    ...

```

3.2 Configuration (config.json)

Note: Solver selection (HiGHS, CPLEX, Gurobi) is determined at compile time via Cargo features due to licensing constraints. Solver parameters, retry strategies, warm-start, and basis reuse are hardcoded per solver implementation and not user-configurable.

```

{
  "$schema": "https://powers-rs.io/schemas/v2/config.schema.json",
  "version": "2.0.0",

  "mpi": {
    "threads_per_rank": "auto",
    "thread_binding": "auto",
    "places": "auto",

    "scheduler_integration": {
      "enabled": true,
      "priority": ["slurm", "pbs", "lsf", "config"],
      "fallback_threads": 4,
      "memory_safety_factor": 0.9,
      "warn_on_override": true
    },

    "communication": {
      "cut_aggregation": "hierarchical",
      "aggregation_tree_fanout": 8,

```

```

    "backward_pipeline": true,
    "use_persistent_collectives": true,
    "use_shared_memory_windows": true
  },

  "memory": {
    "fcf_sharing": "intra_node_shared",
    "numa_aware_allocation": true,
    "first_touch_init": true
  },

  "io": {
    "parallel_warm_start": true,
    "checkpoint_writers": 4,
    "checkpoint_compression": "zstd"
  },

  "solver": {
    "threads_per_solve": 1
  }
},

"modeling": {
  "block_mode": "parallel",
  "inflow_non_negativity": "truncate_zero"
},

"horizon": {
  "mode": "finite",
  "max_horizon_length": 240,
  "cycle_discretization_delta": 0.1
},

"training": {
  "seed": 42,
  "num_forward_passes": 200,
  "stopping_rules": [
    {"type": "iteration_limit", "limit": 50},
    {"type": "statistical", "confidence": 0.95, "tolerance": 0.01}
  ],
  "stopping_mode": "any",
  "cut_formulation": "single",
  "forward_pass": {
    "type": "default"
  },
  "cut_selection": {
    "enabled": true,
    "method": "domination",
    "threshold": 0
  }
},

"upper_bound_evaluation": {
  "enabled": true,
  "initial_iteration": 10,
  "interval_iterations": 5
},

```

```

"policy": {
  "path": "./policy",
  "mode": "fresh",
  "checkpointing": {
    "enabled": true,
    "initial_iteration": 10,
    "interval_iterations": 10,
    "store_basis": true,
    "compress": true
  },
  "validate_compatibility": true
},

"simulation": {
  "enabled": true,
  "num_scenarios": 2000,
  "policy_type": "outer",
  "output_path": "./simulation",
  "output_mode": "streaming",
  "sampling_scheme": {
    "type": "in_sample"
  }
},

"exports": {
  "training": true,
  "cuts": true,
  "states": true,
  "vertices": true,
  "simulation": true,
  "forward_detail": false,
  "backward_detail": false,
  "compression": "zstd"
}
}

```

MPI Configuration (HPC Parameters)

Background: POWE.RS uses hybrid MPI+OpenMP parallelism for distributed computing. The `mpi` section configures communication patterns, memory management, and I/O strategies optimized for production-scale SDDP on HPC clusters.

SLURM/PBS/LSF Integration: Thread and memory configuration should come from the job scheduler, not hardcoded in config. Use `"auto"` mode (default) to respect scheduler allocations.

Scheduler Integration (CRITICAL):

Field	Type	Default	Description
<code>scheduler_integration.enabled</code>	boolean	true	Enable automatic scheduler detection and configuration
<code>scheduler_integration.priority</code>	string	["slurm", "pbs", "lsf", "config"]	Priority order for configuration sources
<code>scheduler_integration.fallback_threads</code>	integer	4	Threads per rank when no scheduler detected
<code>scheduler_integration.memory_safety_factor</code>	float	0.9	Use only this fraction of allocated memory

Field	Type	Default	Description
<code>scheduler_integration.warn_on_override</code>	bool	true	Warn if config overrides scheduler settings

Priority Order: When `threads_per_rank = "auto"`, the application reads from: 1. `SLURM_CPUS_PER_TASK` (if SLURM job) 2. `PBS_NUM_PPN` (if PBS/Torque job) 3. `LSB_MCPU_HOSTS` (if LSF job) 4. `OMP_NUM_THREADS` (if set externally) 5. `scheduler_integration.fallback_threads` (local run)

Thread Binding and Affinity:

Field	Type	Default	Description
<code>threads_per_rank</code>	string or i32	"auto"	"auto" (from scheduler), or explicit thread count
<code>thread_binding</code>	string	"auto"	"auto" (from scheduler), "close", "spread", "master"
<code>places</code>	string	"auto"	"auto" (from scheduler), "cores", "threads", "sockets"

Never hardcode `threads_per_rank` in production. Hardcoded values override scheduler allocations, causing severe performance degradation from thread oversubscription. Example: config says 192 threads but SLURM allocated only 96 CPUs → 2x oversubscription.

EPYC/High-Core Systems: For 192-vCPU EPYC instances with 8 NUMA nodes, let SLURM set threads via `--cpus-per-task=192`. The application will auto-detect and use `thread_binding: "close"` with `places: "cores"` for NUMA locality.

Communication Configuration:

Field	Type	Default	Description
<code>cut_aggregation</code>	string	"flat"	Cut gathering strategy: "flat" (all-to-master), "hierarchical" (tree-based)
<code>aggregation_tree_fanout</code>	i32	8	Children per node in hierarchical aggregation tree
<code>backward_pipeline</code>	bool	false	Enable pipelined backward pass (overlap stage t communication with stage t+1 computation)
<code>use_persistent_collectives</code>	bool	false	Use MPI 4.0 persistent collectives for iterative operations
<code>use_shared_memory_windows</code>	bool	false	Use MPI shared memory windows for intra-node data sharing

Aggregation Strategy Guidance:

Ranks	Recommended Strategy	Rationale
1-16	flat	Master overhead negligible
16-64	hierarchical (fanout 4-8)	Reduces master serialization
64+	hierarchical (fanout 8-16)	Critical for scalability

Hierarchical Aggregation: Instead of N-1 sends to rank 0, use a tree structure where intermediate ranks aggregate their children's cuts before forwarding. For 128 ranks with fanout 8, reduces master receive operations from 127 to ~16.

Flat (128 ranks):

```

R1
R2
R3      R0 (master)
...      (127 receives)
R127

R0-7    L1_0
R8-15   L1_1
R16-23  L1_2      R0 (master)
...      (15 receives)
R120-127 L1_15

```

Memory Configuration:

Field	Type	Default	Description
fcf_sharing	string	"replicated"	FCF storage strategy: "replicated" (full copy per rank), "intra_node_shared" (shared memory window per node)
numa_aware_allocation	bool	false	Enable NUMA-aware memory allocation via first-touch policy
first_touch_init	bool	false	Initialize arrays in parallel to ensure NUMA-local allocation

Intra-Node FCF Sharing: For production scale (18.6 GB cuts), using "intra_node_shared" with MPI shared memory windows reduces per-node memory from $18.6 \text{ GB} \times \text{ranks_per_node}$ to $18.6 \text{ GB} \times 1$. Critical for running multiple ranks per node.

I/O Configuration:

Field	Type	Default	Description
parallel_warm_start	bool	false	Load warm-start cuts in parallel (each rank loads subset of stages)
checkpoint_writers	i32	1	Number of ranks that write checkpoints (reduces filesystem contention)
checkpoint_compression	string	"zstd"	Checkpoint compression: "none", "lz4", "zstd"

Parallel I/O Pattern: For warm-start loading, distribute stage files across ranks: rank r loads stages where $\text{stage_id} \% \text{world_size} == r$, then uses MPI_Bcast to share. Reduces load time from $O(\text{total_size})$ to $O(\text{total_size} / \text{world_size})$.

Solver Configuration:

Field	Type	Default	Description
threads_per_solve	i32	1	Threads allocated to each LP solve (HiGHS internal threading)

Critical: With 192 OpenMP threads per rank solving scenarios in parallel, HiGHS solver threads cause severe oversubscription. **Always set threads_per_solve: 1** for SDDP workloads where parallelism is across scenarios, not within LP solves.

Environment Variables:

The application respects job scheduler environment variables with higher priority than config.json:

Variable	Source	Config Equivalent	Notes
SLURM_CPUS_PER_TASK	SLURM	mpi.threads_per_rank	Highest priority - never override
SLURM_MEM_PER_NODE	SLURM	(memory validation)	Used for memory safety checks
PBS_NUM_PPN	PBS/Torque	mpi.threads_per_rank	PBS cores per node
LSB_MCPU_HOSTS	LSF	mpi.threads_per_rank	LSF CPU allocation
OMP_NUM_THREADS	Scheduler/User	mpi.threads_per_rank	Standard OpenMP variable
OMP_PROC_BIND	Scheduler/User	mpi.thread_binding	Thread binding policy
OMP_PLACES	Scheduler/User	mpi.places	Thread placement
POWERS_MPI_THREADS	User	mpi.threads_per_rank	Lowest priority - override only if scheduler not detected
POWERS_CUT_AGGREGATION	User	mpi.communication.cut_aggregation	Algorithm optimization
POWERS_FCF_SHARING	User	mpi.memory.fcf_sharing	Memory strategy

Scheduler Detection: At startup, the application detects the job scheduler via environment variables:

Environment Variable	Detected Scheduler
SLURM_JOB_ID	SLURM
PBS_JOBID	PBS/Torque
LSB_JOBID	LSF
None	Local run (use config values)

Block Mode Configuration

Mode	Description
parallel	Blocks are independent within a stage. Storage balance applies to the whole stage (sum of block durations). Simpler model, common for medium/long-term planning.
chronological	Blocks are sequential within a stage. Storage can vary between blocks (inter-block storage variables). Enables daily/weekly cycling patterns. More variables and constraints.

Chronological Blocks Details:

- Storage continuity: `storage_end_block[b] = storage_start_block[b+1]`
- Load balance per block: Each block has its own load balance constraint
- Hydro balance per block: Inflow distributed across blocks, turbinized/spillage per block
- State: Only end-of-stage storage is part of the SDDP state (not inter-block)

Inflow Non-Negativity Methods

Background: Autoregressive (AR) models can generate negative inflow values, which are physically impossible. Different treatment methods have trade-offs between physical validity, statistical properties preservation, and computational cost. The methods below are based on SPARHTACUS approaches documented in [Larroyd et al., 2022](#).

Method	SPARHTACUS Name	Description
none	sem_relaxacao	No treatment - negative inflows are passed to the LP. May cause infeasibility. Use only for debugging or when AR models are guaranteed positive.
penalty	penalizacao	Add a slack variable QINC_FINF with penalty in objective function. The LP remains feasible, and the penalty discourages negative values. Recommended for most cases.
truncation	truncamento	Hard truncation: if AR generates negative, set inflow = 0. Simple but may bias the distribution and affect AR dynamics.
truncation_with_penalty	truncamento_penalizacao	Combines truncation with a penalty slack on the AR residual (YP_FINF). Truncates the final inflow but penalizes the statistical violation in the noise term.

Detailed Method Descriptions:

1. none (sem_relaxacao):

- The AR model output is used directly without modification
- If negative inflows occur, they appear in the water balance constraint
- The LP may become infeasible in dry scenarios
- **Use case:** Testing, or when PAR(p) model is calibrated to never produce negatives

2. penalty (penalizacao):

- Adds a non-negative slack variable inflow_slack to the inflow equation: $Q_{inc} = Q_{ar} + inflow_slack$
- The slack has a high penalty cost in the objective function
- The optimizer uses slack only when AR produces negative values
- **Penalty cost:** Configured via inflow_violation_penalty in config (default: 1000.0 \$/m³/s)
- **Pros:** LP always feasible, clear cost signal, easy to track violations
- **Cons:** Adds variables/constraints, affects marginal water values slightly

3. truncation (truncamento):

- Simple rule: $Q_{inc} = \max(0, Q_{ar})$
- Applied during scenario generation, before LP construction
- **Pros:** Simple, fast, no additional LP variables
- **Cons:** Biases the distribution (shifts mean upward), breaks AR temporal correlation when truncation occurs, may affect long-term storage dynamics

4. truncation_with_penalty (truncamento_penalizacao):

- The AR noise term $_t$ is modified: $_t' = _t + YP_FINF$ where $YP_FINF \geq 0$
- The modified noise ensures $Q_{ar}(_t') \geq 0$
- The penalty $YP_FINF \times penalty_cost$ is added to the objective

- **Pros:** Preserves AR structure better than pure truncation, signals statistical violations
- **Cons:** More complex, requires noise adjustment in scenario tree

CEPEL PAR(p) Approach Note: The CEPEL NEWAVE/DECOMP models use a different strategy based on Lognormal 3-parameter distributions and AR order reduction when negative coefficients would contribute. This approach is not directly supported but can be approximated by providing pre-processed PAR models with guaranteed non-negative behavior.

Configuration:

```
{
  "modeling": {
    "inflow_non_negativity": {
      "method": "penalty",
      "penalty_cost": 1000.0
    }
  }
}
```

Field	Type	Default	Description
method	string	"penalty"	One of: "none", "penalty", "truncation", "truncation_with_penalty"
penalty_cost	f64	1000.0	\$/m ³ /s penalty for inflow violation (used by <code>penalty</code> and <code>truncation_with_penalty</code> methods)

Horizon Mode Configuration

Background: Standard SDDP operates on a finite horizon with a terminal cost function (often zero, leading to “end-of-world” effects). The **infinite-horizon approach** addresses this by recognizing that hydrothermal systems are inherently periodic and using a discount factor to ensure convergence. See [Costa et al., 2025](#) for mathematical foundations.

Mode	Description
finite	Standard finite horizon. Stages proceed from first to last, then stop. Terminal cost is zero or user-defined.
infinite_periodic	Infinite horizon with periodic structure. The algorithm detects cycles in the transition graph and shares cuts between stages of the same “season” (same position in cycle). Requires <code>discount_rate > 0</code> in transitions.

How Infinite-Horizon Works:

1. **Cycle Detection:** The algorithm analyzes `transitions` in `stages.json` to find cycles. A cycle is formed when a transition points to an earlier stage (e.g., stage 59 → stage 48).
2. **Cut Sharing:** Stages in the same position of the cycle share their future cost function approximation. For a 12-stage cycle, stages 0, 12, 24, ... share cuts; stages 1, 13, 25, ... share cuts; etc.
3. **Convergence:** The discount factor < 1 ensures the fixed-point iteration converges. The algorithm iterates until the value functions stabilize (change $<$ tolerance).
4. **Forward Pass Cycling:** Once converged, forward passes cycle through the periodic stages indefinitely until the discounted cost contribution becomes negligible.
5. **Max Horizon Length:** Safety bound to prevent infinite loops if convergence is slow. The algorithm stops after this many stages in a single forward pass.

Configuration:

```
{
  "horizon": {
    "mode": "infinite_periodic",
    "max_horizon_length": 240
  }
}
```

Field	Type	Default	Description
mode	string	"finite"	One of: "finite", "infinite_periodic", or "markovian"
max_horizon_length	i32	240	Maximum stages to traverse in a single forward pass (safety bound). Required for <code>infinite_periodic</code> .
cycle_discretization_delta	f64	0.1	Convergence tolerance for cycle value function (for <code>infinite_periodic</code>).

Validation:

Mode	Validation Rules
infinite_periodic	At least one transition must create a cycle (<code>target_id < source_id</code> or equal to an ancestor) All transitions in the cycle must have <code>discount_rate > 0</code> <code>max_horizon_length</code> is required The algorithm will fail with a clear error if no cycle is detected or discount is missing
markovian	<code>markov_states</code> must be defined in <code>stages.json</code> All transitions must specify valid <code>source_markov</code> and <code>target_markov</code> states

Use Case: Long-term planning where you want water values that reflect long-term steady-state behavior rather than an artificial end-of-horizon effect. Particularly useful when the 5-year extension approach (current CEPEL practice) may not be sufficient.

Stopping Rules Configuration

Background: SDDP training can terminate based on multiple criteria. The algorithm supports combining rules via “any” (OR) or “all” (AND) logic. The `iteration_limit` rule is **mandatory** as a safety bound.

```
{
  "training": {
    "stopping_rules": [
      {"type": "iteration_limit", "limit": 50},
      {"type": "time_limit", "seconds": 3600},
      {"type": "statistical", "confidence": 0.95, "tolerance": 0.01}
    ],
    "stopping_mode": "any"
  }
}
```

Rule Type	Parameters	Description
iteration_limit	limit: i32	Mandatory. Stop after N iterations. Safety bound.
time_limit	seconds: f64	Stop after N seconds of training time.
statistical	num_replications: i32, iteration_period: i32, z_score: f64	Stop when deterministic bound falls within simulated confidence interval. See details below.
bound_stalling	iterations: i32, tolerance: f64	Stop when lower bound improvement is below tolerance for N consecutive iterations.
simulation	replications: i32, period: i32, distance_tol: f64, bound_tol: f64	Recommended. Hybrid heuristic combining bound stalling with policy stability.

Statistical Stopping Rule (Detailed)

Reference: Based on [SDDP.jl Statistical](#)

The **statistical** stopping rule performs Monte Carlo simulation of the policy and terminates when the deterministic bound (lower bound for minimization) falls within the confidence interval of simulated costs.

```
{
  "training": {
    "stopping_rules": [
      {"type": "iteration_limit", "limit": 200},
      {
        "type": "statistical",
        "num_replications": 100,
        "iteration_period": 5,
        "z_score": 1.96
      }
    ]
  }
}
```

Parameter	Type	Default	Description
num_replications	i32	100	Number of Monte Carlo simulations per evaluation
iteration_period	i32	1	Evaluate every N iterations
z_score	f64	1.96	Z-score for confidence interval (1.96 = 95% CI)

Convergence Test:

- Compute $\mu = \text{mean}(\text{simulated_objectives})$
- Compute $w = z_score \times \text{std}(\text{simulated_objectives}) / \sqrt{\text{num_replications}}$
- Stop if: $(\mu - w) \leq \text{bound}$ (for minimization)
- Stop if: $\text{bound} \leq (\mu + w)$ (for maximization)

Caution: This stopping rule can be unreliable. Key issues: 1. **Confidence width vs. cost:** Small `num_replications` $\rightarrow$ wide confidence interval $\rightarrow$ premature termination 2. **Non-normal distributions:** Simulated costs are often log-normal, not normal (especially in infinite horizon) 3. **Sequential testing bias:** Repeated testing inflates false positive rate above nominal level

Recommendation: Prefer `simulation` or `bound_stalling` rules for production. Use `statistical` only for research/debugging.

Simulation Stopping Rule (Recommended)

The simulation stopping rule is a hybrid heuristic that's more reliable than pure statistical tests:

```
{
  "training": {
    "stopping_rules": [
      {"type": "iteration_limit", "limit": 500},
      {
        "type": "simulation",
        "replications": 100,
        "period": 20,
        "distance_tol": 0.01,
        "bound_tol": 0.0001
      }
    ]
  }
}
```

Parameter	Type	Default	Description
replications	i32	auto	Number of simulations (-1 = auto-detect based on model)
period	i32	auto	Iterations between simulations (-1 = adaptive: 20 if 100 iter, 100 if 1000, else 500)
distance_tol	f64	0.01	Terminate when consecutive simulations differ by less than this
bound_tol	f64	0.0001	Bound must be stable (relative/absolute) before testing policy

Convergence Test:

1. Check bound stability: $|\text{bound}[k] - \text{bound}[k-5]| < \text{bound_tol} \times \max(1, |\text{bound}|)$
2. If stable, run simulation and compare to previous simulation
3. Terminate if $\sqrt{\Sigma(\text{distance}(\text{new}[i], \text{old}[i]))^2} < \text{distance_tol}$

Stopping Mode:

Mode	Description
any	Stop when any rule triggers (OR logic). Default.
all	Stop only when all rules trigger (AND/chain logic). Useful for ensuring statistical convergence AND minimum iterations.

Validation: At least one `iteration_limit` rule must be present in the `stopping_rules` array.

Forward Pass Configuration

Background: The forward pass samples scenarios and makes decisions based on the current policy. Different forward pass variants can affect exploration and convergence.

```
{
  "training": {
    "forward_pass": {
      "type": "default"
    }
  }
}
```

```
}
}
```

Type	Description	Status
<code>default</code>	Standard forward pass: sample scenarios, make decisions using cuts	Implemented
<code>risk_adjusted</code>	Forward pass with risk-adjusted sampling (oversample tail scenarios)	DEFERRED

Note: Risk-adjusted forward passes can improve convergence for risk-averse problems by exploring more worst-case scenarios during training. This is planned for future implementation.

Backward Pass Configuration

Background: The backward pass computes cuts by walking stages in reverse order. Two modes are supported, trading off convergence rate against parallelization potential.

```
{
  "training": {
    "backward_pass": {
      "mode": "sequential"
    }
  }
}
```

Mode	Description	Status
<code>sequential</code>	Standard backward pass: compute cuts $T \rightarrow 1$, each stage uses freshly-computed V_{t+1}^k . Tighter cuts, faster convergence. Default.	Implemented
<code>pipelined</code>	Overlapped computation/communication: each stage uses V_{t+1}^{k-1} from previous iteration. Looser cuts but reduced wall-clock time.	DEFERRED

Note: The sequential mode follows SDDP.jl's default implementation and produces tighter cuts by using the most recent approximation at each stage. The pipelined mode trades cut quality for reduced latency in distributed settings. See Section 6.5 for detailed trade-off analysis and mathematical justification.

Cut Formulation Configuration

```
{
  "training": {
    "cut_formulation": "single"
  }
}
```

Value	Description	Status
<code>single</code>	Single-cut: one aggregated cut per iteration. Default, currently implemented.	Implemented
<code>multi</code>	Multi-cut: one cut per scenario per iteration. More cuts, faster convergence, larger LPs.	DEFERRED

Note: Multi-cut formulation is documented in Section 3.2.3 (SDDP Algorithm Variants). The code is designed to support multi-cut in the future, but the initial implementation prioritizes robust single-cut behavior.

Numerical Tolerances Configuration

Design Decision: SDDP algorithm tolerances are **compile-time constants** for consistency and performance. LP solver tolerances are **solver-specific** and not user-configurable (use solver defaults optimized for numerical stability).

SDDP Algorithm Tolerances (Compile-Time)

These values are defined as constants in the Rust code and cannot be changed at runtime:

Constant	Value	Description
CUT_VIOLATION_TOL	1e-6	Minimum cut violation to consider cut active
BOUND_IMPROVEMENT_TOL	1e-6	Minimum bound improvement between iterations
STATE_EQUALITY_TOL	1e-8	Tolerance for comparing state vectors
OBJECTIVE_TOL	1e-6	Tolerance for objective value comparisons
CONSTRAINT_TOL	1e-6	Tolerance for constraint satisfaction checks

```
// In solver/constants.rs (compile-time configuration)
pub const CUT_VIOLATION_TOL: f64 = 1e-6;
pub const BOUND_IMPROVEMENT_TOL: f64 = 1e-6;
pub const STATE_EQUALITY_TOL: f64 = 1e-8;
pub const OBJECTIVE_TOL: f64 = 1e-6;
pub const CONSTRAINT_TOL: f64 = 1e-6;
```

LP Solver Tolerances (Solver-Specific)

LP tolerances (primal/dual feasibility, optimality) are **not exposed** to users:

Solver	Primal Feasibility	Dual Feasibility	Optimality Gap
HiGHS	1e-7 (default)	1e-7 (default)	1e-7 (default)
Gurobi	1e-6 (default)	1e-6 (default)	1e-6 (default)
CPLEX	1e-6 (default)	1e-6 (default)	1e-6 (default)

Rationale: LP tolerances require expert knowledge to tune correctly. Incorrect settings can cause:

Setting	Problem
Too tight	Numerical failures, “infeasible” on feasible problems
Too loose	Inaccurate duals → poor cuts → slow/non-convergence

Default solver settings are carefully tuned and sufficient for most SDDP problems.

When Numerical Issues Arise:

If numerical difficulties occur, the algorithm:

1. Logs a warning with the problematic subproblem details
2. Attempts solver reset (re-solve from scratch, no warm-start)
3. If still failing, writes problematic LP to `debug/numerical_issue_stage_XXX.lp`

Users can then analyze the LP file externally or report issues.

Simulation Sampling Scheme Configuration

Background: During simulation, scenarios can be generated using the same distributions as training (`in_sample`), modified distributions (`out_of_sample`), or user-provided deterministic scenarios (`external`).

```

{
  "simulation": {
    "sampling_scheme": {
      "type": "in_sample"
    }
  }
}

```

Type	Description
in_sample	Use same stochastic model as training. Validates policy on training distribution. Default.
out_of_sample	Use modified stochastic model (different seeds, parameters). Tests policy robustness.
external	Use deterministic scenarios from <code>simulation/external_scenarios/</code> directory. For backtesting or specific analysis.

External Scenarios Directory:

When `sampling_scheme.type = "external"`, the algorithm reads scenarios from the fixed path `simulation/external_scenarios/`

```

simulation/
  external_scenarios/
    inflows.parquet  # Inflow scenarios
    loads.parquet   # Load scenarios (optional)

```

External Inflows Schema (`simulation/external_scenarios/inflows.parquet`):

Column	Type	Description
scenario_id	i32	Scenario index (0-based)
stage_id	i32	Stage ID
hydro_id	i32	Hydro plant ID
inflow_m3s	f64	Deterministic inflow value

External Loads Schema (`simulation/external_scenarios/loads.parquet`):

Column	Type	Description
scenario_id	i32	Scenario index (0-based)
stage_id	i32	Stage ID
bus_id	i32	Bus ID
load_mw	f64	Deterministic load value

Note: External scenarios allow backtesting the policy against historical sequences or stress-testing against specific scenarios. The number of scenarios is inferred from the maximum `scenario_id` in the files.

Policy Directory Configuration

Unified Policy Directory: POWERS uses a single `policy/` directory for both reading initial policy data (warm-start, checkpoint resume) and writing updated policy data. This simplifies the user experience: one directory contains all policy-related artifacts.

Field	Type	Default	Description
path	string	<code>"./policy"</code>	Directory for policy data (cuts, states, vertices, basis)
mode	string	<code>"fresh"</code>	How to initialize: <code>"fresh"</code> , <code>"warm_start"</code> , or <code>"resume"</code>
validate_compatibility	bool	true	Verify state dimension and entity compatibility when loading

Policy Modes:

Mode	Behavior
fresh	Start from scratch. Ignore any existing data in <code>policy/</code> .
warm_start	Load existing cuts/states to initialize, but reset iteration count and use fresh RNG seed. Useful for re-running with modified parameters.
resume	Load full algorithm state including RNG, iteration count. Continue exactly where interrupted. Requires <code>metadata.json</code> .

Cut Capacity Calculation:

Key Formula: $\text{capacity} = \text{warm_start_cuts} + (\text{max_iterations} \times \text{forward_passes})$

The cut pool capacity is determined at startup based on the policy mode:

Mode	Capacity Formula	Example
fresh	$\text{max_iterations} \times \text{forward_passes}$	$50 \times 200 = 10,000$
warm_start	$\text{loaded_cuts} + (\text{max_iterations} \times \text{forward_passes})$	$5,000 + 10,000 = 15,000$
resume	Same as checkpoint (no reallocation)	As saved

Memory Implications:

Memory per stage = $\text{capacity} \times \text{state_dimension} \times 8 \text{ bytes} \times 2 \text{ (coefficients + state)}$
 $= 15,000 \times 1,120 \times 8 \times 2$
 268 MB per stage

Total for 120 stages 32 GB

For HPC deployments with 256+ GB RAM, this is acceptable. The full preallocation strategy prioritizes **reproducibility** and **zero runtime allocation** over memory efficiency.

Slot Assignment:

Slots [0 N-1] [N N+M-1]
 |---- Warm-start cuts ----| |----- New training cuts -----|

Where:

N = number of cuts loaded from warm-start policy
 M = $\text{max_iterations} \times \text{forward_passes}$

New cut slot = $N + \text{iteration} \times \text{forward_passes} + \text{forward_pass_idx}$

See Section 5.4.3 for detailed slot management implementation.

Checkpointing Configuration (within policy):

Field	Type	Default	Description
<code>enabled</code>	<code>bool</code>	<code>false</code>	Enable periodic checkpointing
<code>initial_iteration</code>	<code>i32</code>	<code>0</code>	First iteration to write checkpoint (0 = after first iteration)
<code>interval_iterations</code>	<code>i32</code>	<code>10</code>	Checkpoint every N iterations after initial
<code>store_basis</code>	<code>bool</code>	<code>false</code>	Store solver basis for exact reproducibility
<code>compress</code>	<code>bool</code>	<code>true</code>	Compress parquet files (zstd)

Reproducibility Note: If `store_basis = false`, resuming from checkpoint may produce numerically different (but algorithmically equivalent) results compared to a straight run. This is because the solver may choose different pivots when reconstructing the basis. Set `store_basis = true` for exact reproducibility at the cost of ~20-30% larger files and additional I/O time.

Example Workflow:

1. **First run:** "mode": "fresh" → writes cuts/states to `policy/`
2. **Run crashes at iteration 37:** `policy/` contains checkpoint from iteration 30
3. **Resume:** "mode": "resume" → loads metadata, continues from iteration 30
4. **Sensitivity analysis:** Copy `policy/` to `case_b/policy/`, run with "mode": "warm_start" and different loads

Upper Bound Evaluation (Inner Approximation / SIDP)

Background: Standard SDDP constructs an **outer approximation** (lower bound) of the future cost function using cuts. The **inner approximation** (also called SIDP - Stochastic Inner Dynamic Programming) constructs an **upper bound** using vertex interpolation. This is particularly important when using CVaR risk measures, where Monte Carlo simulation cannot directly estimate the upper bound. See [Costa & Leclère, 2023](#) and [Philpott et al., 2013](#) for methodology.

Why Inner Approximation Matters:

1. **Convergence Guarantee:** The gap between lower bound (cuts) and upper bound (vertices) provides a true convergence criterion
2. **CVaR Compatibility:** Unlike Monte Carlo, inner approximation correctly handles risk-averse objectives
3. **Alternative Policy:** The inner approximation gives an “at most Y” guarantee instead of the usual “at least X”—useful for conservative operation planning

How It Works:

1. During training, at configured intervals, the algorithm builds upper-bound approximations $\nabla_t(\mathbf{x})$ using visited states as vertices
2. Each vertex stores (**state**, **cost-to-go value**) computed using the upper approximation of the next stage
3. The upper bound at a new point is computed via Lipschitz interpolation from nearby vertices
4. Requires Lipschitz constants for the value functions (auto-computed from problem structure or user-provided)

Configuration:

```
{
  "upper_bound_evaluation": {
    "enabled": true,
    "initial_iteration": 10,
    "interval_iterations": 5
  }
}
```


Field	Type	Default	Description
enabled	bool	false	Enable inner approximation / SIDP
initial_iteration	i32	10	First iteration to compute upper bounds
interval_iterations	i32	5	Compute upper bounds every N iterations

Note: Upper bounds are computed at iterations `initial_iteration`, `initial_iteration + interval_iterations`, `initial_iteration + 2 × interval_iterations`, etc.

Output: When enabled, vertices are written to `policy/vertices/stage_XXX.bin` (FlatBuffers format, see Section 7.2.1).

Simulation with Inner Approximation:

The `simulation.policy_type` field controls which approximation is used for simulation:

Policy Type	Description
outer	Use cuts (standard). Decisions are based on “future cost is at least X”. Default.
inner	Use vertices. Decisions are based on “future cost is at most Y”. More conservative.

```
{
  "simulation": {
    "policy_type": "inner"
  }
}
```

Lipschitz Constant Computation

Reference: Based on [SDDP.jl Inner Approximation](#) and the backward Lipschitz accumulation approach.

The inner approximation requires Lipschitz constants to bound the maximum rate of change of the value function. This enables valid upper-bound interpolation from nearby visited states.

Key Principle: The Lipschitz constant must be **larger than the largest possible dual multiplier** (subgradient) of the value function at any point in the state space.

Backward Accumulation Algorithm:

For a minimization problem with T stages:

```
L[T] = max_penalty          # Lipschitz at final stage = max penalty coefficient
                          # (e.g., deficit penalty $/MWh)
```

For `t = T-1` down to `1`:

```
    L[t] = L[t+1] + max_stage_penalty[t]    # Accumulates backwards
```

Example: If deficit penalty is \$1,000/MWh and we have 5 stages:

- `L[5]` = 1,000 (final stage)
- `L[4]` = 2,000 (accumulated)
- `L[3]` = 3,000
- `L[2]` = 4,000
- `L[1]` = 5,000 (worst case: deficit at every stage)

Configuration:

```

{
  "upper_bound_evaluation": {
    "enabled": true,
    "initial_iteration": 10,
    "interval_iterations": 5,
    "lipschitz": {
      "mode": "auto",
      "fallback_value": 10000.0
    }
  }
}

```

Field	Type	Default	Description
<code>lipschitz.mode</code>	string	"auto"	"auto" computes from penalties, "manual" uses provided values
<code>lipschitz.fallback_value</code>	float	10000.0	Default Lipschitz if auto-computation fails
<code>lipschitz.per_stage</code>	array	null	Manual per-stage Lipschitz constants (length = num_stages)
<code>lipschitz.scale_factor</code>	float	1.1	Safety multiplier for auto-computed values

Auto-Computation Sources:

The "auto" mode computes Lipschitz constants from:

1. **Deficit penalties** - `stages.json` → `deficit_penalty` field
2. **Curtailement penalties** - `stages.json` → `curtailement_penalty` field
3. **State bounds** - Maximum dual from binding storage constraints
4. **Fuel costs** - Upper bound on thermal generation duals

Per-State-Variable Lipschitz (Advanced):

For tighter bounds, Lipschitz constants can be specified per state variable:

```

{
  "upper_bound_evaluation": {
    "lipschitz": {
      "mode": "per_variable",
      "storage_lipschitz": 1000.0,
      "ar_lag_lipschitz": 100.0
    }
  }
}

```

Implementation Note: Using too-small Lipschitz constants produces invalid upper bounds (optimistic). Using too-large constants produces overly conservative bounds but remains valid. When in doubt, err on the larger side.

Output: When enabled, vertices are written to `policy/vertices/stage_XXX.bin` (FlatBuffers). Each vertex stores its per-vertex Lipschitz constant (see Section 7.2.1 **Vertex** schema).

SDDP Algorithm Variants (DEFERRED)

DEFERRED FEATURES: The following algorithm variations are planned for future implementation. The data model is designed to accommodate them, but they are not yet supported.

1. Markovian Policy Graphs

Background: Standard SDDP assumes **stagewise-independent** uncertainty—the random variables at each stage are independent of previous stages (conditioned on the state). **Markovian policy graphs** extend this by allowing **stagewise-dependent** uncertainty modeled via a Markov chain.

In a Markovian model, each stage may have multiple **Markov states** (e.g., wet/dry climate conditions), and transitions between Markov states are governed by probability matrices. This allows modeling phenomena like:

Phenomenon	Example
Climate persistence	Wet years tend to follow wet years
Economic cycles	Recession/expansion states
Equipment degradation	Degradation states

Reference: [SDDP.jl Markovian Tutorial](#)

Key Concepts:

Concept	Description
Markov State	A discrete state representing some persistent condition (climate, economic regime)
Node	A <code>(stage_id, markov_state)</code> tuple—each combination is a separate node in the policy graph
Transition Matrix	Per-stage matrix where element <code>[i,j]</code> is the probability of transitioning from Markov state <code>i</code> to state <code>j</code>
Separate Cuts	Each node <code>(t, m)</code> has its own set of cuts, since the cost-to-go depends on the Markov state

Planned Data Model Extension:

The current `stages.json` uses simple stage IDs. To support Markovian graphs, we extend it with JSON-based transition specification (no separate Parquet file needed, as Markov state counts are typically small even for large problems):

```
{
  "markov_states": {
    "enabled": true,
    "states": [
      {"id": 1, "name": "wet"},
      {"id": 2, "name": "dry"}
    ]
  },
  "stages": [
    {
      "id": 0,
      "markov_states": [1],
      "...": "..."
    },
    {
      "id": 1,
      "markov_states": [1, 2],
      "...": "..."
    }
  ],
}
```

```

"transitions": [
  {"source_id": 0, "source_markov": null, "target_id": 1, "target_markov": 1, "probability": 1.0},
  {"source_id": 1, "source_markov": 1, "target_id": 2, "target_markov": 1, "probability": 0.75},
  {"source_id": 1, "source_markov": 1, "target_id": 2, "target_markov": 2, "probability": 0.25},
  {"source_id": 1, "source_markov": 2, "target_id": 2, "target_markov": 1, "probability": 0.25},
  {"source_id": 1, "source_markov": 2, "target_id": 2, "target_markov": 2, "probability": 0.75}
]
}

```

Note on JSON vs Parquet: Even in large-scale problems (120+ stages), Markov state counts remain small (typically 2-5 states). The JSON representation is sufficient and maintains consistency with the existing transition format. Validation is conditional—`markov_states` fields are only required when `horizon.mode = "markovian"`.

Impact on Policy Directory:

With Markovian states, cuts are indexed by `(stage_id, markov_state)`:

```

policy/
  cuts/
    stage_000_markov_001.parquet
    stage_001_markov_001.parquet
    stage_001_markov_002.parquet
    ...

```

Why Deferred: Markovian policy graphs substantially increase algorithm complexity:

- Forward passes must track Markov state transitions
- Backward passes generate cuts for each `(stage, markov_state)` node
- State space grows by factor of `|markov_states|`
- Requires careful handling of stagewise-independent noise *within* each Markov state

2. Multi-Cut vs Single-Cut Formulation

Background: In SDDP, the future cost function is approximated by cuts. There are two formulations:

Formulation	Description
Single-Cut	One cut per iteration, aggregating all scenarios: $E[Q_{t+1}(x, \cdot)]$
Multi-Cut	One cut per scenario per iteration: $Q_{t+1}(x, \cdot_i)$ for each i

Reference: [Guigues & Bandarra, 2019](#)

Trade-offs:

Aspect	Single-Cut	Multi-Cut
Cuts per iteration	1	<code> scenarios </code>
LP size	Smaller (1 future cost variable)	Larger (<code> scenarios </code> future cost variables)
Convergence rate	Slower (more iterations)	Faster (fewer iterations)
Time per iteration	Faster	Slower
Memory	Lower	Higher
Numerical stability	More stable	Can have issues with risk measures
Best for	Large scenario counts, risk-averse	Small scenario counts, risk-neutral

LP Formulation Differences:

Single-Cut (current implementation):

$$\min \quad c'x + \alpha \quad (1)$$

$$\text{s.t.} \quad Ax \leq b \quad (2)$$

$$\alpha \geq \text{rhs}_k + \pi'_k(x - x_k) \quad \forall k \quad (3)$$

$$\alpha \geq 0 \quad (4)$$

Multi-Cut (future):

$$\min \quad c'x + \sum_i p_i \times \alpha_i \quad (5)$$

$$\text{s.t.} \quad Ax \leq b \quad (6)$$

$$\alpha_i \geq \text{rhs}_{k,i} + \pi'_{k,i}(x - x_k) \quad \forall k, i \quad (7)$$

$$\alpha_i \geq 0 \quad \forall i \quad (8)$$

Configuration:

```
{
  "training": {
    "cut_formulation": "single"
  }
}
```

Value	Description	Status
single	Standard SDDP with one aggregated cut per iteration. Default.	Implemented
multi	Multi-cut SDDP with one cut per scenario per iteration.	DEFERRED

Note: The code structure is designed to support multi-cut in the future. The initial implementation prioritizes robust single-cut behavior with cut selection.

Impact on Policy Directory:

With multi-cut, cuts would include scenario indexing:

Column	Type	Description
scenario_branch_idx	i32	Scenario branch index (0 to num_scenarios-1). Only present in multi-cut mode.

Why Deferred: Multi-cut requires significant changes:

- LP construction must handle multiple future cost variables
- Cut storage and selection becomes more complex
- Interaction with CVaR risk measures needs careful implementation
- Performance tuning (when to use which formulation) is problem-dependent

3. Objective States (Inner Approximation for Price Processes)

Background: In some problems, the objective function depends on exogenous random processes that don't fit the standard SDDP cut structure. For example, electricity spot prices that follow an AR process cannot be directly incorporated into cuts because they affect the objective coefficients, not the constraints.

Objective states extend SDDP to handle such cases by treating the exogenous process as an additional state variable and using inner approximation (Lipschitz interpolation) for the value function component that depends on it.

Reference: [SDDP.jl Objective States](#)

Key Concepts:

Concept	Description
Objective State	A state variable that affects objective coefficients (e.g., spot price)
Inner Approximation	Lipschitz-based interpolation for value function over objective states
Augmented State	Combined (<code>reservoir_state</code> , <code>objective_state</code>) for policy evaluation

Why Deferred:

- Requires inner approximation infrastructure (Lipschitz bounds, vertex interpolation)
- Interaction with risk measures is complex
- Not typically needed for hydrothermal dispatch where prices are deterministic (marginal cost-based)
- Can often be approximated by scenario-based approaches

4. Belief States (Partially Observable MDPs)

Background: Standard SDDP assumes the state is fully observable. **Belief states** extend SDDP to partially observable Markov decision processes (POMDPs), where the agent maintains a probability distribution (belief) over possible hidden states.

This is useful for modeling scenarios where:

Scenario	Description
Climate regime	Wet/dry is not directly observable but inferred from inflow data
Equipment health	State is estimated from noisy measurements
Economic indicators	Have measurement lag

Reference: [SDDP.jl Belief States](#)

Key Concepts:

Concept	Description
Hidden State	Underlying true state (e.g., climate regime) not directly observable
Observation	Noisy signal correlated with hidden state (e.g., recent inflows)
Belief	Probability distribution over hidden states, updated via Bayes' rule
Augmented State	Combined (<code>physical_state</code> , <code>belief</code>) for policy

Why Deferred:

- Research-level feature, not yet standard in production systems

- Significant complexity in belief propagation and cut generation
- Limited practical benefit for most hydrothermal applications
- Can often be approximated by expanding Markov states

5. Duality Handlers (Lagrangian Relaxation for MIP)

Background: Standard SDDP assumes all subproblems are linear programs (LPs). When integer variables are present (e.g., unit commitment), subproblems become mixed-integer programs (MIPs), breaking the convexity assumption needed for cuts.

Duality handlers provide methods to generate valid cuts from MIP subproblems:

Method	Description
Lagrangian relaxation	Relax integer constraints, solve relaxed LP, use dual to generate cut
Strengthened Benders	Use cutting plane techniques to improve cut quality

Reference: [SDDP.jl Integrality](#)

Key Concepts:

Concept	Description
Lagrangian Relaxation	Relax integer constraints to obtain LP, multiply violations by Lagrange multipliers
Subgradient	Use Lagrangian dual solution as subgradient for cut generation
Policy Heuristic	Round/fix integers in simulation based on relaxed solution

Why Deferred:

- Unit commitment is not in the immediate roadmap for medium/long-term planning
- Significant complexity in Lagrangian multiplier updates
- Cut quality can be poor without sophisticated techniques
- Alternative: solve unit commitment deterministically after SDDP provides marginal values

6. Risk-Adjusted Forward Passes

Background: Standard SDDP samples scenarios uniformly during forward passes. **Risk-adjusted forward passes** oversample scenarios from the tails of the distribution, improving exploration of worst-case outcomes for risk-averse policies.

Reference: [SDDP.jl Alternative Forward Models](#)

Configuration:

```
{
  "training": {
    "forward_pass": {
      "type": "risk_adjusted",
      "alpha": 0.2
    }
  }
}
```

Type	Description	Status
default	Uniform scenario sampling	Implemented
risk_adjusted	Oversample tail scenarios based on <code>alpha</code> parameter	DEFERRED

Why Deferred:

- Requires integration with risk measure configuration
- Performance impact needs careful benchmarking
- Default forward pass is sufficient for most applications

Extensibility Design The current data model is designed to be **extensible** for planned future features:

1. **Node IDs as tuples:** The `transitions` array supports optional `markov_state` fields for future Markovian policy graphs
2. **Cut schema extensibility:** The FlatBuffers cut schema supports additional fields (`markov_state`, `scenario_branch_idx`) for future algorithm variants
3. **Configuration extensibility:** Unknown fields in `config.json` are ignored, allowing incremental addition of new algorithm options without breaking validation
4. **Conditional validation:** Validation rules are applied conditionally based on configured modes (e.g., Markov validation only when `horizon.mode = "markovian"`)

3.2.1 Penalties and Costs

Design Rationale: The LP must always be feasible. Several physical and operational constraints may be impossible to satisfy in extreme scenarios (droughts, equipment failures, etc.). We define a **unified penalty system** with three-tier cascade resolution: 1. **Global defaults** in `penalties.json` (required) 2. **Entity overrides** inline in entity JSON files (optional) 3. **Stage overrides** in Parquet files (optional, sparse)

Global Penalty Defaults (`penalties.json`) This file defines default penalty values for all entities. It is **required** and must be present in the case directory root.

```
{
  "version": "1.1",
  "bus": {
    "deficit_segments": [
      { "depth_mw": 500, "cost": 1000.0 },
      { "depth_mw": 1000, "cost": 3000.0 },
      { "depth_mw": null, "cost": 5000.0 }
    ],
    "excess_cost": 100.0
  },
  "line": {
    "exchange_cost": 2.0
  },
  "hydro": {
    "spillage_cost": 0.01,
    "diversion_cost": 0.1,
    "turbined_violation_below_cost": 500.0,
    "outflow_violation_below_cost": 500.0,
    "outflow_violation_above_cost": 500.0,
    "generation_violation_below_cost": 1000.0,
    "evaporation_violation_cost": 5000.0,
    "water_withdrawal_violation_cost": 1000.0
  }
}
```

Deficit Segments: The last segment **MUST** have `depth_mw: null` to ensure LP feasibility (unbounded extension).

Constraint Violation Categories

Category	Constraint Type	Slack Variable	Direction
System	Load balance	deficit, excess	Lower (deficit), Upper (excess)
Hydro Generation	Min generation	generation_violation_below	Lower bound
Hydro Turbined	Min turbined flow	turbined_violation_below	Lower bound
Hydro Outflow	Min/Max outflow	outflow_violation_below, outflow_violation_above	Both bounds
Hydro Storage	Min/Max storage	Usually hard bounds, but see note	-

Note on Storage Bounds: Storage min/max are typically hard physical limits (reservoir capacity). Violations are handled by emergency spillage (if above max) or infeasibility (if below min due to bad data). We don't add slacks for storage bounds.

Note on Thermals: Thermal plants are modeled as always available within their bounds. No slack variables are needed—if a thermal cannot meet its minimum generation, it indicates a data error (bounds should be adjusted via `thermal_bounds.parquet`).

Penalty Configuration Penalties follow a three-tier cascade resolution: **global defaults** (from `penalties.json`) → **entity overrides** (inline in entity JSON files) → **stage overrides** (from Parquet files).

Key Files: | File | Purpose | | `penalties.json` | Global defaults for all penalty types (required) | | `system/buses.json` | Bus definitions with optional `deficit_segments` override | | `system/lines.json` | Line definitions with optional `exchange_cost` override | | `system/hydros.json` | Hydro definitions with optional penalty overrides | | `constraints/bus_penalties.parquet` | Stage-varying bus penalties (excess only) | | `constraints/hydro_penalties.parquet` | Stage-varying hydro penalties |

Bus Penalties Deficit (Always Piecewise): Deficit is modeled as piecewise linear segments defined in `penalties.json` or overridden per bus. The last segment **MUST** have `depth_mw: null` to ensure LP feasibility.

Excess (Single Value): Simple \$/MWh penalty for over-generation. Can vary by stage via `bus_penalties.parquet`.

Bus Penalties Schema (`constraints/bus_penalties.parquet`)

Column	Type	Nullable	Description
<code>bus_id</code>	u32	No	Bus identifier
<code>stage_id</code>	u32	No	Stage identifier
<code>excess_cost</code>	f64	Yes	\$/MWh for excess generation (null = use default)

Note: Deficit segments cannot be stage-varying (piecewise structure too complex for per-stage override).

Hydro Penalties Schema (`constraints/hydro_penalties.parquet`)

Column	Type	Nullable	Description
<code>hydro_id</code>	u32	No	Hydro identifier
<code>stage_id</code>	u32	No	Stage identifier

Column	Type	Nullable	Description
spillage_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for spilled water (operational cost)
diversion_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for diverted water (operational cost)
turbined_violation_below_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for turbined flow below min
outflow_violation_below_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for outflow below min
outflow_violation_above_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for outflow above max
generation_violation_below_cost	f64	Yes	\$/MWh for generation below min
evaporation_violation_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for evaporation constraint violation
water_withdrawal_violation_cost	f64	Yes	\$/($\text{m}^3/\text{s} \cdot \text{h}$) for unmet water withdrawal

Sparse Storage: Parquet files use sparse storage—only include rows where values differ from defaults.

Penalty Categories Penalties are divided into two categories:

Category	Penalties	Purpose	Typical Range
Operational Costs	spillage_cost, diversion_cost, exchange_cost	Discourage undesirable but feasible operations	0.001 - 10 \$/unit
Violation Penalties	deficit_*, excess_cost, *_violation_*_cost	Ensure LP feasibility, penalize constraint violations	100 - 10000 \$/unit

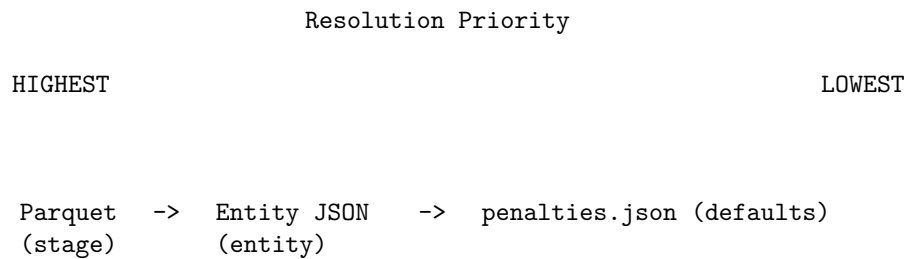
Penalty Semantics

Penalty	Category	Units	Applied To	Purpose
deficit_segments	Violation	\$/MWh	Unmet load per bus	Piecewise cost of load shedding
excess_cost	Violation	\$/MWh	Excess generation per bus	Dumping excess power
exchange_cost	Operational	\$/MWh	Power flow on lines	Discourage unnecessary exchange
spillage_cost	Operational	\$/($\text{m}^3/\text{s} \cdot \text{h}$)	Water spilled	Opportunity cost, incentivizes turbinning
diversion_cost	Operational	\$/($\text{m}^3/\text{s} \cdot \text{h}$)	Water diverted	Opportunity cost (higher than spillage)
turbined_violation_below_cost	Violation	\$/($\text{m}^3/\text{s} \cdot \text{h}$)	Turbined < min_turbined	Equipment/ecological flow
outflow_violation_below_cost	Violation	\$/($\text{m}^3/\text{s} \cdot \text{h}$)	Outflow < min_outflow	Environmental minimum flow
outflow_violation_above_cost	Violation	\$/($\text{m}^3/\text{s} \cdot \text{h}$)	Outflow > max_outflow	Downstream flooding prevention

Penalty	Category	Units	Applied To	Purpose
generation_violation	Violation cost	\$/MWh	Generation < min_generation	Contractual/environmental minimum
evaporation_violation	Violation cost	\$/ (m ³ /s · h)	Evaporation constraint	Physical constraint (bidirectional)
water_withdrawal_violation	Violation cost	\$/ (m ³ /s · h)	Unmet water withdrawal	Human consumption/irrigation

Note on Operational Costs: `spillage_cost`, `diversion_cost`, and `exchange_cost` are NOT violation penalties—they are opportunity costs. `diversion_cost` should be higher than `spillage_cost` because diverted water typically leaves the main cascade entirely. Typical values: `spillage_cost` 0.001-0.01, `diversion_cost` 0.01-0.1.

Penalty Resolution Logic The penalty system uses a three-tier cascade resolution. At runtime, the final penalty value for any entity at any stage is determined by:



Resolution Algorithm:

`resolve_penalty(entity_id, stage_id, penalty_type):`

1. Check stage override in parquet file
→ If found, return parquet value
2. Check entity override in entity JSON (e.g., `hydros.json`)
→ If found, return entity value
3. Return global default from `penalties.json`

Example Resolution:

Query	Stage Override?	Entity Override?	Result
Hydro 0, Stage 30, spillage	No	Yes (0.005)	0.005 (entity)
Hydro 0, Stage 60, spillage	Yes (0.02)	Yes (0.005)	0.02 (stage)
Hydro 1, Stage 30, spillage	No	No	0.01 (global default)

Performance Note: Parquet files should only contain rows where values differ from defaults (sparse storage). This minimizes file size and I/O.

Negative Evaporation (Condensation) Handling

Physical Background: While evaporation is typically positive (water loss from the reservoir surface), the evaporation coefficient can be negative in certain conditions:

Condition	Description
Condensation	In humid climates, water may condense on the reservoir surface
Rainfall contribution	When evaporation models include net precipitation effects
Linearization artifacts	The linear approximation of $Q_{\text{evap}} = f(\text{Volume}, \text{Coefficient})$ may produce negative values at certain volume/coefficient combinations

LP Formulation: The evaporation constraint uses **bidirectional slack variables**:

$$Q_{\text{evaporated}} - \text{evap_slack_positive} + \text{evap_slack_negative} = \text{EvapCoef} \times \text{Area}(V_{\text{avg}})$$

where:

$$\begin{aligned} \text{evap_slack_positive} &= 0 && (\text{actual evap} > \text{computed evap}) \\ \text{evap_slack_negative} &= 0 && (\text{actual evap} < \text{computed evap, including negative target}) \end{aligned}$$

Both slack variables receive the same penalty: `evaporation_violation_cost`. This ensures symmetric treatment regardless of whether the violation is upward (more water lost than expected) or downward (less water lost, or water added).

Water Balance Impact: Negative evaporation effectively adds water to the reservoir. The `Q_evaporated` variable can be negative in the water balance equation:

$$V_{\text{end}} = V_{\text{start}} + \dots \times (\dots - Q_{\text{evaporated}} \dots) \quad // \quad Q_{\text{evaporated}} < 0 \text{ means water addition}$$

Hydro Variables and Bounds Summary

Variable	Lower Bound	Upper Bound	Lower Slack	Upper Slack
storage	<code>min_storage_hm3</code>	<code>max_storage_hm3</code>	Hard	Emergency spill
turbined_flow	<code>min_turbined_m3s</code>	<code>max_turbined_m3s</code>	With penalty	Hard
spillage	0	∞	Hard	-
outflow	<code>min_outflow_m3s</code>	<code>max_outflow_m3s</code>	With penalty	With penalty
generation	Derived from turbined	Derived from turbined	With penalty	Hard
evaporation	$-\infty$ (can be negative)	$+\infty$	With penalty	With penalty

Relationship: `outflow = turbined_flow + spillage`, `generation = productivity × turbined_flow`

Dead-Volume Filling Specifics During the filling period:

- **No turbined flow:** `turbined_flow = 0` (hard constraint—turbines not installed/operational)
- **Outflow = spillage:** All released water goes through non-turbine outlets (spillways, bottom gates, etc.)
- **Min outflow requirement:** Environmental flow must be met via spillage
- If `inflow - filling_retention < min_outflow`, the `outflow_violation_below` slack absorbs the shortfall

The same `outflow_violation_cost` from `hydro_penalties.parquet` applies during filling. Spillage during filling also incurs `spillage_cost`, representing the lost potential of water that could have been stored.

LP Objective Function Impact For each stage t , block b , scenario s :

minimize:

```
// Operational costs
+  $\Sigma_{\text{thermal}}$  (generation  $\times$  cost_per_mwh)
+  $\Sigma_{\text{hydro}}$  (spillage  $\times$  spillage_cost)
+  $\Sigma_{\text{hydro}}$  (diversion  $\times$  diversion_cost)
+  $\Sigma_{\text{line}}$  (|exchange|  $\times$  exchange_cost)
+  $\Sigma_{\text{contract}}$  (import  $\times$  import_price - export  $\times$  export_price)
+  $\Sigma_{\text{pumping\_station}}$  (pumped_flow  $\times$  pumping_cost) // if applicable

// Violation penalties
+  $\Sigma_{\text{bus}} \Sigma_{\text{segment}}$  (deficit_segment  $\times$  segment_cost) // Piecewise deficit
+  $\Sigma_{\text{bus}}$  (excess  $\times$  excess_cost)
+  $\Sigma_{\text{hydro}}$  (turbined_violation_below  $\times$  turbined_violation_below_cost)
+  $\Sigma_{\text{hydro}}$  (outflow_violation_below  $\times$  outflow_violation_below_cost)
+  $\Sigma_{\text{hydro}}$  (outflow_violation_above  $\times$  outflow_violation_above_cost)
+  $\Sigma_{\text{hydro}}$  (generation_violation_below  $\times$  generation_violation_below_cost)
+  $\Sigma_{\text{hydro}}$  (evaporation_violation_positive  $\times$  evaporation_violation_cost)
+  $\Sigma_{\text{hydro}}$  (evaporation_violation_negative  $\times$  evaporation_violation_cost)
+  $\Sigma_{\text{hydro}}$  (water_withdrawal_violation  $\times$  water_withdrawal_violation_cost)

// Future cost function
+ // Cut approximation (future cost)
```

Hydro Water Balance Equation The complete hydro balance equation considering all features:

```
V_end = V_start +  $\Delta t$  * (
  + Q_inflow // Natural inflow (stochastic)
  +  $\Sigma_{\text{upstream}}$  (Q_turbined + Q_spillage + Q_diversion) // From upstream plants
  +  $\Sigma_{\text{pumping\_in}}$  (Q_pumped) // From pumping stations targeting this plant
  - Q_turbined // Turbined water (generates power)
  - Q_spillage // Spilled water (to downstream)
  - Q_diversion // Diverted water (to diversion downstream)
  - Q_evaporated // Evaporated water (can be negative for condensation)
  - Q_withdrawal // Water withdrawal (human/irrigation use)
  -  $\Sigma_{\text{pumping\_out}}$  (Q_pumped) // To pumping stations sourcing from this plant
)
```

where Δt is the time conversion factor ($\text{m}^3/\text{s} \rightarrow \text{hm}^3$)

3.3 Buses (system/buses.json)

Order Invariance: The order of buses in this array does NOT affect results. After loading, all buses are sorted by id. See Section 1.3.

Deficit Modeling: Deficit is modeled as piecewise linear segments. Each segment specifies a depth (MW of unmet demand) and cost. Segments are cumulative: first `depth_mw` MW at first cost, next `depth_mw` MW at second cost, etc. The last segment with `depth_mw`: null extends to infinity (required for LP feasibility). Global defaults are defined in `penalties.json`; entity-level overrides are defined inline here.

```
{
  "buses": [
    {
      "id": 0,
      "name": "SUDESTE",
      "deficit_segments": [
        {"depth_mw": 1000, "cost": 2000.0},
        {"depth_mw": 2000, "cost": 5000.0},

```

```

    {"depth_mw": null, "cost": 10000.0}
  ],
  {
    "id": 1,
    "name": "SUL"
  }
]
}

```

Note on Bus 1 (SUL): No `deficit_segments` defined—uses global default from `penalties.json`.

Bus Fields

Field	Type	Required	Description
<code>id</code>	i32	Yes	Unique bus identifier
<code>name</code>	string	Yes	Human-readable bus name
<code>deficit_segments</code>	array	No	Piecewise deficit cost segments (uses default if omitted)

Deficit Segment Fields

Field	Type	Required	Description
<code>depth_mw</code>	f64 null	Yes	MW of deficit at this segment (<code>null</code> for final infinite segment)
<code>cost</code>	f64	Yes	Cost per MWh of deficit in this segment

3.4 Lines (`system/lines.json`)

Order Invariance: The order of lines in this array does NOT affect results. After loading, all lines are sorted by `id`. See Section 1.3.

Exchange Cost: The `exchange_cost` field (formerly `exchange_penalty`) is an operational cost, NOT a violation penalty. It discourages unnecessary power flow between buses. Default is defined in `penalties.json`.

Line Operative State: Each line has an operative state per stage:

State	When	Behavior
<code>non_existing</code>	Before <code>entry_stage_id</code>	No exchange variables, buses isolated
<code>operating</code>	Between <code>entry_stage_id</code> and <code>exit_stage_id</code>	Normal exchange
<code>decommissioned</code>	After <code>exit_stage_id</code>	No exchange variables

```

{
  "lines": [
    {
      "id": 0,
      "name": "SE-NE",
      "source_bus_id": 0,
      "target_bus_id": 1,
      "entry_stage_id": null,

```

```

    "exit_stage_id": null,
    "capacity": {
      "direct_mw": 5000.0,
      "reverse_mw": 3000.0
    },
    "exchange_cost": 0.01,
    "losses_percent": 2.5
  }
]
}

```

Line Fields

Field	Type	Required	Description
id	i32	Yes	Unique line identifier
name	string	Yes	Human-readable line name
source_bus_id	i32	Yes	Source bus for direct flow
target_bus_id	i32	Yes	Target bus for direct flow
entry_stage_id	i32 null	No	Stage when line enters service (null = always exists)
exit_stage_id	i32 null	No	Stage when line is decommissioned (null = never decommissioned)
capacity.direct_mw	f64	Yes	Maximum flow from source to target (MW)
capacity.reverse_mw	f64	Yes	Maximum flow from target to source (MW)
exchange_cost	f64	No	Cost per MWh exchanged (uses default if omitted)
losses_percent	f64	No	Transmission losses as percentage (default: 0)

Line Operative States

State	Condition	LP Variables
non_existing	Before <code>entry_stage_id</code>	None (buses isolated)
operating	Between <code>entry_stage_id</code> and <code>exit_stage_id</code>	<code>direct_flow</code> , <code>reverse_flow</code>
decommissioned	After <code>exit_stage_id</code>	None (buses isolated)

3.5 Hydro Registry (system/hydros.json)

Order Invariance: The order of hydros in this array does NOT affect results. After loading, hydros are sorted by id. See Section 1.3.

Note: The `generation` field supports multiple modeling approaches for the hydro production function. The choice of model affects LP complexity and accuracy. Different models can be used for different stages via `hydro_production_models.json`. See Sections 3.5.2 and 3.5.3 for detailed production function documentation.

Inflow models are defined per hydro $\times$ stage in `scenarios/inflow_models.parquet`, linked by `hydro_id`.

Operative State: Each hydro has an operative state per stage, determined by `entry_stage_id`, `exit_stage_id`, and `filling.start_stage_id`:

State	When	Behavior
non_existing	Before filling.start_stage_id (or entry_stage_id if no filling)	No variables in LP
filling	Between filling.start_stage_id and entry_stage_id - 1	Reservoir fills, no generation
operating	Between entry_stage_id and exit_stage_id	Normal operation
decommissioned	After exit_stage_id	No variables in LP

Dead-volume filling: During filling stages, the reservoir accumulates water according to constraints. The filling_inflow_m3s is water retained for filling; the remainder ($\text{inflow} - \text{filling_inflow}$) must be released as outflow. Outflow must meet min_outflow_m3s. The modeling during filling is:

Aspect	Constraint
turbined_flow	= 0 (hard constraint, turbines not installed/operational)
outflow	= spillage (all released water goes through bottom outlets)
hydro_balance	$\text{storage_end} = \text{storage_start} + (\text{inflow} - \text{filling_inflow} - \text{outflow}) \times \text{time_factor}$
filling_inflow_m3s	Target filling rate, but if $\text{inflow} - \text{min_outflow} < \text{filling_inflow}$, less water is retained
Slack variables	Handle infeasible scenarios (see penalty files in constraints/ directory)

Outflow: $\text{Outflow} = \text{turbined_flow} + \text{spillage} + \text{diversion}$. Outflow has explicit bounds (min_outflow_m3s, max_outflow_m3s) that can vary per stage via hydro_bounds.parquet.

Generation: The relationship between turbined flow and generation depends on the production function model. For constant_productivity: $g^{\text{hy}} = \times q$. For fpha: $g^{\text{hy}}(v, q, s)$ as a set of linear constraints. Generation can have explicit bounds (min_generation_mw, max_generation_mw) for contractual or operational reasons.

Penalties: Penalty defaults are defined in penalties.json. Entity-level overrides can be specified in an optional penalties block in the hydro definition. Stage-varying overrides are defined in hydro_penalties.parquet. The hydro config defines physical bounds, not penalty values.

Penalty Override Example:

```
{
  "id": 42,
  "name": "SPECIAL_HYDRO",
  "penalties": {
    "spillage_cost": 0.005,
    "turbined_violation_below_cost": 800.0
  },
  ...
}
```

Cascade redirection: The downstream_id always refers to the physical downstream plant. During stages when the downstream plant doesn't exist (non_existing or filling), outflows are automatically redirected to the next operating downstream in the cascade.

Diversion Channel (Canal de Desvio): Some hydro plants have diversion channels that redirect water to a different downstream than the main cascade. The diverted flow goes directly to

diversion_downstream_id without generating power. Unlike models that use Big-M or indicator constraints with storage thresholds, POWERS models diversion as a continuous flow variable bounded by [0, max_flow_m3s] with an associated penalty cost. This approach avoids numerical issues from Big-M constraints and allows the optimizer to find economically optimal diversion flows. The `diversion` field is optional.

```
{
  "hydros": [
    {
      "id": 0,
      "name": "FURNAS",
      "bus_id": 0,
      "downstream_id": 2,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "filling": null,
      "diversion": null,
      "reservoir": {
        "min_storage_hm3": 5733.0,
        "max_storage_hm3": 22950.0
      },
      "outflow": {
        "min_outflow_m3s": 0.0,
        "max_outflow_m3s": null
      },
      "generation": {
        "model": "constant_productivity",
        "productivity_mw_per_m3s": 0.8765,
        "min_turbined_m3s": 0.0,
        "max_turbined_m3s": 1692.0,
        "min_generation_mw": null,
        "max_generation_mw": null
      }
    },
    {
      "id": 5,
      "name": "HYDRO_WITH_DIVERSION",
      "bus_id": 0,
      "downstream_id": 6,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "filling": null,
      "diversion": {
        "downstream_id": 10,
        "max_flow_m3s": 500.0
      },
      "reservoir": {
        "min_storage_hm3": 3000.0,
        "max_storage_hm3": 12000.0
      },
      "outflow": {
        "min_outflow_m3s": 100.0,
        "max_outflow_m3s": null
      },
      "generation": {
        "model": "constant_productivity",
        "productivity_mw_per_m3s": 0.75,

```

```

        "min_turbined_m3s": 0.0,
        "max_turbined_m3s": 800.0,
        "min_generation_mw": null,
        "max_generation_mw": null
    }
},
{
    "id": 10,
    "name": "NEW_HYDRO",
    "bus_id": 0,
    "downstream_id": 0,
    "entry_stage_id": 60,
    "exit_stage_id": null,
    "filling": {
        "start_stage_id": 48,
        "target_storage_hm3": 2500.0
    },
    "diversion": null,
    "reservoir": {
        "min_storage_hm3": 1000.0,
        "max_storage_hm3": 5000.0
    },
    "outflow": {
        "min_outflow_m3s": 50.0,
        "max_outflow_m3s": null
    },
    "generation": {
        "model": "constant_productivity",
        "productivity_mw_per_m3s": 0.95,
        "min_turbined_m3s": 0.0,
        "max_turbined_m3s": 500.0,
        "min_generation_mw": null,
        "max_generation_mw": null
    }
}
]
}

```

Diversion Channel Fields

Field	Type	Description
<code>diversion.downstream_id</code>	i32	Hydro plant receiving diverted water
<code>diversion.max_flow_m3s</code>	f64	Maximum diversion flow

LP Modeling: Diversion creates an additional flow variable `diversion_flow` that:

Property	Behavior
Bounded by	<code>[0, max_flow_m3s]</code>
Balance effect	Subtracted from the plant's balance and added to <code>diversion_downstream_id</code> 's inflow
Power generation	Does NOT generate power (similar to spillage)
Cost	Has an associated <code>diversion_cost</code> from <code>hydro_penalties.parquet</code> (incentive to avoid diversion unless necessary)

Note: Unlike DECOMP which uses threshold-based Big-M constraints for diversion, POWERS uses a penalty-based approach. The `diversion_cost` should be set to reflect the opportunity cost of diverting water (typically higher than spillage cost since diverted water leaves the main cascade). This approach is simpler, avoids numerical issues with Big-M constants, and allows the optimizer to make economically optimal decisions.

Hydro Operative States

State	Condition	LP Variables
<code>non_existing</code>	Before filling or entry (no filling defined)	None
<code>filling</code>	Between <code>filling.start_stage_id</code> and <code>entry_stage_id - 1</code>	storage, outflow (=spillage), outflow_violation_below, evaporation
<code>operating</code>	Between <code>entry_stage_id</code> and <code>exit_stage_id</code>	storage, turbined, spillage, diversion, outflow, generation, evaporation, violation slacks
<code>decommissioned</code>	After <code>exit_stage_id</code>	None

Hydro LP Variables by State

Variable	<code>non_existing</code>	<code>filling</code>	<code>operating</code>	<code>decommissioned</code>
<code>storage</code>				
<code>turbined_flow</code>		(=0)		
<code>spillage</code>				
<code>diversion_flow</code>			(if configured)	
<code>outflow</code>		(=spillage)		
<code>generation</code>		(=0)		
<code>evaporation</code>		(simplified ¹)	(if configured)	
<code>turbined_violation_below</code>				
<code>outflow_violation_below</code>				
<code>outflow_violation_above</code>				
<code>generation_violation_below</code>				
<code>water_withdrawal_violation</code>		(if configured)	(if configured)	
<code>evaporation_violation</code>		(if configured)	(if configured)	

¹ **Evaporation during filling:** During the filling state, the reservoir operates in the dead volume region where geometry data may not be available. If geometry data exists below `min_storage_hm3`, it is used; otherwise, evaporation coefficients are computed using the geometry at `min_storage_hm3`. This is a conservative simplification since smaller volumes have proportionally smaller surface areas.

3.5.1 Hydro Geometry (`system/hydro_geometry.parquet`) - Optional

Purpose: Defines the Volume-Height-Area relationship for reservoirs, enabling accurate evaporation calculation. Instead of complex polynomials, we use a tabular approach with linear interpolation for simplicity and transparency.

Table Contents: Each row specifies a point on the geometry curve: (`volume`, `height`, `area`). Given any storage value `v`, the corresponding area `A(v)` is obtained by linear interpolation between adjacent points. The height `h(v)` is similarly interpolated but used primarily for FPHA production function calculations.

Evaporation Calculation: The evaporated flow depends on the reservoir surface area and the evaporation coefficient:

$$Q_{\text{evap}}(V) = \text{evap_coef_mm} \times A(V) \times \text{conversion_factor}$$

where $\text{conversion_factor} = 1\text{e-}3 / (86400 \times \text{days_in_stage})$ converts mm to m³/s.

Linear Approximation in LP: Since $A(V)$ is a nonlinear function of volume, we use a first-order Taylor approximation around a reference volume V_{ref} :

$$Q_{\text{evap}} \approx k_{\text{evap_0}} + k_{\text{evap_V}} \times V_{\text{avg}}$$

Term	Definition
V_{avg}	$(V_{\text{start}} + V_{\text{end}}) / 2$ is the average storage over the stage
$k_{\text{evap_V}}$	$\text{evap_coef} \times dA/dV$ is the slope (computed from the geometry table at V_{ref})
$k_{\text{evap_0}}$	$\text{evap_coef} \times (A(V_{\text{ref}}) - dA/dV \times V_{\text{ref}})$ is the intercept

The coefficients are recomputed per stage as the reference volume changes based on the previous stage's solution.

Filling State Evaporation: During the filling state (before `entry_stage_id`), the reservoir may operate below `min_storage_hm3`. Since geometry data is only validated between `min_storage_hm3` and `max_storage_hm3`, evaporation during filling uses the geometry at `min_storage_hm3` as a simplification. This is conservative since smaller volumes have smaller areas.

Column	Type	Description
<code>hydro_id</code>	i32	Hydro plant identifier
<code>volume_hm3</code>	f64	Total volume (hm ³) - must include dead volume
<code>height_m</code>	f64	Reservoir surface elevation (m)
<code>area_km2</code>	f64	Water surface area (km ²)

Example rows (for Sobradinho): | hydro_id | volume_hm3 | height_m | area_km2 | |-----|-----|-----|
-|-----| | 42 | 5447.0 | 380.0 | 800.0 | | 42 | 8000.0 | 385.0 | 1200.0 | | 42 | 12500.0 | 390.0 | 2000.0 | | 42 | 18000.0 |
395.0 | 3000.0 | | 42 | 28000.0 | 400.0 | 4200.0 |

Validation:

Rule	Description
Volumes	Must be monotonically increasing per hydro
Heights	Must be monotonically increasing with volume
Areas	Must be monotonically increasing with height
Minimum volume entry	Should be at or below <code>min_storage_hm3</code>
Maximum volume entry	Should be at or above <code>max_storage_hm3</code>
Dead volume geometry	Geometry data below <code>min_storage_hm3</code> is optional; if not provided, evaporation during filling uses the geometry at <code>min_storage_hm3</code>

3.5.2 Hydro Production Models (`system/hydro_production_models.json`) - Optional

Purpose: Configures the hydro production function (HPF) modeling approach per stage range. Different stages can use different accuracy levels—detailed FPHA for near-term stages where precision matters, simplified constant productivity for far-future stages where computational efficiency is preferred.

Background: The hydro production function relates turbined flow to generation:

$$g^{\text{hy}} = (q, v, h_{\text{net}}) = (q, h_{\text{net}}) \times q \times h_{\text{net}}$$

where γ is the specific productivity, q is turbined flow, and h_{net} is the net head (upstream level minus downstream level minus hydraulic losses). This relationship is nonlinear, requiring approximation for LP formulation.

Model Hierarchy (in order of increasing complexity and accuracy): 1. **constant_productivity**: $\gamma^{\text{hy}} = \gamma \times q$ (single multiplication, fastest) 2. **linearized_head**: $\gamma^{\text{hy}} = \gamma \times q \times (k_0 + k_v \times v_{\text{avg}})$ (accounts for head variation with storage) 3. **fpha**: $\gamma^{\text{hy}} = \gamma(v, q, s)$ (full piecewise-linear approximation with spillage effects)

Stage-Dependent Configuration: Users can configure different models for different stage ranges:

Stage Range	Recommended Model	Rationale
Near-term (e.g., 1-24)	FPFA	Accurate representation
Medium-term (e.g., 25-60)	Linearized head	Balance accuracy/speed
Long-term (e.g., 61+)	Constant productivity	Computational efficiency

Default Behavior: If this file is not provided or a hydro is not listed, the model uses the `generation.model` field from `hydros.json` for all stages.

```
{
  "production_models": [
    {
      "hydro_id": 0,
      "stage_ranges": [
        {
          "start_stage_id": 0,
          "end_stage_id": 24,
          "model": "fpha",
          "fpha_config": {
            "source": "computed",
            "volume_discretization_points": 7,
            "turbine_discretization_points": 15,
            "fitting_window": {
              "volume_min_hm3": null,
              "volume_max_hm3": null
            }
          }
        }
      ],
    },
    {
      "start_stage_id": 25,
      "end_stage_id": 60,
      "model": "fpha",
      "fpha_config": {
        "source": "computed",
        "volume_discretization_points": 5,
        "turbine_discretization_points": 10,
        "fitting_window": {
          "volume_min_percentile": 10,
          "volume_max_percentile": 90
        }
      }
    },
    {
      "start_stage_id": 61,
      "end_stage_id": null,
      "model": "constant_productivity"
    }
  ]
}
```

```

    ]
  },
  {
    "hydro_id": 5,
    "stage_ranges": [
      {
        "start_stage_id": 0,
        "end_stage_id": null,
        "model": "fpha",
        "fpha_config": {
          "source": "precomputed"
        }
      }
    ]
  }
]
}

```

Production Model Types

Model	LP Complexity	Accuracy	Use Case
constant_productivity	1 constraint: $g^{hy} = \rho \times q$	Low	Long-term stages, run-of-river plants, quick studies
linearized_head	1 constraint: $g^{hy} = \rho \times q \times h_{\text{linear}}(v)$	Medium	Medium-term stages, reservoirs with moderate head variation
fpha	M constraints: $g^{hy} = \rho_0^m + \rho_v^m \times v + \rho_q^m \times q + \rho_s^m \times s$	High	Near-term stages, reservoirs with significant head variation

Constant Productivity Model The simplest approach assumes constant efficiency and head:

$$g^{hy} = \rho \times q$$

- **Parameters:** `productivity_mw_per_m3s` (from `hydros.json`)
- **LP Variables:** `generation`, `turbined_flow`
- **Constraints:** 1 equality per hydro $\times$ block
- **Pros:** Fast, minimal LP impact, sufficient for run-of-river or far-future planning
- **Cons:** Ignores head variation, may over/underestimate generation

Linearized Head Model Accounts for head variation with storage using a linear approximation:

$$g^{hy} = \rho \times q \times (k_0 + k_v \times v_{avg})$$

where k_0 and k_v are derived from the geometry table at a reference volume.

- **Parameters:** Geometry table, `productivity_mw_per_m3s`
- **LP Variables:** `generation`, `turbined_flow`, `storage`
- **Constraints:** 1 bilinear-approximated constraint (linearized around operating point)
- **Pros:** Better accuracy for reservoirs with head variation
- **Cons:** Still an approximation, doesn't capture spillage effects

FPHA Model (Função de Produção Hidrelétrica Aproximada) Full piecewise-linear approximation following CEPEL methodology:

$$g^{hy} \leq \gamma_0^m + \gamma_v^m \times v_{avg} + \gamma_q^m \times q + \gamma_s^m \times s, \quad \forall m \in \{1, \dots, M\}$$

where M is the number of hyperplanes forming the convex hull approximation.

Construction Algorithm (performed during preprocessing):

1. **Discretize operating window:** Create grid of (v, q) points within $[v_min, v_max] \times [0, q_max]$
2. **Compute exact generation:** For each point, calculate $\hat{g}^{hy}$ using full nonlinear
3. **Build convex hull:** Apply qhull algorithm to find the concave envelope
4. **Apply regression factor:** Minimize squared error between FPFA and
5. **Add spillage secant:** Extend to (v, q, s) space for downstream level effects

Configuration Fields:

Field	Type	Description
source	string	“computed” (fit from topology) or “precomputed” (from fpha_hyperplanes.parquet)
volume_discretization_points	int32	Number of volume points in grid (default: 5)
turbine_discretization_points	int32	Number of turbine flow points (default: 10)
fitting_window.volume_min_hm3	float64?	Explicit minimum volume for fitting (null = use physical min)
fitting_window.volume_max_hm3	float64?	Explicit maximum volume for fitting (null = use physical max)
fitting_window.volume_min_percentile	int32	Minimum as percentile of operating range (e.g., 10 = 10th percentile)
fitting_window.volume_max_percentile	int32	Maximum as percentile of operating range (e.g., 90 = 90th percentile)

Note on fitting_window: Use absolute bounds (volume_min_hm3, volume_max_hm3) OR percentiles, not both. Percentiles are relative to the physical operating range [min_storage_hm3, max_storage_hm3] from hydros.json.

Stage-dependent strategy: Use wider fitting windows and higher resolution for near-term stages (where accuracy matters), narrower windows and lower resolution for far-future stages (where computational efficiency matters). See MATHEMATICAL_FORMULATIONS.md Section 6.2.9.

- **LP Variables:** generation, turbined_flow, storage, spillage
- **Constraints:** M inequalities per hydro × block (typically 5-30 planes)
- **Pros:** Most accurate, captures head variation and spillage effects, matches DECOMP/DESSEM
- **Cons:** More constraints, requires geometry data, computational overhead

Implementation Note: When FPFA is used, the generation variable becomes independent (not directly derived from turbined flow). The FPFA constraints ensure generation stays below the feasible region. The optimizer naturally pushes generation to “touch” the FPFA surface because higher generation is always preferred.

Required Data by Model

Model	hydros.json	hydro_geometry.parquet	hydro_production_data.parquet	fpha_hyperplanes.parquet
constantproduction	required	required	required	not required
linearizedhead	required	required	required	not required

Model	hydros.json	hydro_geometry.parquet	hydro_production_data.parquet	fpha_hyperplanes.parquet
fpha (com- puted)	productivity_mw_per_m3s ¹		(for tailrace/losses)	
fpha (pre- com- puted)	productivity_mw_per_m3s ¹			

¹ Used as fallback for stages without FPHA configuration

Transition Between Models When a hydro transitions from FPHA to simpler models across stages:

- **Dual variables:** The water value computation must account for model changes
- **Cut coefficients:** SDDP cuts use the appropriate model for each stage
- **Validation:** Generation bounds are enforced regardless of model

Recommendation: For production studies, use FPHA for at least the first 12-24 stages (one to two years), then transition to simpler models. This balances accuracy in the planning horizon with computational efficiency for long-term expectations.

3.5.3 Hydro Production Data (system/hydro_production_data.parquet) - Optional

Purpose: Provides additional data for detailed production function modeling: tailrace (canal de fuga) polynomials, hydraulic losses, and efficiency curves.

Column	Type	Description
hydro_id	i32	Hydro plant identifier
tailrace_type	str	“polynomial” or “piecewise”
tailrace_coeffs	[f64]	Polynomial coefficients for $h_{jus}(Q_{jus})$
hydraulic_loss_type	str	“factor” (p.u.) or “constant” (m)
hydraulic_loss_value	f64	Loss factor or constant head loss
efficiency_type	str	“constant”, “flow_dependent”, or “grid”
efficiency_value	f64	Constant efficiency (if type = “constant”)

Note: For fpha model, if this data is not provided, the system uses simplified assumptions:

Assumption	Fallback Value
Tailrace	Constant downstream level from hydro_geometry.parquet lowest point
Hydraulic losses	Zero losses
Efficiency	Constant from productivity_mw_per_m3s

3.5.4 FPHA Hyperplanes (system/fpha_hyperplanes.parquet) - Optional

Purpose: Pre-computed FPHA (Four-Point Head Approximation) hyperplane coefficients for hydro production function modeling. This file allows using externally-fitted FPHA planes instead of computing them at runtime from topology data.

Use Cases:

Use Case	Description
Legacy system migration	Import FPHA coefficients from DECOMP/DESSEM input files
External calibration	Use coefficients fitted by specialized tools with plant-specific validation
Performance optimization	Skip runtime fitting for large systems

Alternative: If this file is not provided, POWE.RS computes FPHA hyperplanes from `hydro_geometry.parquet` and `hydro_production_data.parquet` during preprocessing. See MATHEMATICAL_FORMULATIONS.md Section 6.2.4 for the fitting algorithm.

Relationship with `hydro_production_models.json`: The `fpha_config.source` field controls whether to use pre-computed planes (“precomputed”) or fit them at runtime (“computed”). When “precomputed” is specified but planes are missing for a hydro, the system falls back to runtime computation with a warning.

Column	Type	Required	Description
<code>hydro_id</code>	i32	Yes	Hydro plant identifier
<code>plane_id</code>	i32	Yes	Plane index within hydro (0 to M-1)
<code>gamma_0</code>	f64	Yes	Intercept coefficient (MW)
<code>gamma_v</code>	f64	Yes	Volume coefficient (MW/hm ³)
<code>gamma_q</code>	f64	Yes	Turbined flow coefficient (MW per m ³ /s)
<code>gamma_s</code>	f64	Yes	Spillage coefficient (MW per m ³ /s, typically 0)
<code>alpha_fpha</code>	f64	No	Correction factor (default: 1.0, already applied to <code>gamma_0</code>)
<code>valid_v_min_hm3</code>	f64	No	Volume range minimum where plane is valid
<code>valid_v_max_hm3</code>	f64	No	Volume range maximum where plane is valid
<code>valid_q_max_m3s</code>	f64	No	Maximum turbined flow where plane is valid

Example rows (for Itaipu): | `hydro_id` | `plane_id` | `gamma_0` | `gamma_v` | `gamma_q` | `gamma_s` | `alpha_fpha`
| |-----| |-----| |-----| |-----| |-----| |-----| | 66 | 0 | 1250.5 | 0.0023 | 0.892 | -0.015 | 0.985 | | 66 | 1 | 1180.2 | 0.0031 | 0.875 | -0.012 | 0.985 | | 66 | 2 | 1320.8 | 0.0018 | 0.901 | -0.018 | 0.985 | | 66 | 3 | 1095.4 | 0.0042 | 0.858 | -0.010 | 0.985 | | 66 | 4 | 1410.1 | 0.0012 | 0.915 | -0.022 | 0.985 |

Constraint Form: Each row defines a constraint:

$$\text{GH} \quad \alpha_{\text{fpha}} \times (\gamma_0 + \gamma_v \times V_{\text{avg}} + \gamma_q \times Q + \gamma_s \times S)$$

If `alpha_fpha` is provided in this table, it is assumed to be **not yet applied** to `gamma_0`. The solver will multiply `gamma_0` $\times$ `alpha_fpha` when building constraints.

Alternatively, if `alpha_fpha` = 1.0 (or null), it is assumed `gamma_0` already includes any correction factor.

Validation:

- Each `hydro_id` should have at least 3 planes (minimum for 3D approximation)
- Typical range: 5-30 planes per hydro
- `gamma_v` should be positive (higher storage $\rightarrow$ higher generation)
- `gamma_q` should be positive (more flow $\rightarrow$ more generation)
- `gamma_s` should be negative or zero (spillage reduces effective head)
- If validity ranges are provided, planes are only activated when the hydro is within that range

3.5.5 Pumping Stations (system/pumping_stations.json) - Optional

Purpose: Models pumped storage and water transfer stations (elevatórias) that pump water from a downstream reservoir to an upstream reservoir, consuming electric power.

Applications:

Application	Description
Pumped hydro storage	Store energy by pumping water to upper reservoir during low-demand periods
Inter-basin transfers	Move water between river basins for irrigation or energy optimization
Reversible hydro plants	Plants that can both generate and pump (model as hydro + pumping station pair)

LP Variables: pumped_flow (m³/s), pumping_power_consumption (MW)

Constraints:

Constraint	Formula
Power consumption	$\text{pumping_power_consumption} = \text{pumped_flow} \times \text{consumption_rate}$
Power effect	Pumping power is added to bus load (demand side)
Water balance	Pumped flow is added to destination hydro inflow, subtracted from source hydro balance

```
{
  "pumping_stations": [
    {
      "id": 0,
      "name": "SANTA_CECILIA",
      "bus_id": 0,
      "source_hydro_id": 5,
      "destination_hydro_id": 10,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "consumption_mw_per_m3s": 0.85,
      "flow": {
        "min_m3s": 0.0,
        "max_m3s": 150.0
      }
    }
  ]
}
```

Field	Type	Description
id	i32	Unique station identifier
name	string	Station name
bus_id	i32	Bus where power is consumed
source_hydro_id	i32	Downstream hydro (water origin)
destination_hydro_id	i32	Upstream hydro (water destination)

Field	Type	Description
entry_stage_id	i32?	First operating stage (null = always)
exit_stage_id	i32?	Last operating stage (null = forever)
consumption_mw_per_m3s	f64	Power consumption rate
flow.min_m3s	f64	Minimum pumped flow
flow.max_m3s	f64	Maximum pumped flow

3.5.6 Energy Contracts (system/energy_contracts.json) - Optional

Purpose: Models energy import/export contracts with external systems (e.g., neighboring countries, bilateral contracts). These are external energy sources or sinks with associated prices and quantity limits.

LP Variables: `contract_import` or `contract_export` (MW per block)

Constraints: Energy contracts participate in bus load balance. Import adds to supply, export adds to demand.

```
{
  "contracts": [
    {
      "id": 0,
      "name": "ITAIPU_BR",
      "bus_id": 0,
      "type": "import",
      "entry_stage_id": null,
      "exit_stage_id": null,
      "price_per_mwh": 50.0,
      "limits": {
        "min_mw": 0.0,
        "max_mw": 6000.0
      }
    },
    {
      "id": 1,
      "name": "ARGENTINA_EXPORT",
      "bus_id": 0,
      "type": "export",
      "entry_stage_id": null,
      "exit_stage_id": null,
      "price_per_mwh": -30.0,
      "limits": {
        "min_mw": 0.0,
        "max_mw": 2000.0
      }
    }
  ]
}
```

Field	Type	Description
id	i32	Unique contract identifier
name	string	Contract name
bus_id	i32	Bus connected to contract
type	string	"import" (external→system) or "export" (system→external)

Field	Type	Description
entry_stage_id	i32?	First active stage (null = always)
exit_stage_id	i32?	Last active stage (null = forever)
price_per_mwh	f64	Cost (import) or revenue (export, typically negative)
limits.min_mw	f64	Minimum contract usage
limits.max_mw	f64	Maximum contract usage

Contract Bounds (constraints/contract_bounds.parquet) - Optional

Column	Type	Description
contract_id	i32	Contract identifier
stage_id	i32	Stage index
min_mw	f64	Minimum usage (null = use base)
max_mw	f64	Maximum usage (null = use base)
price_per_mwh	f64	Price override (null = use base)

3.5.7 Non-Controllable Generation Sources (system/non_controllable_sources.json) - DEFERRED

Implementation Status: This feature is designed but **deferred for future implementation**. The data model is specified here to guide future development.

Purpose: Models renewable/intermittent generation sources such as wind farms and solar plants. These sources are characterized by:

Characteristic	Description
Stochastic generation	Output depends on weather conditions (wind speed, solar irradiation)
Non-controllable	Unlike hydros/thermals, output cannot be dispatched up (only curtailed)
Potential correlation	May be correlated with inflows (e.g., wet seasons with lower solar, wind patterns affecting hydrology)

Naming Convention: Sources are named generically (not “wind” or “solar”) to allow flexibility. Common names include "WIND_FARM_NE", "SOLAR_BAHIA", etc.

```
{
  "non_controllable_sources": [
    {
      "id": 0,
      "name": "WIND_NE_1",
      "source_type": "wind",
      "bus_id": 1,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "capacity_mw": 500.0,
      "curtailment": {
        "allowed": true,
        "penalty_per_mwh": 50.0
      }
    }
  ],
}
```

```

{
  "id": 1,
  "name": "SOLAR_BAHIA",
  "source_type": "solar",
  "bus_id": 1,
  "entry_stage_id": 12,
  "exit_stage_id": null,
  "capacity_mw": 200.0,
  "curtailment": {
    "allowed": true,
    "penalty_per_mwh": 30.0
  }
}
]
}

```

Field	Type	Description
id	i32	Unique source identifier
name	string	Source name (user-defined)
source_type	string	Informational type: "wind", "solar", "other"
bus_id	i32	Bus where generation is injected
entry_stage_id	i32?	First operating stage (null = always)
exit_stage_id	i32?	Last operating stage (null = forever)
capacity_mw	f64	Installed capacity (maximum possible generation)
curtailment.allowed	bool	Whether curtailment (spilling generation) is allowed
curtailment.penalty_per_mwh	f64	Penalty for curtailed energy (if allowed)

Non-Controllable Generation Model

Stochastic Process: Similar to load models, non-controllable generation is modeled with mean and standard deviation per source per stage. The generation can optionally participate in the same correlation structure as inflows.

Generation Models Schema (scenarios/non_controllable_models.parquet) - DEFERRED

Column	Type	Description
source_id	i32	Non-controllable source ID
stage_id	i32	Stage ID
mean_mw	f64	Mean generation for this stage
std_mw	f64	Standard deviation (0 = deterministic)

Block Factors: Like load, non-controllable generation can have block-specific factors in a separate file (scenarios/non_controllable_factors.json) following the same structure as load_factors.json.

LP Integration For a **non-controllable** source (controllable = false):

- Generation is fixed to the stochastic realization: $gen = G_{realized}$
- If curtailment.allowed = true: $gen = G_{realized} - curtailment$, with curtailment penalty

Correlation with Inflows Non-controllable sources can be included in correlation.json blocks:

```

{
  "blocks": [

```

```

{
  "name": "ne_hydro_wind_correlation",
  "entities": [
    {"type": "inflow", "id": 50},
    {"type": "inflow", "id": 51},
    {"type": "non_controllable", "id": 0},
    {"type": "non_controllable", "id": 1}
  ],
  "matrix": [
    [1.0, 0.9, -0.3, -0.2],
    [0.9, 1.0, -0.25, -0.15],
    [-0.3, -0.25, 1.0, 0.8],
    [-0.2, -0.15, 0.8, 1.0]
  ]
}
]
}

```

Interpretation: Negative correlation between hydro inflows and wind (dry periods often have more wind in some regions). Wind sources are positively correlated with each other.

3.5.8 Battery Storage (system/batteries.json) - DEFERRED

Implementation Status: This feature is designed but **deferred for future implementation**. The data model is specified here to guide future development.

Purpose: Models battery energy storage systems (BESS) that can store and release electrical energy. Similar conceptually to pumped hydro storage but with different characteristics:

Characteristic	Description
No water	Energy stored directly, no cascade topology
Round-trip efficiency	Energy losses during charge/discharge cycles
Degradation	Long-term capacity reduction (not modeled in LP, but tracked)

Design Principle: We model batteries as **linear storage devices** without integer variables. Features requiring binary decisions (commitment, minimum up/down times, cycle limits) are not supported to maintain LP tractability.

Inspired by CEPEL modeling: Based on NEWAVE/DECOMP battery representation but simplified for linear programming.

```

{
  "batteries": [
    {
      "id": 0,
      "name": "BESS_SE_1",
      "bus_id": 0,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "capacity": {
        "energy_mwh": 400.0,
        "charge_mw": 100.0,
        "discharge_mw": 100.0
      },
      "efficiency": {
        "charge": 0.95,
        "discharge": 0.95
      }
    }
  ]
}

```

```

    },
    "initial_soc_mwh": 200.0,
    "soc_limits": {
        "min_mwh": 40.0,
        "max_mwh": 360.0
    }
}
]
}

```

Field	Type	Description
id	i32	Unique battery identifier
name	string	Battery name
bus_id	i32	Bus where battery is connected
entry_stage_id	i32?	First operating stage (null = always)
exit_stage_id	i32?	Last operating stage (null = forever)
capacity.energy_mwh	f64	Total energy storage capacity
capacity.charge_mw	f64	Maximum charging power (grid→battery)
capacity.discharge_mw	f64	Maximum discharging power (battery→grid)
efficiency.charge	f64	Charging efficiency (0-1), typically 0.90-0.98
efficiency.discharge	f64	Discharging efficiency (0-1), typically 0.90-0.98
initial_soc_mwh	f64	Initial state of charge
soc_limits.min_mwh	f64	Minimum allowed state of charge
soc_limits.max_mwh	f64	Maximum allowed state of charge

LP Variables

Variable	Units	Description
battery_soc	MWh	State of charge at end of stage (state variable)
battery_charge	MW	Charging power (grid→battery) per block
battery_discharge	MW	Discharging power (battery→grid) per block

Energy Balance Constraint

$$SOC_{end} = SOC_{start} + \sum_{blocks} (charge \times _charge - discharge / _discharge) \times block_hours$$

Where:

- $_charge$ = charging efficiency
- $_discharge$ = discharging efficiency
- Round-trip efficiency = $_charge \times _discharge$ (typically 0.81-0.95)

Bus Balance Integration Battery charging adds to bus load, discharging adds to supply:

```

// Bus load balance

$$\sum_{generation} - \sum_{load} + battery\_discharge - battery\_charge = 0$$


```

State Variable in SDDP Battery SOC_{end} is a **state variable** in the SDDP formulation:

- Cuts include coefficients for battery storage
- Initial SOC passed between stages

- Water value analogue: “energy value” of stored electricity

Note on Unsupported Features:

Feature	Reason Not Modeled
Cycle counting	Maximum cycles per period would require integer tracking
Commitment	Minimum charge/discharge periods would require binary variables
Degradation	Capacity fade over time would require state augmentation
Temperature effects	Efficiency variation with temperature not modeled

These limitations maintain LP tractability. For detailed battery modeling, external tools or post-processing may be needed.

Battery Bounds (`constraints/battery_bounds.parquet`) - Optional

Column	Type	Description
<code>battery_id</code>	i32	Battery identifier
<code>stage_id</code>	i32	Stage index
<code>charge_mw</code>	f64	Max charge power override (null = use base)
<code>discharge_mw</code>	f64	Max discharge power override (null = use base)
<code>min_soc_mwh</code>	f64	Min SOC override (null = use base)
<code>max_soc_mwh</code>	f64	Max SOC override (null = use base)

3.6 Thermal Registry (`system/thermals.json`)

Order Invariance: The order of thermals in this array does NOT affect results. After loading, thermals are sorted by id. See Section 1.3.

Note: Use `entry_stage_id` and `exit_stage_id` to model plants entering or exiting the system. Alternatively, use `thermal_bounds.parquet` to set generation to 0 for stages where the plant is offline.

Operative State: Each thermal has an operative state per stage:

State	When	Behavior
<code>non_existing</code>	Before <code>entry_stage_id</code>	No variables in LP
<code>operating</code>	Between <code>entry_stage_id</code> and <code>exit_stage_id</code>	Normal operation
<code>decommissioned</code>	After <code>exit_stage_id</code>	No variables in LP

```
{
  "thermals": [
    {
      "id": 0,
      "name": "ANGRA1",
      "bus_id": 0,
      "entry_stage_id": null,
      "exit_stage_id": null,
      "cost_segments": [
        {"capacity_mw": 640.0, "cost_per_mwh": 15.0}
      ],
      "generation": {
        "min_mw": 500.0,
```



```

    "max_mw": 640.0
  }
}
]
}

```

Thermal Operative States

State	Condition	LP Variables
non_existing	Before entry_stage_id	None
operating	Between entry_stage_id and exit_stage_id	generation per segment
decommissioned	After exit_stage_id	None

GNL (Gas Natural Liquefeito) Thermal Plants

GNL plants require **dispatch anticipation**: the dispatch decision must be made N stages ahead due to fuel ordering lead times. This creates additional state variables representing committed dispatch for future stages.

Data Model:

GNL capability is configured via an optional `gnl_config` field in the thermal definition:

```

{
  "id": 10,
  "name": "GNL_PLANT",
  "bus_id": 2,
  "gnl_config": {
    "lag_stages": 2
  },
  "cost_segments": [
    {"capacity_mw": 500.0, "cost_per_mwh": 200.0}
  ],
  "generation": {
    "min_mw": 0.0,
    "max_mw": 500.0
  }
}

```

Field	Type	Description
<code>gnl_config</code>	object or null	GNL configuration. If null or omitted, thermal is standard (not GNL).
<code>gnl_config.lag_stages</code>	i32	Number of stages ahead for dispatch decision (e.g., 2 means dispatch at stage t is decided at stage t-2).

State Variable Extension:

When a thermal has `gnl_config`, the algorithm adds state variables for the committed dispatch pipeline. At stage `t`, the state includes:

State Variable	Description
<code>gnl_committed[thermal_id, t+1]</code>	Dispatch committed for stage t+1

State Variable	Description
<code>gnl_committed[thermal_id, t+2]</code>	Dispatch committed for stage $t+2$
...	... up to <code>lag_stages</code> ahead

The initial values of this pipeline are specified in `initial_conditions.json` (see Section 3.9).

Backward Pass Impact:

GNL state variables receive cuts during backward passes, capturing the value of having flexible vs. committed dispatch for future stages.

Status: The data model is ready. Implementation is planned but not yet complete.

3.7 Stage Definitions (`stages.json`)

Order Invariance: The order of stages and blocks in their arrays does NOT affect results. After loading, stages are sorted by `id`, and blocks within each stage are sorted by `id`. See Section 1.3.

Note: Each stage defines its own blocks (count and hours). The weight is computed internally from block hours, not user-specified. Block IDs within each stage must be contiguous, starting at 0 (validated: 0, 1, 2, ..., $n-1$).

Pre-study stages: Stages with negative IDs represent historical periods before the study horizon. These are used only for PAR model initialization (providing lag values). Pre-study stages only need `id`, `start_date`, and `end_date`.

Future Extension - Temporal Scope Decoupling: The current design couples stage decomposition with decision period resolution. A future enhancement (documented in [MATHEMATICAL_FORMULATIONS.md Appendix C.7](#)) would allow multiple sequential decision periods within each SDDP stage, enabling configurations like “first month weekly, rest monthly” without cut explosion. This would add a `periods` array within each stage while preserving state dimension at stage boundaries.

Risk Measure (CVaR): The `risk_measure` field can be:

Option	Description
"expectation"	Risk-neutral expected value (default)
Object {"cvar": {...}}	CVaR parameters with <code>alpha</code> (confidence level) and <code>lambda</code> (weight)

CVaR parameter details:

- `alpha`: Confidence level (e.g., 0.95 means 5% worst scenarios)
- `lambda`: Weight of CVaR vs expectation (0 = pure expectation, 1 = pure CVaR)
- Final risk measure: $(1 - \text{lambda}) \times E[\text{cost}] + \text{lambda} \times \text{CVaR_alpha}[\text{cost}]$

CVaR parameters can vary by stage, allowing risk-averse policies in early stages and risk-neutral in later stages.

Scenario Sampling Method: The `sampling_method` field controls how scenarios are generated for each stage. Different methods have different statistical properties:

Scenario Sampling Methods

Method	Description	Use Case
saa	Sample Average Approximation (default). Pure Monte Carlo random sampling from the stochastic process. Simple, unbiased, but may have high variance with few samples.	General purpose, baseline

Method	Description	Use Case
lhs	Latin Hypercube Sampling. Stratified sampling ensuring uniform coverage of the probability space. Reduces variance for the same number of samples.	Medium sample sizes (20-100)
qmc_sobol	Quasi-Monte Carlo (Sobol sequences). Low-discrepancy sequences for better space coverage. Faster convergence than pure random.	High-dimensional problems, deterministic-like scenarios
qmc_halton	Quasi-Monte Carlo (Halton sequences). Alternative low-discrepancy sequence, simpler than Sobol.	Similar to Sobol, less correlated dimensions
selective	Selective/Representative Sampling. Uses clustering (e.g., k-means) on historical data to select representative scenarios with weights.	When historical patterns should guide sampling
historical	Historical Scenarios. Uses actual historical sequences from <code>inflow_history.parquet</code> extended data. No random sampling.	Backtesting, deterministic studies

Note: Sampling method can vary by stage, allowing adaptive strategies (e.g., more sophisticated sampling in early stages, simpler in later stages).

```
{
  "pre_study_stages": [
    {"id": -6, "start_date": "2023-07-01", "end_date": "2023-08-01"},
    {"id": -5, "start_date": "2023-08-01", "end_date": "2023-09-01"},
    {"id": -4, "start_date": "2023-09-01", "end_date": "2023-10-01"},
    {"id": -3, "start_date": "2023-10-01", "end_date": "2023-11-01"},
    {"id": -2, "start_date": "2023-11-01", "end_date": "2023-12-01"},
    {"id": -1, "start_date": "2023-12-01", "end_date": "2024-01-01"}
  ],
  "stages": [
    {
      "id": 0,
      "start_date": "2024-01-01",
      "end_date": "2024-02-01",
      "blocks": [
        {"id": 0, "name": "LEVE", "hours": 168},
        {"id": 1, "name": "MEDIA", "hours": 336},
        {"id": 2, "name": "PESADA", "hours": 168}
      ],
      "risk_measure": {"cvar": {"alpha": 0.95, "lambda": 0.50}},
      "state_variables": "storage_and_inflow",
      "num_scenarios": 20,
      "sampling_method": "lhs"
    },
    {
      "id": 1,
      "start_date": "2024-02-01",
      "end_date": "2024-03-01",
      "blocks": [
        {"id": 0, "name": "LEVE", "hours": 168},
        {"id": 1, "name": "MEDIA", "hours": 336},
        {"id": 2, "name": "PESADA", "hours": 168}
      ],
    }
  ],
}
```

```

    "risk_measure": {"cvar": {"alpha": 0.95, "lambda": 0.25}},
    "state_variables": "storage_and_inflow",
    "num_scenarios": 20,
    "sampling_method": "lhs"
  },
  {
    "id": 2,
    "start_date": "2024-03-01",
    "end_date": "2024-04-01",
    "blocks": [
      {"id": 0, "name": "LEVE", "hours": 168},
      {"id": 1, "name": "MEDIA", "hours": 336},
      {"id": 2, "name": "PESADA", "hours": 168}
    ],
    "risk_measure": "expectation",
    "state_variables": "storage_and_inflow",
    "num_scenarios": 20,
    "sampling_method": "saa"
  }
],
"transitions": [
  {"source_id": 0, "target_id": 1, "probability": 1.0, "discount_rate": 0.0},
  {"source_id": 1, "target_id": 2, "probability": 1.0, "discount_rate": 0.0}
]
}

```

Stage Field Reference

Field	Type	Required	Default	Description
id	i32	Yes	-	Unique stage identifier
start_date	string	Yes	-	Stage start date (ISO 8601)
end_date	string	Yes	-	Stage end date (ISO 8601)
blocks	array	Yes	-	Load blocks within stage
risk_measure	string/object	No	"expectation"	Risk measure: "expectation" or {"cvar": {...}}
state_variables	string	No	"storage_only"	State configuration
num_scenarios	i32	Yes	-	Number of scenarios for this stage
sampling_method	string	No	"saa"	Sampling method (see table above)

3.8 Uncertainty Models (scenarios/inflow_models.parquet)

Note: Uncertainty models are now defined per entity per stage in tabular format. This enables:

Advantage	Description
Variable time resolutions	Daily, weekly, monthly, quarterly stages
No cycling complexity	Each stage has explicit parameters
Easy bulk editing	Programmatic generation
AR order 0	Independent noise (no temporal correlation)

The AR model for stage t uses lags from previous stages. AR coefficients reference normalized residuals from stages $t-1$, $t-2$, ..., t -order.

Inflow Models Schema (`scenarios/inflow_models.parquet`)

Column	Type	Description
<code>hydro_id</code>	i32	Hydro plant ID (must exist in <code>hydros.json</code>)
<code>stage_id</code>	i32	Stage ID (must exist in <code>stages.json</code>)
<code>mean_m3s</code>	f64	Mean inflow for this stage
<code>std_m3s</code>	f64	Standard deviation (0 = deterministic)
<code>ar_order</code>	i32	AR order (0 = independent, 1-12 typical)
<code>ar_coef_01</code>	f64	AR coefficient for lag 1 (null if order < 1)
<code>ar_coef_02</code>	f64	AR coefficient for lag 2 (null if order < 2)
<code>ar_coef_03</code>	f64	AR coefficient for lag 3 (null if order < 3)
<code>ar_coef_04</code>	f64	AR coefficient for lag 4 (null if order < 4)
<code>ar_coef_05</code>	f64	AR coefficient for lag 5 (null if order < 5)
<code>ar_coef_06</code>	f64	AR coefficient for lag 6 (null if order < 6)
<code>ar_coef_07</code>	f64	AR coefficient for lag 7 (null if order < 7)
<code>ar_coef_08</code>	f64	AR coefficient for lag 8 (null if order < 8)
<code>ar_coef_09</code>	f64	AR coefficient for lag 9 (null if order < 9)
<code>ar_coef_10</code>	f64	AR coefficient for lag 10 (null if order < 10)
<code>ar_coef_11</code>	f64	AR coefficient for lag 11 (null if order < 11)
<code>ar_coef_12</code>	f64	AR coefficient for lag 12 (null if order < 12)

Load Models Schema (`scenarios/load_models.parquet`)

Column	Type	Description
<code>bus_id</code>	i32	Bus ID
<code>stage_id</code>	i32	Stage ID (must exist in <code>stages.json</code>)
<code>mean_mw</code>	f64	Mean load for this stage
<code>std_mw</code>	f64	Standard deviation (0 = deterministic)

Note: Load models are typically independent (no AR), so no AR columns are included. If AR load models are needed in the future, a similar structure can be added.

3.9 Initial Conditions (`initial_conditions.json`)

Note: Initial storage is the reservoir level at the start of the study. For hydros with `entry_stage_id`, this is the storage when they enter the system (not at stage 0).

Inflow history is in a separate Parquet file indexed by `hydro_id` and `stage_id` (using pre-study stage IDs). Only hydros active from the start need pre-study inflow history.

```
{
  "storage": [
    {"hydro_id": 0, "value_hm3": 15000.0},
    {"hydro_id": 1, "value_hm3": 8500.0},
    {"hydro_id": 10, "value_hm3": 2500.0}
  ],
  "gnl_pipeline": [
    {"thermal_id": 10, "stage_offset": 1, "committed_mw": 250.0},

```

```

    {"thermal_id": 10, "stage_offset": 2, "committed_mw": 300.0}
  ]
}

```

Validation:

Rule	Description
Hydro coverage	Every hydro in <code>hydros.json</code> must have an entry in <code>storage</code>
Storage bounds	Storage value must be within <code>[min_storage_hm3, max_storage_hm3]</code>
Late-entry hydros	For hydros entering later, this is their initial storage at entry

GNL Pipeline Initial Conditions When GNL thermals are configured (see Section 3.6), their initial committed dispatch pipeline is specified here:

Field	Type	Description
<code>thermal_id</code>	i32	ID of the GNL thermal (must have <code>gnl_config</code> defined)
<code>stage_offset</code>	i32	Future stage offset (1 = stage 1, 2 = stage 2, etc.)
<code>committed_mw</code>	f64	Committed dispatch in MW for that future stage

Validation:

Rule	Description
<code>gnl_pipeline</code>	Optional. If omitted, GNL thermals start with zero committed dispatch
Stage coverage	Each GNL thermal with <code>gnl_config.lag_stages = N</code> should have entries for <code>stage_offset</code> 1 through N
Thermal reference	<code>thermal_id</code> must reference a thermal with <code>gnl_config</code> defined
Bounds	<code>committed_mw</code> must be within the thermal's generation bounds

Example: If thermal 10 has `gnl_config.lag_stages = 2`, it means dispatch for stage t is decided at stage $t-2$. At the start (stage 0), we need to know:

- `stage_offset: 1` → Dispatch committed for stage 1 (decided at stage -1, before study starts)
- `stage_offset: 2` → Dispatch committed for stage 2 (decided at stage 0, the first decision stage)

Inflow History Schema (`scenarios/inflow_history.parquet`)

Note: Contains realized inflow values for pre-study stages (negative stage IDs). Used to initialize AR model lags.

Validation:

Rule	Description
Pre-study coverage	Pre-study stages must cover at least the maximum AR order used (if max order is 6, need stages -6 to -1)
Hydro entries	Each hydro active from stage 0 must have entries for all required pre-study stages based on its AR order
Late-entry hydros	Hydros entering later (<code>entry_stage_id > 0</code>) do not need pre-study history; their AR is initialized from study stages

Column	Type	Description
hydro_id	i32	Hydro plant ID
stage_id	i32	Pre-study stage ID (negative, e.g., -6, -5, ..., -1)
inflow_m3s	f64	Realized inflow value

Example (for AR order up to 6, with 2 hydros active from start):

```
hydro_id | stage_id | inflow_m3s
-----|-----|-----
0        | -6       | 45.2
0        | -5       | 52.1
0        | -4       | 58.3
0        | -3       | 61.0
0        | -2       | 55.8
0        | -1       | 48.5
1        | -6       | 120.5
1        | -5       | 135.2
...
```

3.10 Load Factors by Block (scenarios/load_factors.json) - Optional

Note: This file is **optional**. If missing, all block factors default to 1.0.

Load uncertainty generates a base load realization in **MW** per stage (similar to how inflows are generated in m³/s). The MW unit is power, independent of time duration. To get energy (MWh), multiply by block hours.

Block factors are **multipliers** applied to the stochastic load value to get block-level loads. For example, if stage load is 1000 MW and block factors are [0.85, 1.00, 1.15], the blocks get [850, 1000, 1150] MW respectively.

```
{
  "load_factors": [
    {
      "bus_id": 0,
      "stage_id": 0,
      "block_factors": [
        {"block_id": 0, "factor": 0.85},
        {"block_id": 1, "factor": 1.00},
        {"block_id": 2, "factor": 1.15}
      ]
    }
  ]
}
```

3.11 Exchange Factors by Block (scenarios/exchange_factors.json) - Optional

Note: This file is **optional**. If missing, all block factors default to 1.0.

Exchange (transmission) limits may vary by block due to thermal limits, contractual constraints, or operational patterns. Block factors are **multipliers** applied to the stage-level line capacity to get block-level limits.

For example, if a line has 5000 MW direct capacity and block factors are [0.90, 1.00, 1.10], the blocks get [4500, 5000, 5500] MW respectively.

```
{
  "exchange_factors": [
    {
```

```

    "line_id": 0,
    "stage_id": 0,
    "block_factors": [
      {"block_id": 0, "direct_factor": 0.90, "reverse_factor": 0.90},
      {"block_id": 1, "direct_factor": 1.00, "reverse_factor": 1.00},
      {"block_id": 2, "direct_factor": 1.10, "reverse_factor": 1.10}
    ]
  }
]
}

```

3.12 Correlation (scenarios/correlation.json)

Purpose: Defines spatial correlation between stochastic processes (inflows, loads, non-controllable generation). Uses Cholesky decomposition to transform independent standard normal samples into correlated samples.

Profile-Based Time-Varying Correlation: Instead of storing element-wise overrides (which would be $O(\text{stages} \times \text{entities}^2)$ rows), POWERS uses a **profile-based system**:

- Named profiles:** Define multiple correlation matrices in `correlation.json` (e.g., “default”, “wet_season”, “dry_season”)
- Schedule table:** A compact Parquet file maps each stage to a profile name

This design reduces storage from potentially millions of rows to $\sim T$ rows (one per stage) plus a few matrix definitions.

```

{
  "$schema": "https://powers-rs.io/schemas/v2/correlation.schema.json",
  "method": "cholesky",
  "profiles": {
    "default": {
      "blocks": [
        {
          "name": "southeast_cascade",
          "entities": [
            {"type": "inflow", "id": 0},
            {"type": "inflow", "id": 1},
            {"type": "inflow", "id": 2}
          ],
          "matrix": [
            [1.0, 0.75, 0.60],
            [0.75, 1.0, 0.70],
            [0.60, 0.70, 1.0]
          ]
        }
      ]
    },
    "wet_season": {
      "blocks": [
        {
          "name": "southeast_cascade",
          "entities": [
            {"type": "inflow", "id": 0},
            {"type": "inflow", "id": 1},
            {"type": "inflow", "id": 2}
          ],
          "matrix": [
            [1.0, 0.90, 0.80],
            [0.90, 1.0, 0.85],
            [0.80, 0.85, 1.0]
          ]
        }
      ]
    }
  }
}

```



```

    ]
  }
]
},
"dry_season": {
  "blocks": [
    {
      "name": "southeast_cascade",
      "entities": [
        {"type": "inflow", "id": 0},
        {"type": "inflow", "id": 1},
        {"type": "inflow", "id": 2}
      ],
      "matrix": [
        [1.0, 0.60, 0.45],
        [0.60, 1.0, 0.55],
        [0.45, 0.55, 1.0]
      ]
    }
  ]
}
}
}
}
}

```

Correlation Profile Fields

Field	Type	Required	Description
method	string	Yes	Correlation method: "cholesky" (currently the only supported method)
profiles	object	Yes	Map of profile names to correlation block definitions
profiles.<name>.blocks	array	Yes	Array of correlation blocks for this profile
profiles.<name>.blocks[<id>].name	string	Yes	Unique name for correlation block
profiles.<name>.blocks[<id>].entities	array	Yes	Entities in this correlation group
profiles.<name>.blocks[<id>].matrix	array	Yes	Correlation matrix (must be positive semi-definite)

Note: The profile named **"default"** is required and used for any stage not explicitly mapped in the schedule.

Time-Varying Correlation Schedule (scenarios/correlation_schedule.parquet) - Optional

Purpose: Maps stages to correlation profiles. If this file is missing, all stages use the **"default"** profile.

Column	Type	Description
stage_id	i32	Stage ID
profile_name	string	Profile name (must exist in correlation.json)

Example (12-month seasonal pattern, repeated for 5 years):

stage_id	profile_name
0	wet_season
1	wet_season
2	wet_season
3	wet_season
4	default
5	dry_season
6	dry_season
7	dry_season
8	dry_season
9	dry_season
10	default
11	wet_season
12	wet_season
...	...

Storage Comparison: For a system with 160 hydros in a single correlation block over 60 stages:

Format	Storage
Old element-wise format	$60 \times 160 \times 160 / 2$ 768,000 rows (upper triangle only)
New profile-based format	60 rows + ~ 3 profiles $\times 160 \times 160$ matrix entries in JSON negligible

Validation: The loader verifies: 1. All profile names in schedule exist in `correlation.json` 2. All correlation matrices are positive semi-definite 3. Entity IDs in correlation blocks exist in the system

Correlation Input Options Summary

Approach	Files Required	Use Case
Static correlation	<code>correlation.json</code> with only "default" profile	Same correlation for all stages
Seasonal correlation	<code>correlation.json</code> + <code>correlation_schedule.parquet</code>	Different profiles by season/stage
Computed from history	External preprocessing $\rightarrow$ <code>correlation.json</code>	When user has inflow history and wants to derive correlation

Note: Computing correlation from historical inflows is outside POWERS scope. Users should use tools like GEVAZP, Python/R statistical packages, or custom scripts to estimate correlation matrices from historical data, then provide the results in the profile format above.

Alternative Approaches: CEPTEL NEWAVE computes correlation internally from inflow history, assuming regular monthly stages. SPARHTACUS supports receiving either raw history or pre-computed PAR models with correlation data. POWERS takes a flexible approach: the user provides pre-computed correlation profiles.

3.13 Constraints (constraints/)

Note: Time-varying bounds allow entities to have different operational limits per stage. This is more direct than availability factors - the LP uses these bounds directly.

Thermal Bounds Schema (`constraints/thermal_bounds.parquet`) - Optional

Note: If a thermal is not present for a stage, uses bounds from `thermals.json`. Partial overrides allowed (only specify stages that differ from base).

Column	Type	Description
<code>thermal_id</code>	i32	Thermal unit identifier
<code>stage_id</code>	i32	Stage index
<code>min_generation_mw</code>	f64	Minimum generation (null = use base)
<code>max_generation_mw</code>	f64	Maximum generation (null = use base)

Hydro Bounds Schema (`constraints/hydro_bounds.parquet`) - Optional

Note: If a hydro is not present for a stage, uses bounds from `hydros.json`. Useful for maintenance outages, seasonal restrictions, environmental constraints.

Note: Specifies the filling inflow and minimum outflow constraints during dead-volume filling stages. Required for each hydro with `filling` configured, for each stage in the filling period.

Parameter	Description
<code>filling_inflow_m3s</code>	Water retained for reservoir filling (removed from cascade)
If insufficient inflow	If <code>inflow - filling_inflow < min_outflow</code> , a slack variable with <code>outflow_violation_penalty</code> is used

Water Withdrawal (Retirada de Água): Water removed from the reservoir for human consumption, irrigation, industrial use, etc. Positive values represent water leaving the system; negative values represent external water additions (transpositions). The withdrawal is subtracted from the water balance equation. A slack variable with `water_withdrawal_violation_cost` is used when inflow cannot meet the withdrawal target.

Evaporation Coefficient: Monthly evaporation rate (mm/month) applied to the reservoir surface area. The actual evaporated flow is computed from the volume-area relationship (see `system/hydro_geometry.parquet`). When not specified, evaporation is not considered for that stage.

Column	Type	Description
<code>hydro_id</code>	i32	Hydro plant identifier
<code>stage_id</code>	i32	Stage index
<code>min_turbined_m3s</code>	f64	Minimum turbined flow (null = use base)
<code>max_turbined_m3s</code>	f64	Maximum turbined flow (null = use base)
<code>min_storage_hm3</code>	f64	Minimum storage (null = use base)
<code>max_storage_hm3</code>	f64	Maximum storage (null = use base)
<code>min_outflow_m3s</code>	f64	Minimum outflow (null = use base)
<code>max_outflow_m3s</code>	f64	Maximum outflow (null = use base)
<code>filling_inflow_m3s</code>	f64	Water retained for filling
<code>water_withdrawal_m3s</code>	f64	Water withdrawal (positive = remove, negative = add)
<code>evaporation_coef_mm</code>	f64	Monthly evaporation coefficient (mm/month)

Line Bounds Schema (`constraints/line_bounds.parquet`) - Optional

Note: If a line is not present for a stage, uses bounds from `lines.json`. Useful for planned transmission upgrades or temporary capacity reductions.

Column	Type	Description
<code>line_id</code>	i32	Transmission line identifier
<code>stage_id</code>	i32	Stage index
<code>direct_mw</code>	f64	Direct flow capacity (null = use base)
<code>reverse_mw</code>	f64	Reverse flow capacity (null = use base)

Order Invariance: The order of constraints in `generic_constraints.json` does NOT affect results. After loading, constraints are sorted by `id`. See Section 1.3.

Design Rationale: Users may need to express custom linear constraints that combine multiple LP variables. These “generic” or “free” constraints allow modeling:

Constraint Type	Example
Regional generation limits	Minimum/maximum total hydro generation per region
Energy contracts	Sum of generation from specific plants
Irrigation agreements	Sum of outflows
Environmental corridors	Combined outflow requirements
Fuel availability	Sum of thermal generation
Custom	Any other linear combination of optimization variables

CEPEL Constraint Types Mapping

Context: CEPEL models (NEWAVE, DECOMP) define several specialized constraint types. In POWE.RS v2.0, all these are expressed as generic constraints. This table shows how to model each CEPEL constraint type:

CEPEL Type	Name	Description	POWE.RS Generic Constraint Expression
RHQ	Restrição Hidráulica de Quantidade	Maximum outflow as linear function of storage/time	<code>hydro_outflow(id) <= bound</code> (bound varies by stage in <code>constraint_bounds.parquet</code>)
RE	Restrição Elétrica	Electrical generation constraints per region/system	$\Sigma \text{hydro_generation}(\text{id}) + \Sigma \text{thermal_generation}(\text{id}) \geq \text{bound}$
RHE	Restrição de Energia Hidráulica	Minimum/maximum hydraulic energy (generation $\times$ time) per region	$\Sigma \text{hydro_generation}(\text{id}) \geq \text{bound}$ (bound in MWavg or MW depending on formulation)
RHV	Restrição de Volume Hidráulico	Storage constraints for single plants or groups (flood control, navigation)	<code>hydro_storage(id) <= bound</code> or $\Sigma \text{hydro_storage}(\text{id}) \leq \text{bound}$
GHMIN	Geração Hidráulica Mínima	Minimum hydraulic generation for a subsystem	$\Sigma \text{hydro_generation}(\text{ids_in_subsystem}) \geq \text{min_gh}$

CEPEL Type	Name	Description	POWE.RS Generic Constraint Expression
GTMIN	Geração Térmica Mínima	Minimum thermal generation for a subsystem	$\Sigma \text{ thermal_generation(ids_in_subsystem)} \geq \text{min_gt}$
DEFMAX	Déficit Máximo	Maximum allowed deficit per subsystem	$\text{bus_deficit}(\text{bus_id}) \leq \text{max_deficit}$

Example: RHQ (Maximum Outflow) as Generic Constraint:

```
{
  "id": 10,
  "name": "RHQ_FURNAS_flood_control",
  "description": "Maximum outflow from Furnas during wet season (flood control)",
  "expression": "hydro_outflow(5)",
  "sense": "<=",
  "slack": {
    "enabled": true,
    "penalty": 10000.0
  }
}
```

With time-varying bounds in `constraint_bounds.parquet`:

constraint_id	stage_id	bound
10	0	3000.0
10	1	3500.0
10	2	4000.0
...	...	...

Note on REE (Reservatório Equivalente de Energia): POWE.RS uses **individual hydro representation** (like DECOMP) rather than aggregated energy reservoirs (like NEWAVE). REE-based modeling is not in scope. Users requiring REE-level analysis should use NEWAVE or aggregate outputs post-processing.

Variable Reference Syntax Variables are referenced using a function-like syntax: `variable_type(entity_id)`. For block-specific variables (when using chronological blocks), use `variable_type(entity_id, block_id)` or omit `block_id` to sum over all blocks.

Variable Name	Syntax	Units	Description
hydro_storage	hydro_storage(id)	hm ³	End-of-stage storage
hydro_turbined	hydro_turbined(id) or hydro_turbined(id, block)	m ³ /s	Turbined flow
hydro_spillage	hydro_spillage(id) or hydro_spillage(id, block)	m ³ /s	Spillage

Variable Name	Syntax	Units	Description
hydro_diversion	hydro_diversion(id) or hydro_diversion(id, block)	m ³ /s	Diversion flow
hydro_outflow	hydro_outflow(id) or hydro_outflow(id, block)	m ³ /s	Total outflow
hydro_generation	hydro_generation(id) or hydro_generation(id, block)	MW	Power generation
hydro_evaporation	hydro_evaporation(id)	m ³ /s	Evaporated flow
hydro_withdrawal	hydro_withdrawal(id)	m ³ /s	Water withdrawal (target)
thermal_generation	thermal_generation(id) or thermal_generation(id, block)	MW	Power generation
line_direct	line_direct(id) or line_direct(id, block)	MW	Direct flow
line_reverse	line_reverse(id) or line_reverse(id, block)	MW	Reverse flow
bus_deficit	bus_deficit(id) or bus_deficit(id, block)	MW	Deficit
bus_excess	bus_excess(id) or bus_excess(id, block)	MW	Excess
pumping_flow	pumping_flow(id) or pumping_flow(id, block)	m ³ /s	Pumped water flow
pumping_power	pumping_power(id) or pumping_power(id, block)	MW	Pumping power consumption
contract_import	contract_import(id) or contract_import(id, block)	MW	Contract import
contract_export	contract_export(id) or contract_export(id, block)	MW	Contract export

Expression Grammar Expressions are linear combinations of variables with numeric coefficients:

```

expression    ::= term (('+' | '-') term)*
term          ::= coefficient? variable | number
coefficient   ::= number '*'
variable      ::= var_name '(' entity_id (',' block_id)? ')'
var_name      ::= 'hydro_storage' | 'hydro_turbined' | 'hydro_spillage' | 'hydro_diversion'
               | 'hydro_outflow' | 'hydro_generation' | 'hydro_evaporation' | 'hydro_withdrawal'
               | 'thermal_generation' | 'line_direct' | 'line_reverse'
               | 'bus_deficit' | 'bus_excess'
               | 'pumping_flow' | 'pumping_power' | 'contract_import' | 'contract_export'
entity_id     ::= integer
block_id      ::= integer
number        ::= float | integer

```

Examples:

- `hydro_generation(10) + hydro_generation(11) + hydro_generation(12)` — sum of generation from 3 hydros
- `2.5 * thermal_generation(5) - hydro_generation(3)` — weighted combination
- `hydro_outflow(7) + hydro_outflow(8)` — combined outflow from two plants
- `thermal_generation(0) + thermal_generation(1) + 100.0` — sum with constant offset
- `pumping_power(0) + pumping_power(1)` — total pumping station power consumption
- `contract_import(0) - contract_export(1)` — net energy import
- `hydro_turbined(5, 0) + hydro_turbined(5, 1)` — sum of turbined in blocks 0 and 1

Constraint Definition (`constraints/generic_constraints.json`)

```

{
  "$schema": "https://powers-rs.io/schemas/v2/generic_constraints.schema.json",
  "constraints": [
    {
      "id": 0,
      "name": "min_southeast_hydro",
      "description": "Minimum hydro generation in Southeast region",
      "expression": "hydro_generation(0) + hydro_generation(1) + hydro_generation(2)",
      "sense": ">=",
      "slack": {
        "enabled": true,
        "penalty": 5000.0
      }
    },
    {
      "id": 1,
      "name": "itaipu_contract",
      "description": "Itaipu energy contract - must deliver at least base amount",
      "expression": "hydro_generation(50)",
      "sense": ">=",
      "slack": {
        "enabled": true,
        "penalty": 8000.0
      }
    },
    {
      "id": 2,
      "name": "environmental_flow",
      "description": "Combined outflow requirement for river stretch",
      "expression": "hydro_outflow(10) + hydro_outflow(11)",
      "sense": ">=",
      "slack": {

```

```

        "enabled": true,
        "penalty": 3000.0
    }
},
{
    "id": 3,
    "name": "gas_availability",
    "description": "Total gas thermal generation limited by pipeline",
    "expression": "thermal_generation(5) + thermal_generation(6) + thermal_generation(7)",
    "sense": "<=",
    "slack": {
        "enabled": false
    }
}
]
}

```

Constraint Fields

Field	Type	Required	Description
id	i32	Yes	Unique constraint identifier
name	string	Yes	Short name for reports
description	string	No	Human-readable description
expression	string	Yes	Linear expression (parsed using grammar above)
sense	string	Yes	">=" (lower bound), "<=" (upper bound), or "==" (equality)
slack.enabled	bool	Yes	Whether to add slack variable for feasibility
slack.penalty	f64	If enabled	Penalty per unit of violation

Constraint Bounds (constraints/constraint_bounds.parquet) Bounds can vary by stage (and optionally by block for block-specific constraints). This follows the same pattern as entity bounds.

Column	Type	Description
constraint_id	i32	References constraint definition
stage_id	i32	Stage index
block_id	i32	Block index (null = applies to all blocks)
bound	f64	RHS value for the constraint

Example rows: | constraint_id | stage_id | block_id | bound | | 0 | 0 | null | 5000.0 | | 0 | 1 | null | 5200.0 | | 1 | 0 | null | 6000.0 | | 2 | 0 | null | 150.0 | | 3 | 0 | 0 | 2000.0 | | 3 | 0 | 1 | 2500.0 |

LP Integration For constraint c with sense $\geq$:

$$\sum (\text{coef}_i \times \text{var}_i) + \text{slack_below}[c] \geq \text{bound}[c]$$

For constraint c with sense $\leq$:

$$\sum (\text{coef}_i \times \text{var}_i) - \text{slack_above}[c] \leq \text{bound}[c]$$

For constraint c with sense $=$:

$$\sum (\text{coef}_i \times \text{var}_i) + \text{slack_below}[c] - \text{slack_above}[c] = \text{bound}[c]$$

Slack variables are only created if `slack.enabled = true`.

Validation Rules

1. All entity IDs referenced in expressions must exist in the system
2. Block IDs (if specified) must be valid for the stage
3. Expressions must parse correctly according to the grammar
4. `constraint_bounds.parquet` must have entries for all study stages (0 to `num_stages-1`) for each constraint
5. Constraint IDs must be unique and contiguous (0, 1, 2, ...)
6. If `slack.enabled = true`, `slack.penalty` must be provided and positive

3.14 Policy Directory (policy/)

Unified Policy Directory: POWE.RS uses a single `policy/` directory that serves both as **input** (loading existing cuts/states) and **output** (writing updated policy data). This unified approach simplifies the user experience:

Aspect	Description
No separate directories	One directory contains all policy artifacts
Read-modify-write pattern	The program loads existing data, continues training, and updates the same directory
Mode-based behavior	The <code>policy.mode</code> configuration determines whether to start fresh, warm-start, or resume

Policy Directory Structure

```
policy/
  metadata.json          # Algorithm state, RNG, bounds (optional on input)
  state_dictionary.json  # State variable mapping (required if cuts exist)
  cuts/                  # Outer approximation (standard SDDP cuts)
    stage_000.parquet
    stage_001.parquet
    ...
  states/                # Visited states for cut selection
    stage_000.parquet
    ...
  vertices/              # Inner approximation (SIDP upper bounds)
    stage_000.parquet    # Only present if upper_bound_evaluation.enabled
    ...
  basis/                 # Solver basis for exact reproducibility (optional)
    stage_000.parquet
    ...
```

Policy Modes

Mode	Reads From	Behavior
<code>fresh</code>	Nothing	Start from scratch. Any existing files in <code>policy/</code> are ignored (but not deleted).
<code>warm_start</code>	<code>cuts/</code> , <code>states/</code> , <code>state_dictionary.json</code>	Load existing cuts/states to initialize policy, but reset iteration count and use fresh RNG seed. Useful for re-running with modified parameters.

Mode	Reads From	Behavior
resume	Everything including <code>metadata.json</code>	Load full algorithm state including RNG, iteration count. Continue exactly where interrupted.

Example Workflows:

1. Fresh Start:

```
{"policy": {"path": "./policy", "mode": "fresh"}}
```

- Creates new `policy/` directory
- Writes cuts/states as training progresses
- On completion, `policy/` contains the final policy

2. Resume After Crash:

```
{"policy": {"path": "./policy", "mode": "resume"}}
```

- Loads `metadata.json` to get iteration count, RNG state
- Continues from last checkpoint
- Updates `policy/` in place

3. Warm-Start for Sensitivity Analysis:

```
cp -r base_case/policy/ sensitivity_case/policy/
```

```
{"policy": {"path": "./policy", "mode": "warm_start"}}
```

- Loads cuts/states but resets iteration counter
- Uses new RNG seed
- Can run with different load/inflow scenarios

4. Extend Horizon:

- Run 60 stages → `policy/` contains cuts for stages 0-59
- Modify `stages.json` to add stages 60-119
- Run with `"mode": "warm_start"` → extends policy with new stages

State Dictionary (`state_dictionary.json`)

Critical for Compatibility: This file defines the mapping between coefficient/component indices and the actual state variables. Without this, it's impossible to correctly interpret `coefficient_0`, `coefficient_1`, etc.

The dictionary follows the **canonical ordering** (see Section 1.3): state variables are ordered by entity type, then by entity ID.

```
{
  "$schema": "https://powers-rs.io/schemas/v2/state_dictionary.schema.json",
  "version": "2.0.0",
  "state_dimension": 320,
  "state_variables": [
    {"index": 0, "type": "storage", "entity_type": "hydro", "entity_id": 0, "name": "FURNAS"},
    {"index": 1, "type": "storage", "entity_type": "hydro", "entity_id": 1, "name": "MARIMBONDO"},
    {"index": 2, "type": "storage", "entity_type": "hydro", "entity_id": 2, "name": "ITUMBIARA"},
    // ... storage for all hydros (canonical order by ID)
    {"index": 160, "type": "inflow_lag_1", "entity_type": "hydro", "entity_id": 0, "name": "FURNAS"},
    {"index": 161, "type": "inflow_lag_1", "entity_type": "hydro", "entity_id": 1, "name": "MARIMBONDO"},
    // ... inflow lags for all hydros (if using inflow as state)
  ],
}
```

```

    "checksum": "sha256:abc123..."
}

```

State Dictionary Fields

Field	Type	Description
index	i32	Column index in cuts/states (coefficient_N, component_N)
type	string	Variable type: "storage", "inflow_lag_1", "inflow_lag_2", ...
entity_type	string	"hydro", "thermal", "bus", etc.
entity_id	i32	Entity ID (matches entity registries)
name	string	Human-readable entity name (for debugging)

Compatibility Validation When loading policy data (warm-start or resume), the system MUST verify:

1. `state_dictionary.json` exists and matches current system
2. Entity IDs in dictionary exist in current system
3. State dimension matches current configuration
4. Checksum matches to detect file corruption

If validation fails, the load is rejected with a clear error message.

Cuts Schema (policy/cuts/stage_XXX.bin - FlatBuffers)

Format: FlatBuffers binary (see Section 7.2.1 for schema). The columnar representation below shows logical fields.

Column	Type	Description
cut_id	i64	Unique cut identifier (unique within stage)
iteration	i32	Iteration when generated
forward_pass_idx	i32	Forward pass index within iteration
scenario_idx	i32	Scenario that generated this cut
rhs	f64	Cut RHS value (c_0)
is_active	bool	Whether cut is active in final model
coefficient_0	f64	Coefficient for state variable 0 (see dictionary)
coefficient_1	f64	Coefficient for state variable 1
...	...	(up to state_dimension - 1)

Interpretation (Standard Benders Notation): A cut for stage t approximates the future cost function:

$$_{t+1} \text{ intercept} + 'x_t$$

Term	Definition
$_{t+1}$	The cost-to-go variable (future cost)
intercept	$\text{rhs} = - ' \hat{x}$ (pre-computed for efficiency) [coefficient_0, coefficient_1, ...] (dual multipliers / subgradient)
x_t	The current state vector
$\hat{x}$	The state at which the cut was generated (stored for cut selection)

Equivalently: $v_{t+1} = v_t + \beta(x_t - \hat{x})$ where β is the value function at generation point.

The coefficient indices map to state variables via `state_dictionary.json`.

Note: The FlatBuffers schema (Section 7.2.1) stores cuts in this exact format for zero-copy loading.

Cut Coefficient Sign Convention

CRITICAL: The cut coefficient sign convention must be consistent across all solver implementations to ensure correct cut interpretation.

Mathematical Derivation:

Consider the water balance constraint at stage t :

$$v_t = v_{t-1} + a_t - u_t - s_t$$

Where:

- v_t = storage at end of stage t (state variable)
- a_t = inflow (uncertain)
- u_t = turbined outflow (decision)
- s_t = spillage (decision)

Rearranging: $v_{t-1} - v_t + a_t - u_t - s_t = 0$

In standard LP form with dual multiplier π_{water} :

$$\pi_{water} \times (v_{t-1} - v_t + a_t - u_t - s_t) = 0$$

The Benders cut coefficient for state variable v_{t-1} is:

$$\beta_v = \frac{\partial Q_t}{\partial v_{t-1}} = \pi_{water} \quad (\text{positive if water is valuable})$$

Sign Convention Rules:

Constraint Type	LP Form	Dual Sign	Cut Coefficient Sign
Water balance (equality)	$v_{t-1} + \dots = 0$	unrestricted	$= +$
Storage upper bound	$v_t \leq v_{max}$	0	$= -$ (positive)
Storage lower bound	$v_t \geq v_{min}$	0	$= +$

Implementation Requirement:

All solver implementations **MUST** normalize dual multipliers such that:

- **Positive** means: increasing the state variable **decreases** future cost (water is valuable)
- **Negative** means: increasing the state variable **increases** future cost (state is costly)

```

/// Extract cut coefficient for storage state variable
fn extract_storage_cut_coefficient(
    water_balance_dual: f64, // Already normalized by solver
) -> f64 {
    // = + for water balance constraint where storage appears
    // with coefficient +1 on v_{t-1}
    water_balance_dual
}

```

Verification Test:

A cut coefficient is correct if:

$$\beta'(x_2 - x_1) \approx V(x_2) - V(x_1)$$

for small perturbations, where V is the value function.

States Schema (policy/states/stage_XXX.bin - FlatBuffers)

Format: FlatBuffers binary (see Section 7.2.1 for schema). The columnar representation below shows logical fields.

Column	Type	Description
state_id	i64	Unique state identifier (unique within stage)
iteration	i32	Iteration when visited
forward_pass_idx	i32	Forward pass index
scenario_idx	i32	Scenario that visited this state
dominating_cut_id	i64	Cut ID currently dominating this state
dominating_objective	f64	Objective value at dominating cut
component_0	f64	State variable 0 value (see dictionary)
component_1	f64	State variable 1 value
...	...	(up to state_dimension - 1)

Vertices Schema (policy/vertices/stage_XXX.bin - FlatBuffers) - Optional

Format: FlatBuffers binary (see Section 7.2.1 for schema). The columnar representation below shows logical fields.

Purpose: Store inner approximation vertices for upper bound computation and inner-policy simulation. Only written when `upper_bound_evaluation.enabled = true`.

Interpretation: A vertex stores the upper-bound cost-to-go value at a visited state point. The inner approximation at a new point is computed via Lipschitz interpolation from nearby vertices.

Column	Type	Description
vertex_id	i64	Unique vertex identifier (unique within stage)
iteration	i32	Iteration when computed
forward_pass_idx	i32	Forward pass index
scenario_idx	i32	Scenario that visited this state
upper_bound_value	f64	Upper bound cost-to-go at this state
component_0	f64	State variable 0 value (see dictionary)
component_1	f64	State variable 1 value
...	...	(up to state_dimension - 1)

Solver Basis Schema (policy/basis/stage_XXX.parquet) - Optional

Purpose: Store solver basis information to achieve exact reproducibility when resuming. Without this, the solver may choose different pivots, leading to different (but equivalent) optimal solutions and thus different cuts.

Trade-off: Storing basis significantly increases policy directory size and I/O time. For most use cases, slight numerical differences are acceptable.

Column	Type	Description
variable_idx	i32	LP variable index
basis_status	i8	Basis status: 0=lower, 1=basic, 2=upper, 3=free, 4=fixed
row_idx	i32	Constraint row index (for row status)
row_status	i8	Row basis status

Note: Basis format is solver-dependent. The implementation should serialize in a generic format and translate to solver-specific format on load.

Metadata (policy/metadata.json)

Purpose: Store algorithm state for resume capability and audit trail. This file is written on every checkpoint and at run completion.

Note: When loading in warm_start mode, only state_dictionary_checksum and state_dimension are used for validation. The algorithm_state section is ignored and reset.

```
{
  "$schema": "https://powers-rs.io/schemas/v2/policy_metadata.schema.json",
  "version": "2.0.0",
  "created_at": "2026-01-15T12:00:00Z",
  "powers_version": "2.0.0",
  "solver": "highs",
  "solver_version": "1.7.0",

  "algorithm_state": {
    "completed_iterations": 25,
    "last_forward_pass": 199,
    "rng_state": "base64:...",
    "final_lower_bound": 1234567.89,
    "best_upper_bound": 1245678.90,
    "gap_percent": 0.89,
    "best_iteration": 23
  },

  "data_integrity": {
    "state_dimension": 320,
    "total_cuts": 500000,
    "active_cuts": 450000,
    "total_states": 500000,
    "total_vertices": 125000,
    "config_hash": "sha256:abc123...",
    "system_hash": "sha256:def456...",
    "state_dictionary_checksum": "sha256:789xyz..."
  },

  "partitioning": {
    "num_stages": 120,
    "cuts_by_stage": [4000, 4200, "..."],
    "states_by_stage": [4000, 4200, "..."],
    "vertices_by_stage": [1000, 1050, "..."],
    "has_basis": true,
    "has_vertices": true
  },
}
```

```

    "reproducibility": {
      "basis_stored": true,
      "exact_resume_supported": true,
      "notes": "Basis stored for all stages. Exact reproducibility expected."
    }
  }
}

```

Metadata Fields

Section	Field	Description
algorithm_state	completed_iterations	Number of iterations completed
	last_forward_pass	Last forward pass index (for scenario indexing)
	rng_state	Serialized RNG state for exact scenario reproducibility
	final_lower_bound	Best lower bound at policy save
	best_upper_bound	Best upper bound (inner approximation or simulation)
data_integrity	config_hash	SHA-256 hash of config.json
	system_hash	SHA-256 hash of system topology + entities
	state_dictionary_checksum	Checksum of state_dictionary.json
	total_vertices	Number of vertices (0 if inner approx disabled)
partitioning	cuts_by_stage	Number of cuts per stage file
	vertices_by_stage	Number of vertices per stage (if inner approx enabled)
	has_basis	Whether solver basis is stored
reproducibility	has_vertices	Whether inner approximation vertices are stored
	basis_stored	Whether basis files exist
	exact_resume_supported	Whether exact reproducibility is possible

Resume Validation On resume (`policy.mode = "resume"`), the loader MUST verify:

1. **Version compatibility:** `powers_version` is compatible with current version
2. **Config hash match:** Current config matches `config_hash` (or explicit override flag)
3. **System hash match:** Current system matches `system_hash` (entities must be identical)
4. **State dictionary match:** Dictionary checksum matches and all entities exist
5. **File completeness:** All partitioned files exist for all stages
6. **Optional basis check:** If `exact_resume_supported = true` and user requests exact resume, verify basis files exist

Warm-Start Validation On warm-start (`policy.mode = "warm_start"`), the loader MUST verify:

1. **State dictionary exists:** `state_dictionary.json` is present
2. **State dimension match:** Dictionary state dimension matches current system
3. **Entity compatibility:** All entity IDs in dictionary exist in current system
4. **Cuts exist:** At least one stage has cuts

Note: `metadata.json` is optional for warm-start. If present, only `data_integrity` section is checked.

Reproducibility Guarantees

Scenario	Reproducibility	Notes
Straight run (no prior policy)	Bit-for-bit identical	Same seed → same results
Resume with basis	Bit-for-bit identical	Solver basis restored → same pivots
Resume without basis	Equivalent optimum	Same optimum, but possibly different dual values → different cuts
Warm-start	Different run	Fresh RNG, may converge differently
Resume with modified config	Not supported	Hash mismatch → error
Resume with modified system	Not supported	Hash mismatch → error

User Guidance: For critical studies requiring exact reproducibility: 1. Enable `policy.checkpointing.store_basis = true` in config 2. Accept the additional I/O overhead (~20-30% larger policy directory) 3. For less critical studies, disable basis storage and accept minor numerical differences

4. Output Data Model

The output data model defines all files produced by POWE.RS during training (policy construction) and simulation (policy evaluation) phases. The design prioritizes:

- **Parallel write efficiency:** MPI ranks write directly to Hive-partitioned directories
- **Query performance:** Columnar Parquet format with partition pruning
- **Interoperability:** Standard formats readable by Python, R, Spark, DuckDB
- **Completeness:** All decision variables, dual values, and diagnostic information

4.1 Directory Structure Overview

```

output/
  simulation/
    _manifest.json
    _SUCCESS
    costs/
      scenario_id=XXXX/data.parquet
    hydros/
      scenario_id=XXXX/data.parquet
    thermals/
      scenario_id=XXXX/data.parquet
    exchanges/
      scenario_id=XXXX/data.parquet
    buses/
      scenario_id=XXXX/data.parquet
    pumping_stations/
      scenario_id=XXXX/data.parquet
    contracts/
      scenario_id=XXXX/data.parquet
    batteries/
      scenario_id=XXXX/data.parquet
    non_controllables/
      scenario_id=XXXX/data.parquet
    inflow_lags/

```

Hive-partitioned simulation results
Mandatory: checksums, row counts, partitions
Marker written on successful completion

Optional: only if pumping stations exist

Optional: only if contracts exist

DEFERRED: Future implementation

DEFERRED: Future implementation

Optional: only if AR order > 0


```

    scenario_id=XXXX/data.parquet
violations/
  generic/
    scenario_id=XXXX/data.parquet

training/
  _manifest.json          # Training phase outputs
  _SUCCESS                # Mandatory: checksums, metadata
  convergence.parquet     # Marker written on successful completion
  timing/                 # Iteration-level convergence data
    iterations.parquet    # Per-iteration timing breakdown
    mpi_ranks.parquet     # Per-rank timing statistics
  dictionaries/
    codes.json            # Categorical code mappings
    bounds.parquet        # Entity bounds by stage/block
    state_dictionary.json  # State space definition
    variables.csv          # Variable metadata
    entities.csv           # Entity metadata
  metadata.json           # Run configuration and system info

```

Note on Optional Files: Files in `simulation/` are only written if the corresponding entities exist in the input model. An empty system with no pumping stations will not have the `pumping_stations/` directory.

4.2 Design Principles

4.2.1 Hive Partitioning Strategy Simulation outputs use Hive-style partitioning by `scenario_id` to enable:

1. **Parallel writes:** Each MPI rank writes exclusively to its assigned scenario partitions
2. **Partition pruning:** Queries filtering by scenario read only relevant files
3. **Incremental updates:** Individual scenarios can be recomputed without rewriting all data

Partition naming convention:

```
{entity_type}/scenario_id={scenario_id:04d}/data.parquet
```

Example paths:

```

simulation/hydros/scenario_id=0001/data.parquet
simulation/hydros/scenario_id=0002/data.parquet
simulation/costs/scenario_id=0001/data.parquet

```

4.2.2 Categorical Encoding All categorical columns use integer codes with mappings defined in `dictionaries/codes.json`. This approach:

- Reduces storage size (i8 vs. variable-length strings)
- Enables efficient filtering and grouping
- Maintains human-readable documentation in the codes file

Column naming convention: Categorical columns end with `_code` suffix (e.g., `operative_state_code`, `storage_binding_code`).

4.2.3 Constraint Violation Handling Constraint violations are handled through two mechanisms:

1. **Slack columns in entity files:** Physical bound violations (e.g., `turbined_slack_m3s`, `outflow_slack_m3s`) appear as dedicated columns with value 0 when no violation occurs
2. **Generic violations file:** User-defined generic constraint violations stored in `violations/generic/`

This hybrid approach keeps entity-specific violations co-located with entity data while centralizing generic constraint violations.

4.2.4 File Naming Conventions

Convention	Example	Rationale
Plural entity names	hydros.parquet, thermals.parquet	Indicates multiple records
Lowercase with underscores data.parquet in partitions	pumping_stations/ scenario_id=0001/data.parquet	Consistent, filesystem-safe Standard Hive convention

4.3 Categorical Code Definitions

The file `training/dictionaries/codes.json` defines all categorical value mappings:

```
{
  "version": "2.0.0",
  "generated_at": "2026-01-18T12:00:00Z",
  "operative_state": {
    "0": "non_existing",
    "1": "filling_dead_volume",
    "2": "operating",
    "3": "decommissioned"
  },
  "storage_binding": {
    "0": "none",
    "1": "min",
    "2": "max",
    "3": "target"
  },
  "contract_type": {
    "0": "import",
    "1": "export"
  },
  "entity_type": {
    "0": "hydro",
    "1": "thermal",
    "2": "bus",
    "3": "line",
    "4": "pumping_station",
    "5": "contract",
    "6": "battery",
    "7": "non_controllable"
  },
  "bound_type": {
    "0": "storage_min",
    "1": "storage_max",
    "2": "turbined_min",
    "3": "turbined_max",
    "4": "outflow_min",
    "5": "outflow_max",
    "6": "generation_min",
    "7": "generation_max",
    "8": "flow_min",
    "9": "flow_max"
  }
}
```

Usage in analysis tools:

```
import json
import polars as pl

# Load code mappings
with open("training/dictionaries/codes.json") as f:
    codes = json.load(f)

# Decode categorical columns
df = pl.read_parquet("simulation/hydros/")
df = df.with_columns(
    pl.col("operative_state_code")
      .map_dict({int(k): v for k, v in codes["operative_state"].items()})
      .alias("operative_state")
)
```

4.4 Dictionary Files

4.4.1 Bounds Dictionary (training/dictionaries/bounds.parquet) Centralizes all entity bounds by stage and block, eliminating redundant bound columns from entity output files.

Schema:

Column	Type	Nullable	Description
entity_type_code	i8	No	Entity type code (see <code>codes.json</code>)
entity_id	i32	No	Entity identifier
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index (null = applies to all blocks)
bound_type_code	i8	No	Bound type code (see <code>codes.json</code>)
bound_value	f64	No	Bound value in native units

Example data:

entity_type_code	entity_id	stage_id	block_id	bound_type_code	bound_value
0	1	0	null	0	500.0
0	1	0	null	1	12000.0
0	1	0	0	2	100.0
0	1	0	0	3	1500.0
1	5	0	0	6	0.0
1	5	0	0	7	500.0

Notes:

- When `block_id` is null, the bound applies to all blocks in the stage
- Bounds are stored only when they differ from default/infinite values
- Entity type 0 = hydro, 1 = thermal, 3 = line (see `entity_type` in `codes.json`)

4.4.2 State Dictionary (training/dictionaries/state_dictionary.json) Documents the state space structure for the SDDP policy. See Section 3.14 for full schema.

4.4.3 Variables Metadata (training/dictionaries/variables.csv) Provides metadata for all output variables across entity files.

Column	Type	Description
file	string	Source file (e.g., hydros , thermals)
column	string	Column name
type	string	Data type (i8 , i32 , i64 , f64 , bool)
unit	string	Physical unit or null
description	string	Human-readable description
nullable	bool	Whether null values are allowed

4.4.4 Entities Metadata (training/dictionaries/entities.csv) Maps entity IDs to names and properties.

Column	Type	Description
entity_type_code	i8	Entity type code
entity_id	i32	Entity identifier
name	string	Entity name from input
bus_id	i32	Connected bus (if applicable)
system_id	i32	System/subsystem identifier

4.5 Simulation Output Schemas

All simulation files are Hive-partitioned by **scenario_id**. The **scenario_id** column is NOT stored in the Parquet data (it is derived from the partition path).

4.5.1 Costs (simulation/costs/) Stage and block-level cost breakdown for economic analysis.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index (null for stage-level aggregates)
total_cost	f64	No	Total stage cost (sum of all components)
immediate_cost	f64	No	Stage immediate cost (excluding future cost)
thermal_cost	f64	No	Thermal generation cost
deficit_cost	f64	No	Deficit (unmet demand) penalty
excess_cost	f64	No	Excess generation penalty
spillage_cost	f64	No	Spillage penalty (all hydros)
exchange_cost	f64	No	Exchange losses and tariffs
pumping_cost	f64	No	Pumping energy cost
contract_cost	f64	No	Import/export contract cost
violation_cost	f64	No	Generic constraint violation penalties
future_cost	f64	No	Future cost function value ()
discount_factor	f64	No	Cumulative discount factor for this stage

Row count per scenario: $\text{num_stages} \times (1 + \text{num_blocks})$ (stage-level + block-level rows)

Cost relationship:

```
total_cost = immediate_cost + future_cost
immediate_cost = thermal_cost + deficit_cost + excess_cost + spillage_cost
                + exchange_cost + pumping_cost + contract_cost + violation_cost
```

4.5.2 Hydros (simulation/hydros/) Hydroelectric plant operational results including water values.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index (null for stage-level)
hydro_id	i32	No	Hydro plant identifier
turbined_m3s	f64	No	Turbined outflow (m ³ /s)
spillage_m3s	f64	No	Spillage (m ³ /s)
outflow_m3s	f64	No	Total outflow: turbined + spillage (m ³ /s)
evaporation_m3s	f64	Yes	Evaporation loss (m ³ /s), null if not modeled
diverted_inflow_m3s	f64	Yes	Inflow diverted from upstream plants
diverted_outflow_m3s	f64	Yes	Outflow diverted to downstream plants
incremental_inflow_m3s	f64	No	Realized incremental inflow (m ³ /s)
inflow_m3s	f64	No	Total inflow including upstream contributions
storage_initial_hm3	f64	No	Storage at stage/block start (hm ³)
storage_final_hm3	f64	No	Storage at stage/block end (hm ³)
generation_mw	f64	No	Power generation (MW)
generation_mwh	f64	No	Energy generation (MWh)
productivity_mw_per_m3s	f64	Yes	Effective productivity (MW per m ³ /s)
turbined_slack_m3s	f64	No	Minimum turbined violation (0 if none)
outflow_slack_m3s	f64	No	Minimum outflow violation (0 if none)
generation_slack_mw	f64	No	Minimum generation violation (0 if none)
spillage_cost	f64	No	Spillage penalty cost
water_value_per_hm3	f64	No	Marginal value of stored water (\$/hm ³)
storage_binding_code	i8	No	Storage bound binding status (see codes.json)
operative_state_code	i8	No	Operative state (see codes.json)

Row count per scenario: $\text{num_stages} \times \text{num_blocks} \times \text{num_hydros}$

Water balance equation:

$\text{storage_final} = \text{storage_initial} + (\text{inflow} - \text{outflow} - \text{evaporation} + \text{diverted_inflow} - \text{diverted_outflow}) \times \Delta t$

Slack column interpretation:

- $\text{turbined_slack_m3s} > 0$: Minimum turbined constraint was relaxed

- `outflow_slack_m3s > 0`: Minimum outflow (ecological flow) constraint was relaxed
- `generation_slack_mw > 0`: Minimum generation constraint was relaxed

4.5.3 Thermals (`simulation/thermals/`) Thermal generation unit results including GNL (Gas Natural Liquefado) commitment.

Column	Type	Nullable	Description
<code>stage_id</code>	i32	No	Stage index (0-based)
<code>block_id</code>	i32	Yes	Block index
<code>thermal_id</code>	i32	No	Thermal unit identifier
<code>generation_mw</code>	f64	No	Power generation (MW)
<code>generation_mwh</code>	f64	No	Energy generation (MWh)
<code>generation_cost</code>	f64	No	Generation cost
<code>is_gnl</code>	bool	No	Whether unit has GNL configuration
<code>gnl_committed_mw</code>	f64	Yes	GNL committed capacity for this stage (null if not GNL)
<code>gnl_decision_mw</code>	f64	Yes	GNL decision made this stage for future (null if not GNL)
<code>operative_state_code</code>	i8	No	Operative state (see codes.json)

Row count per scenario: `num_stages × num_blocks × num_thermals`

GNL modeling notes:

- `gnl_committed_mw`: Capacity committed in a previous stage, available this stage
- `gnl_decision_mw`: Decision made this stage that will be available in future stages
- GNL decisions are state variables that couple stages

4.5.4 Exchanges (`simulation/exchanges/`) Transmission line flow results.

Column	Type	Nullable	Description
<code>stage_id</code>	i32	No	Stage index (0-based)
<code>block_id</code>	i32	Yes	Block index
<code>line_id</code>	i32	No	Transmission line identifier
<code>net_flow_mw</code>	f64	No	Net flow: direct - reverse (MW)
<code>net_flow_mwh</code>	f64	No	Net energy flow (MWh)
<code>exchange_cost</code>	f64	No	Exchange cost (losses + tariffs)
<code>operative_state_code</code>	i8	No	Operative state (see codes.json)

Row count per scenario: `num_stages × num_blocks × num_lines`

Sign convention:

- Positive `net_flow_mw`: Flow from `bus_from` to `bus_to`
- Negative `net_flow_mw`: Flow from `bus_to` to `bus_from`

4.5.5 Buses (`simulation/buses/`) Bus-level load balance results. This schema is simplified to contain only bus-specific variables; generation by source is available in entity-specific files.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index
bus_id	i32	No	Bus identifier
load_mw	f64	No	Realized load after curtailment (MW)
load_mwh	f64	No	Realized load energy (MWh)
deficit_mw	f64	No	Unmet demand (MW)
deficit_mwh	f64	No	Unmet demand energy (MWh)
excess_mw	f64	No	Excess generation (MW)
excess_mwh	f64	No	Excess generation energy (MWh)
spot_price	f64	No	Marginal cost of energy (\$/MWh)

Row count per scenario: num_stages × num_blocks × num_buses

Load balance equation:

generation_total + imports - exports + deficit - excess = load

Note: To compute generation by source at each bus, join with `hydros`, `thermals`, and other entity files using `bus_id` from `entities.csv`.

4.5.6 Pumping Stations (simulation/pumping_stations/) — Optional Pumping station operational results. Only generated if pumping stations exist in the system.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index
pumping_station_id	i32	No	Pumping station identifier
pumped_flow_m3s	f64	No	Pumped water flow (m ³ /s)
pumped_volume_hm3	f64	No	Pumped volume (hm ³)
power_consumption_mw	f64	No	Power consumed (MW)
energy_consumption_mwh	f64	No	Energy consumed (MWh)
pumping_cost	f64	No	Total pumping cost
operative_state_code	i8	No	Operative state (see codes.json)

Row count per scenario: num_stages × num_blocks × num_pumping_stations

4.5.7 Contracts (simulation/contracts/) — Optional Import/export contract results. Only generated if contracts exist in the system.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index
contract_id	i32	No	Contract identifier
contract_type_code	i8	No	Contract type: 0=import, 1=export
power_mw	f64	No	Contracted power (MW)
energy_mwh	f64	No	Contracted energy (MWh)
price_per_mwh	f64	No	Contract price (\$/MWh)

Column	Type	Nullable	Description
total_cost	f64	No	Total contract cost
operative_state_code	i8	No	Operative state (see codes.json)

Row count per scenario: num_stages × num_blocks × num_contracts

4.5.8 Batteries (simulation/batteries/) — DEFERRED

Implementation Status: This entity type is planned for future implementation. The schema is documented here for completeness but the output is not currently generated.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index
battery_id	i32	No	Battery identifier
charge_mw	f64	No	Charging power (MW)
discharge_mw	f64	No	Discharging power (MW)
soc_initial_mwh	f64	No	State of charge at start (MWh)
soc_final_mwh	f64	No	State of charge at end (MWh)
cycle_cost	f64	No	Cycling degradation cost
operative_state_code	i8	No	Operative state (see codes.json)

4.5.9 Non-Controllables (simulation/non_controllables/) — DEFERRED

Implementation Status: This entity type is planned for future implementation. The schema is documented here for completeness but the output is not currently generated.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
block_id	i32	Yes	Block index
non_controllable_id	i32	No	Non-controllable source identifier
generation_mw	f64	No	Realized generation (MW)
generation_mwh	f64	No	Realized generation (MWh)
curtailment_mw	f64	No	Curtailed generation (MW)
curtailment_mwh	f64	No	Curtailed generation (MWh)
operative_state_code	i8	No	Operative state (see codes.json)

4.5.10 Inflow Lags (simulation/inflow_lags/) — Optional Autoregressive inflow lag values. Only generated when AR order > 0.

Column	Type	Nullable	Description
stage_id	i32	No	Stage index (0-based)
hydro_id	i32	No	Hydro plant identifier
lag_index	i32	No	Lag index (1 = t-1, 2 = t-2, ...)
inflow_m3s	f64	No	Inflow value for this lag (m ³ /s)

Row count per scenario: num_stages × num_hydros × max_ar_order

Notes:

- `lag_index` uses 1-based indexing: lag 1 is t-1, lag 2 is t-2, etc.
- Maximum lag index equals the AR model order
- These values are state variables that affect inflow sampling

4.5.11 Generic Violations (`simulation/violations/generic/`) Generic constraint violation results.

Column	Type	Nullable	Description
<code>stage_id</code>	i32	No	Stage index (0-based)
<code>block_id</code>	i32	Yes	Block index
<code>constraint_id</code>	i32	No	Generic constraint identifier
<code>slack_value</code>	f64	No	Violation amount (in constraint units)
<code>slack_cost</code>	f64	No	Penalty cost incurred

Row count per scenario: `num_stages` × `num_blocks` × `num_generic_constraints` (only non-zero violations may be stored)

4.6 Training Output Schemas

Training outputs capture SDDP algorithm convergence, cut generation, and MPI performance metrics.

4.6.1 Convergence Log (`training/convergence.parquet`) Iteration-level convergence metrics for the SDDP training process.

Column	Type	Nullable	Description
<code>iteration</code>	i32	No	Iteration number (1-based)
<code>lower_bound</code>	f64	No	Lower bound (expected cost-to-go from stage 0)
<code>upper_bound_mean</code>	f64	Yes	Upper bound mean (null if UB evaluation disabled)
<code>upper_bound_std</code>	f64	Yes	Upper bound standard deviation
<code>gap_percent</code>	f64	No	Optimality gap: $(UB - LB) / LB * 100$
<code>cuts_added</code>	i32	No	Cuts added this iteration
<code>cuts_removed</code>	i32	No	Cuts removed by cut selection
<code>cuts_active</code>	i64	No	Total active cuts across all stages
<code>time_forward_ms</code>	i64	No	Forward pass wall time (ms)
<code>time_backward_ms</code>	i64	No	Backward pass wall time (ms)
<code>time_total_ms</code>	i64	No	Total iteration wall time (ms)
<code>memory_peak_mb</code>	i64	No	Peak memory usage during iteration (MB)
<code>forward_passes</code>	i32	No	Number of forward scenarios this iteration
<code>lp_solves</code>	i64	No	Total LP solves this iteration

Row count: `num_iterations`

Notes:

- `gap_percent` is computed as $(\text{upper_bound_mean} - \text{lower_bound}) / \text{abs}(\text{lower_bound}) * 100$ when upper bound is available
- If upper bound evaluation is disabled, `gap_percent` shows gap from previous iteration's lower bound
- `cuts_active` is the sum across all stages (useful for monitoring memory growth)

4.6.2 Iteration Timing (`training/timing/iterations.parquet`) Detailed timing breakdown per iteration (production default: always written).

Column	Type	Nullable	Description
<code>iteration</code>	i32	No	Iteration number (1-based)
<code>forward_solve_ms</code>	i64	No	LP solve time in forward pass (ms)
<code>forward_sample_ms</code>	i64	No	Scenario sampling time (ms)
<code>backward_solve_ms</code>	i64	No	LP solve time in backward pass (ms)
<code>backward_cut_ms</code>	i64	No	Cut computation and storage time (ms)
<code>cut_selection_ms</code>	i64	No	Cut selection/pruning time (ms)
<code>mpi_allreduce_ms</code>	i64	No	MPI AllReduce communication time (ms)
<code>mpi_broadcast_ms</code>	i64	No	MPI Broadcast communication time (ms)
<code>io_write_ms</code>	i64	No	Output writing time (ms)
<code>overhead_ms</code>	i64	No	Unaccounted overhead (ms)

Row count: `num_iterations`

4.6.3 MPI Rank Timing (`training/timing/mpi_ranks.parquet`) Per-rank timing for load balancing analysis in distributed training.

Column	Type	Nullable	Description
<code>iteration</code>	i32	No	Iteration number (1-based)
<code>rank</code>	i32	No	MPI rank (0-based)
<code>forward_time_ms</code>	i64	No	Forward pass time on this rank (ms)
<code>backward_time_ms</code>	i64	No	Backward pass time on this rank (ms)
<code>communication_time_ms</code>	i64	No	MPI communication time (ms)
<code>idle_time_ms</code>	i64	No	Time waiting for other ranks (ms)
<code>lp_solves</code>	i64	No	LP solves executed on this rank
<code>scenarios_processed</code>	i32	No	Scenarios processed on this rank

Row count: `num_iterations * num_mpi_ranks`

Notes:

- Use this data to identify load imbalance (high `idle_time_ms` on some ranks)
- Sum of `scenarios_processed` per iteration equals `forward_passes`
- Communication patterns reveal MPI bottlenecks

4.7 Manifest Files

Manifest files enable crash recovery and incremental writes. They track completion status and are updated atomically.

4.7.1 Simulation Manifest (simulation/_manifest.json)

```
{
  "$schema": "https://powers-rs.io/schemas/v2/simulation_manifest.schema.json",
  "version": "2.0.0",
  "status": "complete",
  "started_at": "2026-01-17T10:00:00Z",
  "completed_at": "2026-01-17T10:15:00Z",
  "scenarios": {
    "total": 2000,
    "completed": 2000,
    "failed": 0
  },
  "partitions_written": [
    "scenario_id=0/",
    "scenario_id=1/",
    "..."
  ],
  "checksum": {
    "algorithm": "xxhash64",
    "value": "a1b2c3d4e5f6"
  },
  "mpi_info": {
    "world_size": 128,
    "ranks_participated": 128
  }
}
```

Field	Type	Description
status	string	"running", "complete", "failed", "partial"
started_at	string	ISO 8601 timestamp
completed_at	string	ISO 8601 timestamp (null if not complete)
scenarios.total	i32	Total scenarios to simulate
scenarios.completed	i32	Successfully completed scenarios
scenarios.failed	i32	Failed scenarios
partitions_written	array	List of Hive partition directories written
checksum	object	Integrity checksum for validation
mpi_info.world_size	i32	Number of MPI ranks
mpi_info.ranks_participated	i32	Ranks that wrote data

Crash Recovery Protocol:

1. On startup, check if _manifest.json exists with status: "running"
2. If found, read partitions_written to identify completed work
3. Resume from incomplete scenarios
4. Update manifest atomically on completion

4.7.2 Training Manifest (training/_manifest.json)

```
{
  "$schema": "https://powers-rs.io/schemas/v2/training_manifest.schema.json",
  "version": "2.0.0",
  "status": "complete",
  "started_at": "2026-01-17T08:00:00Z",
  "completed_at": "2026-01-17T12:30:00Z",
}
```

```

"iterations": {
  "target": 100,
  "completed": 100,
  "converged_at": 87
},
"convergence": {
  "achieved": true,
  "final_gap_percent": 0.45,
  "termination_reason": "gap_tolerance"
},
"cuts": {
  "total_generated": 1250000,
  "total_active": 980000,
  "peak_active": 1100000
},
"checksum": {
  "algorithm": "xxhash64",
  "policy_value": "f1e2d3c4b5a6",
  "convergence_value": "1a2b3c4d5e6f"
},
"mpi_info": {
  "world_size": 128,
  "forward_passes_per_iteration": 8
}
}

```

Field	Type	Description
status	string	"running", "complete", "failed", "converged"
iterations.target	i32	Maximum iterations configured
iterations.completed	i32	Iterations actually run
iterations.converged_at	i32	Iteration where convergence achieved (null if not converged)
convergence.achieved	bool	Whether gap tolerance was reached
convergence.final_gap_percent	f64	Final optimality gap
convergence.termination_reason	string	"gap_tolerance", "max_iterations", "time_limit", "user_interrupt"
cuts.total_generated	i64	Total cuts generated during training
cuts.total_active	i64	Active cuts at termination
cuts.peak_active	i64	Peak active cuts during training

4.8 Metadata File (training/metadata.json)

Comprehensive metadata for reproducibility, audit trails, and debugging.

```

{
  "$schema": "https://powers-rs.io/schemas/v2/training_metadata.schema.json",
  "version": "2.0.0",
  "run_info": {
    "run_id": "uuid-v4-here",
    "started_at": "2026-01-17T08:00:00Z",
    "completed_at": "2026-01-17T12:30:00Z",
    "duration_seconds": 16200,
    "powers_version": "2.0.0",

```

```

    "solver": "highs",
    "solver_version": "1.7.2",
    "hostname": "compute-node-001",
    "user": "scheduler"
  },
  "configuration_snapshot": {
    "num_iterations": 100,
    "num_forward_passes": 8,
    "convergence_tolerance": 0.5,
    "cut_selection": {
      "enabled": true,
      "strategy": "level_one",
      "max_cuts_per_stage": 10000
    },
    "upper_bound": {
      "enabled": true,
      "frequency": 10,
      "num_scenarios": 1000
    },
    "policy_mode": "fresh",
    "seed": 42
  },
  "problem_dimensions": {
    "num_stages": 120,
    "num_blocks_per_stage": [730, 730, 672, 744, 720, 744, 720, 744, 744, 720, 744, 720],
    "num_hydros": 160,
    "num_thermals": 200,
    "num_buses": 5,
    "num_lines": 8,
    "num_pumping_stations": 3,
    "num_contracts": 2,
    "num_generic_constraints": 15,
    "state_dimension": 320,
    "lp_dimensions": {
      "variables_per_stage_avg": 1500,
      "constraints_per_stage_avg": 2000,
      "nonzeros_per_stage_avg": 8500
    }
  },
  "performance_summary": {
    "total_lp_solves": 125000000,
    "avg_lp_time_us": 145,
    "median_lp_time_us": 132,
    "p99_lp_time_us": 450,
    "peak_memory_mb": 16384,
    "total_communication_time_seconds": 850,
    "io_write_time_seconds": 45
  },
  "data_integrity": {
    "input_hash": "sha256:abc123...",
    "config_hash": "sha256:def456...",
    "policy_hash": "sha256:789xyz...",
    "convergence_hash": "sha256:uvw012..."
  },
  "environment": {
    "mpi_implementation": "OpenMPI",
    "mpi_version": "4.1.5",

```

```

    "num_ranks": 128,
    "cpus_per_rank": 4,
    "memory_per_rank_gb": 32,
    "numa_binding": true,
    "omp_num_threads": 1
}
}

```

4.9 MPI Direct Hive Partitioning

The output system uses MPI-native Hive-style partitioning where each rank writes directly to partition directories without coordination.

4.9.1 Writing Strategy

```

simulation/
  costs/
    scenario_id=0/data.parquet    # Written by rank 0
    scenario_id=1/data.parquet    # Written by rank 0
    scenario_id=2/data.parquet    # Written by rank 1
    ...
  hydros/
    scenario_id=0/data.parquet
    ...
  _manifest.json                  # Written by rank 0 only

```

Scenario Assignment:

- Scenarios are distributed round-robin across ranks: $\text{rank} = \text{scenario_id} \% \text{world_size}$
- Each rank writes only its assigned scenarios
- No inter-rank coordination during writes (embarrassingly parallel)

4.9.2 Write Protocol

```

// Pseudo-code for MPI Hive-partitioned writes
fn write_simulation_results(results: &SimulationResults, config: &OutputConfig) {
    let rank = mpi::comm_world().rank();
    let world_size = mpi::comm_world().size();

    // Each rank writes its assigned scenarios
    for scenario_id in (rank..num_scenarios).step_by(world_size) {
        let partition_path = format!(
            "{}scenario_id={}/data.parquet",
            config.simulation_path,
            scenario_id
        );

        // Write Parquet file (no coordination needed)
        write_parquet(&results[scenario_id], &partition_path)?;
    }

    // Barrier before manifest write
    mpi::comm_world().barrier();

    // Only rank 0 writes manifest
    if rank == 0 {
        write_manifest(&manifest)?;
    }
}

```

```

    }
}

```

4.9.3 Failure Handling

Failure Type	Detection	Recovery
Rank crash mid-write	Missing partitions in manifest	Re-run failed scenarios only
Partial file write	Parquet read failure	Delete and re-write partition
Manifest corruption	JSON parse error	Rebuild from partition listing
Disk full	Write error	Alert, do not corrupt existing data

Atomic Write Pattern:

1. Write to temporary file: `data.parquet.tmp`
2. Sync to disk: `fsync()`
3. Atomic rename: `rename("data.parquet.tmp", "data.parquet")`

4.9.4 Reading Partitioned Data

```

# Python example using PyArrow
import pyarrow.parquet as pq
import pyarrow.dataset as ds

# Read all scenarios (automatic partition discovery)
dataset = ds.dataset(
    "simulation/hydros/",
    format="parquet",
    partitioning="hive"
)
table = dataset.to_table()

# Filter to specific scenarios
table = dataset.to_table(filter=ds.field("scenario_id") < 100)

# Read single scenario
single = pq.read_table("simulation/hydros/scenario_id=42/data.parquet")

// Rust example using polars
use polars::prelude::*;

// Read all partitions with lazy evaluation
let df = LazyFrame::scan_parquet(
    "simulation/hydros/**/data.parquet",
    ScanArgsParquet::default()
)?
.with_column(
    // Extract scenario_id from path if needed
    col("scenario_id")
)
.collect()?;

// Filter during scan (partition pruning)
let df = LazyFrame::scan_parquet(
    "simulation/hydros/scenario_id=42/data.parquet",
    ScanArgsParquet::default()

```

```

)?
.collect()?;

```

4.10 Output Configuration

Control output generation via `config.json` settings.

```

{
  "output": {
    "simulation_path": "./simulation",
    "training_path": "./training",
    "simulation": {
      "enabled": true,
      "entities": {
        "costs": true,
        "hydros": true,
        "thermals": true,
        "exchanges": true,
        "buses": true,
        "pumping_stations": true,
        "contracts": true,
        "batteries": false,
        "non_controllables": false,
        "inflow_lags": true,
        "violations": true
      },
      "compression": "zstd",
      "compression_level": 3
    },
    "training": {
      "enabled": true,
      "convergence": true,
      "timing": {
        "iterations": true,
        "mpi_ranks": true
      },
      "compression": "snappy"
    },
    "dictionaries": {
      "enabled": true,
      "codes": true,
      "bounds": true,
      "state_dictionary": true,
      "variables": true,
      "entities": true
    }
  }
}

```

Field	Type	Default	Description
<code>simulation_path</code>	string	<code>"./simulation"</code>	Simulation output directory
<code>training_path</code>	string	<code>"./training"</code>	Training output directory
<code>simulation.enabled</code>	bool	<code>true</code>	Enable simulation outputs
<code>simulation.entities.enabled</code>	bool	varies	Per-entity output control
<code>simulation.compression</code>	string	<code>"zstd"</code>	Parquet compression codec

Field	Type	Default	Description
<code>simulation.compression_level</code>	int	3	Compression level (codec-specific)
<code>training.enabled</code>	bool	true	Enable training outputs
<code>training.timing.iterations</code>	bool	true	Write iteration timing
<code>training.timing.mpi_ranks</code>	bool	true	Write per-rank timing
<code>dictionaries.enabled</code>	bool	true	Write dictionary files

Compression Options: | Codec | Speed | Ratio | Use Case | |———|———|———|———| | **none** | Fastest | 1.0x | Temporary/debugging | | **snappy** | Fast | ~2x | Training logs (frequent writes) | | **zstd** | Medium | ~4x | Simulation outputs (recommended) | | **gzip** | Slow | ~3.5x | Archival/compatibility |

4.11 Production Scale Reference

Reference sizes for production-scale SDDP runs (Brazilian interconnected system scale).

4.11.1 Typical Problem Dimensions

Dimension	Small	Medium	Large	Extra Large
Stages	60	120	360	600
Hydros	50	160	200	250
Thermals	100	200	300	400
Buses	4	5	8	12
Scenarios (sim)	200	2,000	5,000	10,000
Iterations	50	100	200	500
Forward passes	4	8	16	32
MPI ranks	16	128	512	2,048

4.11.2 Output Size Estimates

Output	Small	Medium	Large	Extra Large
<code>simulation/costs/</code>	50 MB	800 MB	4 GB	20 GB
<code>simulation/hydros/</code>	200 MB	5 GB	30 GB	150 GB
<code>simulation/thermals/</code>	150 MB	4 GB	25 GB	120 GB
<code>training/convergence.parquet</code>	10 KB	50 KB	100 KB	250 KB
<code>training/timing/</code>	1 MB	15 MB	120 MB	1.2 GB
<code>policy/ (cuts)</code>	500 MB	8 GB	40 GB	200 GB
Total	~1 GB	~20 GB	~100 GB	~500 GB

Storage Recommendations:

- Use SSD/NVMe for training (frequent random writes)
- Network filesystem acceptable for simulation (sequential writes)
- Consider parallel filesystem (Lustre, GPFS) for >100 GB outputs
- Enable compression for network transfers

4.11.3 I/O Bandwidth Requirements

Scale	Write Throughput	Duration	Bottleneck
Small	50 MB/s	20s	None
Medium	200 MB/s	100s	Network
Large	500 MB/s	200s	Filesystem
Extra Large	1+ GB/s	500s	Parallel FS

4.12 Validation and Integrity

4.12.1 Schema Validation All Parquet outputs can be validated against JSON Schema definitions:

```
# Validate simulation output schema
powers validate-output --type simulation --path ./simulation/

# Validate training output schema
powers validate-output --type training --path ./training/

# Validate specific entity
powers validate-output --type simulation --entity hydros --path ./simulation/hydros/
```

4.12.2 Data Integrity Checks

Check	Method	Frequency
Parquet file integrity	Footer checksum	On read
Partition completeness	Manifest comparison	Post-run
Row count consistency	Cross-entity validation	Post-run
Value range validation	Min/max from bounds.parquet	Optional

Cross-Entity Validation:

```
# Verify consistent row counts across entities
def validate_scenario(scenario_id: int) -> bool:
    costs = pq.read_table(f"simulation/costs/scenario_id={scenario_id}/")
    hydros = pq.read_table(f"simulation/hydros/scenario_id={scenario_id}/")

    # Costs should have num_stages rows
    expected_stages = costs.num_rows

    # Hydros should have num_stages * num_blocks * num_hydros rows
    # (accounting for nullable block_id for stage-level data)
    return validate_row_counts(costs, hydros, expected_stages)
```

4.12.3 Reproducibility Verification The `data_integrity` section in `metadata.json` enables reproducibility verification:

```
# Verify inputs haven't changed since training
powers verify-inputs --metadata training/metadata.json --input-dir ./input/

# Compare two training runs
powers diff-runs --run1 ./training_v1/ --run2 ./training_v2/
```

Hash Computation:

- input_hash: SHA-256 of concatenated input file hashes
 - config_hash: SHA-256 of normalized config.json
 - policy_hash: SHA-256 of policy/cuts.parquet content
 - convergence_hash: SHA-256 of training/convergence.parquet content
-

5. Internal Data Structures

5.1 Core Algorithm Structures

Order Invariance: All entity vectors must be sorted by ID after loading (see Section 1.3). The `canonicalize()` method must be called before any processing.

```

/// Production-scale system representation
///
/// # Order Invariance
/// After loading from input files, `canonicalize()` MUST be called to sort
/// all entity collections by ID. This ensures deterministic behavior regardless
/// of declaration order in input files.
pub struct System {
    pub buses: Vec<Bus>,
    pub lines: Vec<Line>,
    pub thermals: Vec<Thermal>,
    pub hydros: Vec<Hydro>,
    pub meta: SystemMeta,
}

impl System {
    /// Sort all entity collections by ID for order-invariant processing.
    /// MUST be called after loading and before any algorithm execution.
    pub fn canonicalize(&mut self) {
        self.buses.sort_by_key(|b| b.id);
        self.lines.sort_by_key(|l| l.id);
        self.hydros.sort_by_key(|h| h.id);
        self.thermals.sort_by_key(|t| t.id);
    }
}

/// Operative state for entities (computed per stage)
#[derive(Clone, Copy, PartialEq, Eq)]
pub enum OperativeState {
    NonExisting,    // Before entry or after exit - no LP variables
    Filling,        // Hydro only: reservoir filling, no generation
    Operating,      // Normal operation
    Decommissioned, // After exit - no LP variables
}

/// Hydro with full production features
pub struct Hydro {
    pub id: u32,
    pub name: String,
    pub bus_id: u32,
    pub downstream_id: Option<u32>,
    pub upstream_ids: Vec<u32>,

    // Entry/exit for system changes modeling
    pub entry_stage_id: Option<i32>, // None = exists from start
    pub exit_stage_id: Option<i32>,  // None = exists until end
}

```

```

    // Dead-volume filling (for plants entering later)
    pub filling: Option<FillingConfig>,

    // Diversion channel (optional)
    pub diversion: Option<DiversionConfig>,

    // Base reservoir bounds (can be overridden per stage)
    pub base_min_storage: f64,
    pub base_max_storage: f64,

    // Base outflow bounds (can be overridden per stage)
    pub base_min_outflow: f64,
    pub base_max_outflow: Option<f64>, // None = unlimited

    // Generation (currently constant_productivity, extensible)
    pub generation: HydroGeneration,
}

impl Hydro {
    /// Compute operative state for a given stage
    pub fn operative_state(&self, stage_id: i32) -> OperativeState {
        let entry = self.entry_stage_id.unwrap_or(i32::MIN);
        let exit = self.exit_stage_id.unwrap_or(i32::MAX);
        let filling_start = self.filling.as_ref().map(|f| f.start_stage_id);

        if stage_id > exit {
            OperativeState::Decommissioned
        } else if stage_id >= entry {
            OperativeState::Operating
        } else if let Some(start) = filling_start {
            if stage_id >= start {
                OperativeState::Filling
            } else {
                OperativeState::NonExisting
            }
        } else {
            OperativeState::NonExisting
        }
    }
}

/// Dead-volume filling configuration
pub struct FillingConfig {
    pub start_stage_id: i32,
    pub target_storage_hm3: f64,
}

/// Diversion channel configuration
pub struct DiversionConfig {
    pub downstream_id: u32, // Hydro receiving diverted water
    pub max_flow_m3s: f64, // Maximum diversion flow
}

/// Penalty tables loaded from Parquet files
/// All penalties are explicitly declared per entity × stage
pub struct PenaltyTables {

```

```

    /// Bus penalties indexed by (bus_id, stage_id)
    pub bus_penalties: HashMap<(u32, i32), BusPenalties>,
    /// Hydro penalties indexed by (hydro_id, stage_id)
    pub hydro_penalties: HashMap<(u32, i32), HydroPenalties>,
}

/// Bus penalties (from bus_penalties.parquet)
#[derive(Clone, Copy)]
pub struct BusPenalties {
    pub deficit_cost: f64,    // $/MWh
    pub excess_cost: f64,    // $/MWh
}

/// Hydro penalties (from hydro_penalties.parquet)
#[derive(Clone, Copy)]
pub struct HydroPenalties {
    pub spillage_cost: f64,    // $/(m³/s·h) - opportunity cost
    pub diversion_cost: f64,   // $/(m³/s·h) - diversion opportunity cost
    pub turbined_violation_cost: f64, // $/(m³/s·h) - min turbined violation
    pub outflow_violation_cost: f64, // $/(m³/s·h) - outflow bounds violation
    pub generation_violation_cost: f64, // $/MWh - min generation violation
}

impl PenaltyTables {
    /// Get bus penalties for a specific stage
    pub fn bus(&self, bus_id: u32, stage_id: i32) -> Option<&BusPenalties> {
        self.bus_penalties.get(&(bus_id, stage_id))
    }

    /// Get hydro penalties for a specific stage
    pub fn hydro(&self, hydro_id: u32, stage_id: i32) -> Option<&HydroPenalties> {
        self.hydro_penalties.get(&(hydro_id, stage_id))
    }
}

// =====
// Generic Constraints
// =====

/// Variable reference in a generic constraint expression
#[derive(Clone, Debug, PartialEq)]
pub enum VariableRef {
    HydroStorage { hydro_id: u32 },
    HydroTurbined { hydro_id: u32, block_id: Option<u32> },
    HydroSpillage { hydro_id: u32, block_id: Option<u32> },
    HydroOutflow { hydro_id: u32, block_id: Option<u32> },
    HydroGeneration { hydro_id: u32, block_id: Option<u32> },
    ThermalGeneration { thermal_id: u32, block_id: Option<u32> },
    LineDirect { line_id: u32, block_id: Option<u32> },
    LineReverse { line_id: u32, block_id: Option<u32> },
    BusDeficit { bus_id: u32, block_id: Option<u32> },
    BusExcess { bus_id: u32, block_id: Option<u32> },
}

/// A term in a linear expression: coefficient × variable
#[derive(Clone, Debug)]
pub struct LinearTerm {

```

```

    pub coefficient: f64,
    pub variable: VariableRef,
}

/// Parsed linear expression (sum of terms plus constant)
#[derive(Clone, Debug)]
pub struct LinearExpression {
    pub terms: Vec<LinearTerm>,
    pub constant: f64,
}

/// Constraint sense
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum ConstraintSense {
    GreaterEqual,    // >=
    LessEqual,       // <=
    Equal,           // ==
}

/// Slack configuration for a constraint
#[derive(Clone, Debug)]
pub struct SlackConfig {
    pub enabled: bool,
    pub penalty: f64,    // Only used if enabled
}

/// A generic constraint definition (parsed from JSON)
#[derive(Clone, Debug)]
pub struct GenericConstraint {
    pub id: u32,
    pub name: String,
    pub description: Option<String>,
    pub expression: LinearExpression,
    pub sense: ConstraintSense,
    pub slack: SlackConfig,
}

/// Bound for a generic constraint at a specific stage/block
#[derive(Clone, Copy, Debug)]
pub struct ConstraintBound {
    pub constraint_id: u32,
    pub stage_id: i32,
    pub block_id: Option<u32>,    // None = applies to all blocks
    pub bound: f64,
}

/// Collection of all generic constraints
pub struct GenericConstraints {
    pub constraints: Vec<GenericConstraint>,
    /// Bounds indexed by (constraint_id, stage_id, block_id)
    /// block_id = u32::MAX means "all blocks"
    pub bounds: HashMap<(u32, i32, u32), f64>,
}

impl GenericConstraints {
    /// Get bound for a constraint at a specific stage and block
    pub fn get_bound(&self, constraint_id: u32, stage_id: i32, block_id: u32) -> Option<f64> {

```

```

    // Try specific block first
    self.bounds.get(&(constraint_id, stage_id, block_id))
        .or_else(|| self.bounds.get(&(constraint_id, stage_id, u32::MAX)))
        .copied()
}

/// Validate all entity references exist in the system
pub fn validate_references(&self, system: &System) -> Result<(), String> {
    for constraint in &self.constraints {
        for term in &constraint.expression.terms {
            match &term.variable {
                VariableRef::HydroStorage { hydro_id } |
                VariableRef::HydroTurbin { hydro_id, .. } |
                VariableRef::HydroSpillage { hydro_id, .. } |
                VariableRef::HydroOutflow { hydro_id, .. } |
                VariableRef::HydroGeneration { hydro_id, .. } => {
                    if !system.hydros.iter().any(|h| h.id == *hydro_id) {
                        return Err(format!(
                            "Constraint '{}': hydro {} not found",
                            constraint.name, hydro_id
                        ));
                    }
                }
                VariableRef::ThermalGeneration { thermal_id, .. } => {
                    if !system.thermals.iter().any(|t| t.id == *thermal_id) {
                        return Err(format!(
                            "Constraint '{}': thermal {} not found",
                            constraint.name, thermal_id
                        ));
                    }
                }
                VariableRef::LineDirect { line_id, .. } |
                VariableRef::LineReverse { line_id, .. } => {
                    if !system.lines.iter().any(|l| l.id == *line_id) {
                        return Err(format!(
                            "Constraint '{}': line {} not found",
                            constraint.name, line_id
                        ));
                    }
                }
                VariableRef::BusDeficit { bus_id, .. } |
                VariableRef::BusExcess { bus_id, .. } => {
                    if !system.buses.iter().any(|b| b.id == *bus_id) {
                        return Err(format!(
                            "Constraint '{}': bus {} not found",
                            constraint.name, bus_id
                        ));
                    }
                }
            }
        }
    }
    Ok(())
}

/// Thermal with entry/exit support

```

```

pub struct Thermal {
  pub id: u32,
  pub name: String,
  pub bus_id: u32,

  // Entry/exit for system changes modeling
  pub entry_stage_id: Option<i32>,
  pub exit_stage_id: Option<i32>,

  // Cost segments
  pub cost_segments: Vec<CostSegment>,

  // Base generation bounds (can be overridden per stage)
  pub base_min_generation: f64,
  pub base_max_generation: f64,
}

impl Thermal {
  /// Compute operative state for a given stage
  pub fn operative_state(&self, stage_id: i32) -> OperativeState {
    let entry = self.entry_stage_id.unwrap_or(i32::MIN);
    let exit = self.exit_stage_id.unwrap_or(i32::MAX);

    if stage_id > exit {
      OperativeState::Decommissioned
    } else if stage_id >= entry {
      OperativeState::Operating
    } else {
      OperativeState::NonExisting
    }
  }
}

/// Transmission line with entry/exit support
pub struct Line {
  pub id: u32,
  pub name: String,
  pub source_bus_id: u32,
  pub target_bus_id: u32,

  // Entry/exit for transmission expansion
  pub entry_stage_id: Option<i32>,
  pub exit_stage_id: Option<i32>,

  // Base capacity (can be overridden per stage and block via factors)
  pub base_direct_mw: f64,
  pub base_reverse_mw: f64,
  pub exchange_penalty: f64,
  pub losses_percent: f64,
}

impl Line {
  /// Compute operative state for a given stage
  pub fn operative_state(&self, stage_id: i32) -> OperativeState {
    let entry = self.entry_stage_id.unwrap_or(i32::MIN);
    let exit = self.exit_stage_id.unwrap_or(i32::MAX);
  }
}

```



```

        if stage_id > exit {
            OperativeState::Decommissioned
        } else if stage_id >= entry {
            OperativeState::Operating
        } else {
            OperativeState::NonExisting
        }
    }
}

/// Generation modeling for hydro plants (extensible)
pub enum HydroGeneration {
    ConstantProductivity {
        productivity: f64,
        base_min_turbined: f64, // Can be overridden per stage
        base_max_turbined: f64, // Can be overridden per stage
        base_min_generation: Option<f64>, // Derived from turbined if None
        base_max_generation: Option<f64>, // Derived from turbined if None
    },
    // Future: HeightVolumeTable, TailwaterCurve, EfficiencyCurve, etc.
}

/// Stage with blocks (study stages have id >= 0, pre-study have id < 0)
pub struct Stage {
    pub id: i32, // Negative for pre-study stages
    pub start_date: DateTime<Utc>,
    pub end_date: DateTime<Utc>,
    pub blocks: Vec<Block>, // Empty for pre-study stages
    pub risk_measure: RiskMeasure,
    pub state_config: StateConfig,
    pub num_scenarios: u32,
}

/// Block within a stage
pub struct Block {
    pub id: u32,
    pub name: String,
    pub hours: f64,
    // weight is computed as hours / sum(all block hours in stage)
}

/// Per-stage inflow model parameters
pub struct InflowModel {
    pub hydro_id: u32,
    pub stage_id: i32,
    pub mean: f64,
    pub std: f64,
    pub ar_order: u32,
    pub ar_coefficients: [f64; 6], // Fixed size, unused slots = 0
}

/// Per-stage load model parameters
pub struct LoadModel {
    pub bus_id: u32,
    pub stage_id: i32,
    pub mean: f64,
    pub std: f64,

```

```

}

/// Per-stage bounds override
pub struct StageBounds {
    pub entity_id: u32,
    pub stage_id: i32,
    pub min_bound: Option<f64>,
    pub max_bound: Option<f64>,
}

```

5.2 LP Subproblem Structure

```

/// Subproblem for a (stage, block) pair
pub struct BlockSubproblem {
    pub stage_id: u32,
    pub block_id: u32,

    // Solver interface
    pub solver: Box<dyn LpSolver>,

    // Variable indices (for solution extraction)
    pub variables: VariableLayout,

    // Constraint indices (for dual extraction)
    pub constraints: ConstraintLayout,

    // State management
    pub state: Box<dyn State>,

    // Precomputed data for hot path
    pub uncertainty_data: Vec<UncertaintyObservationData>,
    pub cut_constraint_slots: Vec<usize>,
}

/// Variable layout for O(1) extraction
pub struct VariableLayout {
    pub deficit: Range<usize>,
    pub direct_exchange: Option<Range<usize>>,
    pub reverse_exchange: Option<Range<usize>>,
    pub thermal_gen: Range<usize>,
    pub thermal_gen_segments: Vec<Range<usize>>,
    pub turbined_flow: Range<usize>,
    pub spillage: Range<usize>,
    pub storage_final: Range<usize>,
    pub storage_inter_block: Option<Range<usize>>,
    pub alpha: usize, // Future cost variable
}

```

5.3 FCF with Replication

Cut Preallocation Strategy: Full preallocation with dynamic capacity. This design ensures: 1. **Bit-for-bit reproducibility:** Checkpoints restore exact LP state for identical results 2. **Zero runtime allocation:** No thread-safety concerns during parallel solve 3. **Warm-start support:** Capacity = existing_cuts + new_training_cuts

See Section 5.4.3 for detailed slot management.

```

/// Future Cost Function (replicated per MPI rank)
pub struct FutureCostFunction {

```

```

    /// Cut pool for this stage
    pub cuts: CutPool,

    /// Visited states for this stage
    pub states: StatePool,

    /// Stage metadata
    pub stage_id: u32,

    /// Current iteration watermark (for incremental sync)
    pub sync_iteration: u32,
}

/// Cut pool with full preallocation and dynamic capacity
///
/// # Capacity Calculation
///
/// ```text
/// capacity = warm_start_cuts + (max_iterations * forward_passes)
///           = N + M
/// ```
///
/// Where:
/// - N = number of cuts loaded from warm-start policy (0 if fresh start)
/// - M = max_iterations * forward_passes (cuts to be generated)
///
/// # Reproducibility Guarantee
///
/// For bit-for-bit identical results on checkpoint/resume:
/// - ALL slots are serialized (active AND inactive)
/// - Slot indices are preserved exactly
/// - Bound states (active/inactive) are preserved
/// - Solver basis is saved for warm-starting
pub struct CutPool {
    /// Preallocated cuts
    /// capacity = warm_start_cuts + (max_iterations * forward_passes)
    pub pool: Vec<BendersCut>,

    /// Number of warm-start cuts (offset for new cut indexing)
    pub warm_start_count: usize,

    /// Number of populated cuts (warm_start + generated so far)
    pub populated_count: usize,

    /// Active cut bitmap for O(1) lookup
    pub active_bitmap: BitVec,

    /// State dimension (for validation)
    pub state_dimension: u32,
}

/// Single Benders cut
#[repr(C, align(64))] // Cache-line aligned
pub struct BendersCut {
    pub id: u64,
    pub rhs: f64,
    pub iteration: u32,

```

```

pub forward_pass_idx: u32,
pub is_active: AtomicBool,
pub domination_count: AtomicU32,
pub slot_index: AtomicU32,
_padding: [u8; 4],
/// Coefficients stored separately for SIMD alignment
pub coefficients_offset: usize,
}

```

5.4 Solver Interface Specification

Design Rationale: The SDDP algorithm must be solver-agnostic. The solver interface abstracts LP solver details (HiGHS, CPLEX, Gurobi) behind a unified trait hierarchy. Key design decisions:

1. **Compile-time solver selection** via Cargo features (avoids vtable overhead on hot path)
2. **Encapsulated retry logic** - each solver handles its own numerical difficulties internally
3. **Pre-allocated cut slots** - LP structure remains static; cuts enabled/disabled via bound toggling
4. **Cuts stored in physical units** - scaling transformations applied at solve time

5.4.1 Trait Hierarchy

SOLVER TRAIT HIERARCHY

LpProblem (Data Holder)

- Owns row/column data, bounds, objective, coefficients
- Solver-agnostic problem representation
- Pre-allocates cut constraint rows at construction

LpScaling (Pre-processing)

- Row/column scaling factors for numerical stability
- Transforms problem scaled space
- Persisted for cut coefficient computation

LpSolver (Execution)

- solve() → Result<LpSolution, SolverError>
- Encapsulates retry logic, warm-start, basis management
- Compile-time selected via Cargo features

LpSolution (Result Data)

- Primal values, dual values (constraint & variable)
- Objective value, solve status
- Basis information (optional, for warm-starting)

Static Data vs Transient Solver State:

Aspect	LpProblem (Static Data)	Solver State (Transient)
Lifetime	Per-stage, entire algorithm	Per-solve invocation
Ownership	Shared read-only across threads	Thread-local
Contents	Matrix, bounds, coefficients	Basis factors, working arrays
Memory	Pre-allocated once	Created/reused per solve
Analogies	HiGHS: <code>HighsLp</code> , CLP: <code>ClpModel</code>	HiGHS: <code>Highs::run()</code> state, CLP: <code>ClpSimplex</code>

This separation enables the key optimization: problem data is read-only and shared, while each thread maintains its own solver workspace for warm-starting.

5.4.2 Core Solver Trait

```

/// Core solver trait - the primary interface for SDDP
///
/// Implementations handle:
/// - Problem setup and solving
/// - Retry logic for numerical difficulties (solver-specific)
/// - Warm-starting and basis reuse
/// - Solution extraction with dual normalization
///
/// IMPORTANT: The SDDP algorithm never sees retry details. It calls solve()
/// and receives either a solution or an error.
pub trait LpSolver: Send + Sync {
    /// Solver identifier for logging
    fn name(&self) -> &'static str;

    /// Solve the LP problem
    ///
    /// This is the main entry point. Implementations handle:
    /// - Translating LpProblem to solver format
    /// - Applying solver-specific parameters
    /// - Internal retry logic for numerical issues
    /// - Solution extraction and dual normalization
    ///
    /// Returns `Ok(LpSolution)` on success (possibly after internal retries).
    fn solve(&mut self, problem: &LpProblem) -> Result<LpSolution, SolverError>;

    /// Solve with warm-start from previous basis
    fn solve_with_basis(
        &mut self,
        problem: &LpProblem,
        basis: &Basis,
    ) -> Result<LpSolution, SolverError>;

    /// Update problem and re-solve (incremental for stage re-use)
    ///
    /// More efficient than full solve when only RHS/bounds change.
    fn update_and_solve(
        &mut self,
        updates: &ProblemUpdates,
    ) -> Result<LpSolution, SolverError>;

    /// Reset solver state (clear caches, basis, etc.)
    fn reset(&mut self);

```

```

    /// Get accumulated solver statistics
    fn statistics(&self) -> SolverStatistics;
}

```

5.4.3 Pre-allocated Cut Constraint Design

Key Insight: Instead of dynamically adding/removing LP rows for Benders cuts, we pre-allocate all cut constraint rows at LP construction time. Cuts are **enabled/disabled by toggling their bounds**, not by row insertion/deletion. This preserves cache locality and enables warm-starting.

Design Decision: Full preallocation with dynamic capacity was chosen over dynamic slot management because: 1. **Reproducibility:** Checkpoint/resume requires identical LP structure for bit-for-bit results 2. **Thread Safety:** No runtime allocation eliminates need for atomic operations 3. **Performance:** Zero allocation overhead during parallel solve loops 4. **HPC Scale:** 10-20 GB memory is acceptable for production HPC nodes (256+ GB)

Mathematical Foundation:

A Benders cut has the form: $\text{cost} + \mathbf{c}^T \mathbf{x}$ where:

- cost is the future cost variable
- $\mathbf{c}$ is the cut intercept (RHS)
- $\mathbf{c}$ is the vector of cut coefficients (dual multipliers)
- $\mathbf{x}$ is the state vector from the previous stage

Rearranging to standard LP row form: $-\mathbf{c}^T \mathbf{x}$

Bound Toggling:

State	Row Lower Bound	Row Upper Bound	Effect
Active	(cut RHS)	$+\infty$	Cut enforced
Inactive	$-\infty$	$+\infty$	Always satisfied (free row)

Capacity Calculation:

```

capacity = warm_start_cuts + (max_iterations * forward_passes)
          = N + M

```

Example:

- Warm-start with 5,000 existing cuts: $N = 5,000$
- Training for 50 iterations $\times$ 200 forward passes: $M = 10,000$
- Total capacity per stage: 15,000 cuts
- Memory at 1120 state dimension: $\sim 15,000 \times 1120 \times 8$ 134 MB/stage
- Total for 120 stages: ~ 16 GB

Deterministic Slot Assignment:

```

/// Cut slot manager with deterministic indexing
///
/// # Design Rationale
///
/// Slots are COMPUTED, not allocated at runtime. This eliminates:
/// - Thread-safety concerns (no concurrent allocation)
/// - Non-determinism (same inputs + same slot assignments)
/// - Runtime overhead ( $O(1)$  slot computation)
///
/// # Slot Layout
///
/// ```text
/// Slots:      [0 ..... N-1] [N ..... N+M-1]

```

```

///          |--- Warm-start ---|  |----- New Training -----|
///
/// New cut slot = N + iteration * forward_passes + forward_pass_idx
/// ...
pub struct CutSlotManager {
    /// Total pre-allocated cut slots in LP
    /// capacity = warm_start_count + max_iterations * forward_passes
    pub capacity: usize,

    /// Base LP row index for first cut constraint
    /// Cuts occupy rows [base_row, base_row + capacity)
    pub base_row_index: usize,

    /// Number of warm-start cuts (slot offset for new cuts)
    pub warm_start_count: usize,

    /// Number of forward passes per iteration
    pub forward_passes: usize,

    /// Active cut bitmap: bit k = 1 if cut slot k is active
    pub active_bitmap: BitVec,

    /// Count of active cuts (avoids bitmap scan)
    pub active_count: u32,
}

impl CutSlotManager {
    /// Create a new slot manager with computed capacity
    pub fn new(
        warm_start_count: usize,
        max_iterations: usize,
        forward_passes: usize,
        base_row_index: usize,
    ) -> Self {
        let capacity = warm_start_count + max_iterations * forward_passes;
        Self {
            capacity,
            base_row_index,
            warm_start_count,
            forward_passes,
            active_bitmap: BitVec::repeat(false, capacity),
            active_count: 0,
        }
    }

    /// Compute slot for a new cut (deterministic, NOT allocation)
    ///
    /// This is a pure function with no side effects.
    /// Thread-safe by design: no mutable state.
    #[inline]
    pub const fn slot_for_new_cut(&self, iteration: u32, forward_pass_idx: u32) -> u32 {
        self.warm_start_count as u32
            + iteration * self.forward_passes as u32
            + forward_pass_idx
    }

    /// Get LP row index for a cut slot

```

```

#[inline]
pub const fn row_index(&self, slot: u32) -> usize {
    self.base_row_index + slot as usize
}

/// Activate a cut slot (update bitmap, called after LP bound is set)
///
/// Note: This modifies bitmap only. LP bound update is separate.
/// Called from single-threaded cut addition code.
pub fn activate(&mut self, slot: u32) {
    debug_assert!((slot as usize) < self.capacity);
    if !self.active_bitmap[slot as usize] {
        self.active_bitmap.set(slot as usize, true);
        self.active_count += 1;
    }
}

/// Deactivate a cut slot (for Level 1 selection or cut purging)
pub fn deactivate(&mut self, slot: u32) {
    debug_assert!((slot as usize) < self.capacity);
    if self.active_bitmap[slot as usize] {
        self.active_bitmap.set(slot as usize, false);
        self.active_count -= 1;
    }
}

/// Check if a slot is active
#[inline]
pub fn is_active(&self, slot: u32) -> bool {
    self.active_bitmap[slot as usize]
}

/// Iterate over all active slots (for checkpoint serialization)
pub fn active_slots(&self) -> impl Iterator<Item = u32> + '_ {
    self.active_bitmap.iter_ones().map(|i| i as u32)
}

}

/// LP problem with pre-allocated cut slots
impl LpProblem {
    /// Enable a cut by setting its row bound to the cut RHS
    ///
    /// Changes: row_lb[slot] = -∞ → row_lb[slot] = (cut RHS)
    ///
    /// # Thread Safety
    ///
    /// This method modifies a single row bound. Multiple threads can
    /// enable different cuts concurrently (different rows).
    pub fn enable_cut(&mut self, slot: u32, rhs: f64, cut_manager: &CutSlotManager) {
        let row = cut_manager.row_index(slot);
        self.row_lower[row] = rhs;
        // row_upper remains +∞
    }

    /// Disable a cut by setting its row bound to -∞
    ///
    /// Changes: row_lb[slot] = → row_lb[slot] = -∞

```



```

pub fn disable_cut(&mut self, slot: u32, cut_manager: &CutSlotManager) {
    let row = cut_manager.row_index(slot);
    self.row_lower[row] = f64::NEG_INFINITY;
}

/// Set cut coefficients for a slot
///
/// Called once when the cut is first created. For full preallocation,
/// coefficients are written to preallocated rows.
pub fn set_cut_coefficients(
    &mut self,
    slot: u32,
    rhs: f64,
    coefficients: &[f64],
    cut_manager: &CutSlotManager,
) {
    let row = cut_manager.row_index(slot);

    // Set coefficients: - 'x' + coefficient for x[j] is - [j]
    for (state_idx, &coef) in coefficients.iter().enumerate() {
        let col = self.state_variable_columns[state_idx];
        self.set_coefficient(row, col, -coef);
    }

    // coefficient is always +1
    self.set_coefficient(row, self.future_cost_column, 1.0);

    // Set the RHS (enables the cut)
    self.row_lower[row] = rhs;
}

/// Update cut coefficients in-place (for cut strengthening)
///
/// Note: Only updates coefficients, not the bound.
/// The cut must be re-enabled with the new RHS after this.
pub fn update_cut_coefficients(
    &mut self,
    slot: u32,
    coefficients: &[f64],
    cut_manager: &CutSlotManager,
) {
    let row = cut_manager.row_index(slot);
    // Update sparse matrix row
    for (state_idx, &coef) in coefficients.iter().enumerate() {
        let col = self.state_variable_columns[state_idx];
        // Coefficient sign: - 'x' + coefficient for x[j] is - [j]
        self.set_coefficient(row, col, -coef);
    }
    // coefficient is always +1
    self.set_coefficient(row, self.future_cost_column, 1.0);
}
}

```

5.4.4 Solver Error Types

```

/// Solver error visible to SDDP algorithm
///

```

```

/// This is the only error type SDDP sees. Solver implementations
/// convert internal errors to this type.
#[derive(Debug, Error)]
pub enum SolverError {
    /// Problem is primal infeasible (no feasible solution exists)
    /// SDDP Action: Data error - check bounds, constraints
    #[error("LP infeasible at stage {stage}: {reason}")]
    Infeasible {
        stage: u32,
        reason: String,
        ray: Option<Vec<f64>>, // Infeasibility ray if available
    },

    /// Problem is dual infeasible (unbounded)
    /// SDDP Action: Modeling error - check objective signs
    #[error("LP unbounded at stage {stage}: {reason}")]
    Unbounded {
        stage: u32,
        reason: String,
        direction: Option<Vec<f64>>, // Unbounded direction if available
    },

    /// Numerical difficulties (retries exhausted)
    /// SDDP Action: Log warning, may have partial solution
    #[error("Numerical difficulty at stage {stage}: {reason}")]
    NumericalDifficulty {
        stage: u32,
        reason: String,
        partial_solution: Option<LpSolution>,
        suggestion: NumericalRecoverySuggestion,
    },

    /// Time limit exceeded
    #[error("Time limit exceeded at stage {stage} after {elapsed_secs:.2}s")]
    TimeLimit {
        stage: u32,
        elapsed_secs: f64,
        best_solution: Option<LpSolution>,
    },

    /// Iteration limit exceeded
    #[error("Iteration limit at stage {stage} after {iterations} iterations")]
    IterationLimit {
        stage: u32,
        iterations: u64,
        best_solution: Option<LpSolution>,
    },

    /// Unrecoverable internal error
    #[error("Solver internal error: {message}")]
    Internal {
        message: String,
        code: Option<i32>,
        retryable: bool,
    },
}

```

```

/// Suggestions for recovering from numerical difficulties
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum NumericalRecoverySuggestion {
    /// Problem may be poorly scaled
    ApplyScaling,
    /// Try cold start instead of warm start
    ColdStart,
    /// Tighten solver tolerances
    TightenTolerances,
    /// Problem is inherently ill-conditioned
    ProblemIllConditioned,
    /// No specific suggestion
    Unknown,
}

impl SolverError {
    /// Check if error has a usable (possibly suboptimal) solution
    pub fn has_solution(&self) -> bool {
        match self {
            SolverError::NumericalDifficulty { partial_solution, .. } => partial_solution.is_some(),
            SolverError::TimeLimit { best_solution, .. } => best_solution.is_some(),
            SolverError::IterationLimit { best_solution, .. } => best_solution.is_some(),
            _ => false,
        }
    }
}

```

5.4.5 Solver-Specific Retry Logic

Important: Retry logic is encapsulated within each solver implementation. The SDDP algorithm never sees retry details - it only receives the final result from `solve()`.

Each solver implementation defines its own retry sequence based on its failure modes:

```

/// Example: HiGHS retry configuration (internal to HiGHS implementation)
///
/// This is NOT part of the public API - SDDP algorithm doesn't see this.
pub(crate) struct HighsRetryConfig {
    pub max_attempts: u32,
    pub time_budget: Duration,
    pub strategies: Vec<HighsRetryStrategy>,
}

pub(crate) enum HighsRetryStrategy {
    ClearBasis, // Attempt 1 failure: clear warm-start
    DisablePresolve, // Attempt 2 failure: disable presolve
    SwitchToIPM, // Attempt 3 failure: use interior point
    RelaxTolerances { primal: f64, dual: f64 }, // Attempt 4: relax tolerances
}

impl Default for HighsRetryConfig {
    fn default() -> Self {
        Self {
            max_attempts: 5,
            time_budget: Duration::from_secs(30),
            strategies: vec![
                HighsRetryStrategy::ClearBasis,
                HighsRetryStrategy::DisablePresolve,
            ],
        }
    }
}

```

```

        HighsRetryStrategy::SwitchToIPM,
        HighsRetryStrategy::RelaxTolerances { primal: 1e-6, dual: 1e-6 },
    ],
}
}
}

```

5.4.6 Dual Variable Normalization

Critical: Different solvers report dual multipliers with different sign conventions. The solver implementation MUST normalize duals to the canonical form before returning to SDDP.

Canonical Sign Convention:

- Positive dual means: increasing RHS **increases** objective (binding upper bound)
- For constraint $Ax \leq b$ with positive dual: shadow price is $z^*/b > 0$

```

/// Dual normalization within solver implementation
impl HiHSSolver {
    /// Normalize dual value to canonical form (internal method)
    fn normalize_dual(&self, raw_dual: f64, constraint_type: ConstraintType) -> f64 {
        // HiGHS convention: positive for constraints
        match constraint_type {
            ConstraintType::LessEqual => raw_dual,           // Already correct
            ConstraintType::GreaterEqual => -raw_dual,       // Flip sign
            ConstraintType::Equality => raw_dual,           // Unrestricted
        }
    }
}

```

5.4.7 Basis Storage for Warm-Starting

```

/// Basis information for warm-starting across iterations
///
/// Stored in ORIGINAL problem space (not presolved) for portability.
#[derive(Debug, Clone)]
pub struct Basis {
    /// Basis status for each column (variable)
    pub column_status: Vec<BasisStatus>,

    /// Basis status for each row (constraint)
    pub row_status: Vec<BasisStatus>,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum BasisStatus {
    /// Variable is at its lower bound
    AtLower = 0,
    /// Variable is basic (in the basis)
    Basic = 1,
    /// Variable is at its upper bound
    AtUpper = 2,
    /// Variable is free (superbasic)
    Free = 3,
    /// Variable is fixed
    Fixed = 4,
}

```

5.4.8 Compile-Time Solver Selection

```
# Cargo.toml feature flags
[features]
default = ["solver-highs"]
solver-highs = ["highs"]
solver-cplex = ["cplex-rs"]
solver-gurobi = ["gurobi-rs"]

/// Type alias selected at compile time via features
#[cfg(feature = "solver-highs")]
pub type ActiveSolver = HighsSolver;

#[cfg(feature = "solver-cplex")]
pub type ActiveSolver = CplexSolver;

#[cfg(feature = "solver-gurobi")]
pub type ActiveSolver = GurobiSolver;

/// Create solver instance (compile-time selected)
pub fn create_solver(config: SolverConfig) -> ActiveSolver {
    ActiveSolver::new(config)
}
```

5.4.9 Thread-Local Solver Infrastructure

Design Rationale: LP solvers (HiGHS, CLP, CPLEX) are **NOT thread-safe**. Each OpenMP thread requires its own solver instance. This section specifies the thread-local workspace pattern that:

1. **Eliminates contention:** Each thread owns its solver instance exclusively
2. **Enables warm-starting:** Basis persists across solves within same thread
3. **Optimizes memory:** Thread-local buffers avoid allocation during hot loops
4. **NUMA-aware:** Workspaces allocated on local NUMA node for memory bandwidth

Key Insight: Unlike a pool-based approach (borrow/return), we use **persistent ownership**. Each thread keeps its solver for the entire SDDP run. This eliminates synchronization overhead and enables optimal warm-starting.

Thread-Local Workspace Structure:

```
/// Complete workspace for one OpenMP thread's LP solving
///
/// Lifetime: Created once at startup, persists for entire SDDP run.
/// Ownership: Exclusively owned by one thread (no sharing).
///
/// Memory Layout (for 1120-state problem):
///   - Solver instance: ~15 MB (HiGHS with pre-allocated cuts)
///   - RHS buffers: ~100 KB
///   - Solution storage: ~100 KB
///   - Basis storage: ~50 KB
///   - Total per thread: ~16 MB
///   - Total for 192 threads: ~3 GB
#[repr(C)] // Predictable layout for NUMA allocation
pub struct ThreadSolverWorkspace {
    /// The solver instance (HiGHS, CLP, etc.)
    ///
    /// IMPORTANT: This solver instance is NEVER shared between threads.
    /// It persists for the entire SDDP run, enabling warm-starting.
    pub solver: ActiveSolver,
```

```

    /// Thread-local RHS buffer for stage problem updates
    ///
    /// During forward pass, we update RHS values (inflows, demands, state)
    /// without modifying the shared LpProblem. This buffer holds the
    /// thread-specific RHS values passed to solver.update_and_solve().
    pub rhs_buffer: Vec<f64>,

    /// Thread-local solution storage
    ///
    /// Avoids allocation when extracting solution from solver.
    pub primal_solution: Vec<f64>,
    pub dual_solution: Vec<f64>,
    pub reduced_costs: Vec<f64>,

    /// Cached basis from last solve (for warm-starting)
    ///
    /// When solving same stage with different RHS (common in forward pass),
    /// the basis is typically valid and warm-starting is very effective.
    pub cached_basis: Option<Basis>,

    /// Last stage solved (for basis validity tracking)
    ///
    /// If current stage != last_stage, basis may not be applicable.
    pub last_stage: Option<u32>,

    /// Statistics accumulated by this thread
    pub stats: ThreadSolverStats,

    /// NUMA node this workspace is allocated on
    pub numa_node: u32,

    /// Cache line padding to prevent false sharing between workspaces
    _padding: [u8; 64],
}

/// Statistics tracked per thread
#[derive(Debug, Default, Clone)]
pub struct ThreadSolverStats {
    /// Total LP solves performed
    pub solves: u64,

    /// Warm-started solves (basis reused)
    pub warm_starts: u64,

    /// Cold starts (no basis or basis invalid)
    pub cold_starts: u64,

    /// Solve attempts that required retry
    pub retries: u64,

    /// Total simplex iterations
    pub simplex_iterations: u64,

    /// Total solve time (seconds)
    pub total_solve_time: f64,

    /// Maximum solve time for single LP

```

```

    pub max_solve_time: f64,
}

impl ThreadSolverWorkspace {
    /// Create workspace with NUMA-aware allocation
    ///
    /// MUST be called from the thread that will own this workspace
    /// (first-touch policy ensures memory is allocated on local NUMA node).
    pub fn new_numa_local(
        config: &SolverConfig,
        problem_dimensions: &ProblemDimensions,
        numa_node: u32,
    ) -> Self {
        // Pin to NUMA node during allocation
        let _guard = numa_bind_guard(numa_node);

        // Create solver (allocates internal working memory)
        let solver = ActiveSolver::new(config.clone());

        // Pre-allocate buffers with first-touch initialization
        let num_rows = problem_dimensions.num_constraints;
        let num_cols = problem_dimensions.num_variables;

        let mut rhs_buffer = vec![0.0; num_rows];
        let mut primal_solution = vec![0.0; num_cols];
        let mut dual_solution = vec![0.0; num_rows];
        let mut reduced_costs = vec![0.0; num_cols];

        // First-touch to ensure NUMA-local allocation
        for v in rhs_buffer.iter_mut() { *v = 0.0; }
        for v in primal_solution.iter_mut() { *v = 0.0; }
        for v in dual_solution.iter_mut() { *v = 0.0; }
        for v in reduced_costs.iter_mut() { *v = 0.0; }

        Self {
            solver,
            rhs_buffer,
            primal_solution,
            dual_solution,
            reduced_costs,
            cached_basis: None,
            last_stage: None,
            stats: ThreadSolverStats::default(),
            numa_node,
            _padding: [0u8; 64],
        }
    }

    /// Solve stage LP with automatic warm-starting
    ///
    /// This is the main entry point for forward/backward pass solving.
    /// Handles basis caching and warm-start decisions internally.
    pub fn solve_stage(
        &mut self,
        stage: u32,
        problem: &LpProblem,
        rhs_updates: &[(usize, f64)], // (row_index, value) pairs
    ) {

```

```

) -> Result<StageSolution, SolverError> {
    let start = std::time::Instant::now();

    // Prepare RHS buffer with updates
    self.prepare_rhs(problem, rhs_updates);

    // Determine warm-start eligibility
    let use_warm_start = self.can_warm_start(stage);

    // Solve
    let result = if use_warm_start {
        self.stats.warm_starts += 1;
        let basis = self.cached_basis.as_ref().unwrap();
        self.solver.solve_with_basis(problem, basis)
    } else {
        self.stats.cold_starts += 1;
        self.solver.solve(problem)
    };

    // Update statistics
    let elapsed = start.elapsed().as_secs_f64();
    self.stats.solves += 1;
    self.stats.total_solve_time += elapsed;
    self.stats.max_solve_time = self.stats.max_solve_time.max(elapsed);

    // Handle result
    match result {
        Ok(solution) => {
            // Cache basis for next solve
            if let Some(basis) = solution.basis.clone() {
                self.cached_basis = Some(basis);
                self.last_stage = Some(stage);
            }

            // Extract solution into thread-local buffers
            self.extract_solution(&solution);

            self.stats.simplex_iterations += solution.simplex_iterations;

            Ok(StageSolution::from_lp_solution(solution))
        }
        Err(e) => {
            // Invalidate cached basis on failure
            self.cached_basis = None;
            self.last_stage = None;
            self.stats.retries += 1;
            Err(e)
        }
    }
}

/// Check if warm-starting is likely beneficial
fn can_warm_start(&self, stage: u32) -> bool {
    match (&self.cached_basis, self.last_stage) {
        (Some(_), Some(last)) => {
            // Same stage: basis is highly likely to be valid
            // Adjacent stage: basis may still be useful

```



```

        // Different stage: cold start is safer
        stage == last || stage.abs_diff(last) <= 1
    }
    - => false,
}

}

/// Prepare RHS buffer with scenario-specific updates
fn prepare_rhs(&mut self, problem: &LpProblem, updates: &[(usize, f64)]) {
    // Copy base RHS
    self.rhs_buffer.copy_from_slice(&problem.row_rhs);

    // Apply updates
    for &(row, value) in updates {
        self.rhs_buffer[row] = value;
    }
}

/// Extract solution into thread-local storage
fn extract_solution(&mut self, solution: &LpSolution) {
    self.primal_solution.copy_from_slice(&solution.primal);
    self.dual_solution.copy_from_slice(&solution.dual);
    if let Some(rc) = &solution.reduced_costs {
        self.reduced_costs.copy_from_slice(rc);
    }
}

/// Reset workspace for new iteration (optional, for memory cleanup)
pub fn reset_for_iteration(&mut self) {
    // Optionally clear cached basis at iteration boundary
    // This can help numerical stability but reduces warm-start benefit
    // Typically: keep basis, it's often still useful
}

/// Get accumulated statistics
pub fn statistics(&self) -> &ThreadSolverStats {
    &self.stats
}

/// Merge statistics from multiple workspaces (called after parallel region)
pub fn merge_statistics(workspaces: &[ThreadSolverWorkspace]) -> ThreadSolverStats {
    let mut merged = ThreadSolverStats::default();
    for ws in workspaces {
        merged.solves += ws.stats.solves;
        merged.warm_starts += ws.stats.warm_starts;
        merged.cold_starts += ws.stats.cold_starts;
        merged.retries += ws.stats.retries;
        merged.simplex_iterations += ws.stats.simplex_iterations;
        merged.total_solve_time += ws.stats.total_solve_time;
        merged.max_solve_time = merged.max_solve_time.max(ws.stats.max_solve_time);
    }
    merged
}
}

```

Thread-Local Workspace Manager:

```

/// Manager for all thread-local workspaces in a rank

```

```

///
/// Creates and owns one workspace per OpenMP thread.
/// Provides access pattern that OpenMP threads use via thread ID.
pub struct WorkspaceManager {
    /// One workspace per thread, indexed by thread ID
    ///
    /// IMPORTANT: This Vec is never modified after initialization.
    /// Threads access their workspace by index: workspaces[omp_get_thread_num()]
    workspaces: Vec<ThreadSolverWorkspace>,

    /// Number of NUMA nodes on this system
    num_numa_nodes: u32,

    /// Threads per NUMA node
    threads_per_numa: u32,
}

impl WorkspaceManager {
    /// Initialize all workspaces with NUMA-aware allocation
    ///
    /// MUST be called from an OpenMP parallel region where each thread
    /// initializes its own workspace (first-touch policy).
    ///
    /// ```rust
    /// // Initialization pattern (pseudo-code for OpenMP)
    /// let mut workspaces = Vec::with_capacity(num_threads);
    ///
    /// #[omp_parallel]
    /// {
    ///     let tid = omp_get_thread_num();
    ///     let numa = tid / threads_per_numa;
    ///
    ///     let ws = ThreadSolverWorkspace::new_numa_local(&config, &dims, numa);
    ///
    ///     #[omp_critical]
    ///     workspaces.push((tid, ws));
    /// }
    ///
    /// // Sort by thread ID and extract workspaces
    /// workspaces.sort_by_key(|(tid, _)| *tid);
    /// let workspaces: Vec<_> = workspaces.into_iter().map(|(_, ws)| ws).collect();
    /// ```
    pub fn new(
        num_threads: usize,
        num_numa_nodes: u32,
        config: &SolverConfig,
        dims: &ProblemDimensions,
    ) -> Self {
        let threads_per_numa = (num_threads as u32 + num_numa_nodes - 1) / num_numa_nodes;

        // This allocation happens in serial, but workspace contents are
        // initialized via first-touch in parallel (see documentation above)
        let workspaces = (0..num_threads)
            .map(|tid| {
                let numa = (tid as u32) / threads_per_numa;
                // Note: In actual implementation, this should be called from
                // within an OpenMP parallel region for proper first-touch
            })
    }
}

```

```

        ThreadSolverWorkspace::new_numa_local(config, dims, numa)
    })
    .collect();

    Self {
        workspaces,
        num_numa_nodes,
        threads_per_numa,
    }
}

/// Get workspace for current thread (called from OpenMP parallel region)
///
/// # Safety
/// Caller must ensure they are the only thread accessing this workspace.
/// This is guaranteed by OpenMP's thread ID uniqueness.
#[inline]
pub fn get(&self, thread_id: usize) -> &ThreadSolverWorkspace {
    &self.workspaces[thread_id]
}

/// Get mutable workspace for current thread
///
/// # Safety
/// Caller must ensure exclusive access. In OpenMP, this is guaranteed
/// when each thread only accesses workspaces[omp_get_thread_num()].
#[inline]
pub fn get_mut(&mut self, thread_id: usize) -> &mut ThreadSolverWorkspace {
    &mut self.workspaces[thread_id]
}

/// Get aggregated statistics across all threads
pub fn aggregate_statistics(&self) -> ThreadSolverStats {
    ThreadSolverWorkspace::merge_statistics(&self.workspaces)
}

/// Number of workspaces (equals number of threads)
pub fn len(&self) -> usize {
    self.workspaces.len()
}
}

```

Thread Safety Invariants for LpProblem:

Critical Design Rule: The LpProblem struct is treated as **read-only during parallel forward pass**. All scenario-specific modifications (RHS values) go through thread-local `rhs_buffer` in ThreadSolverWorkspace.

Operation	Thread Safety	When
Read LP structure	Safe (immutable)	Forward pass (parallel)
Read cut coefficients	Safe (immutable during forward)	Forward pass (parallel)
Update RHS for scenario	Thread-local buffer	Forward pass (parallel)

Operation	Thread Safety	When
Enable/disable cuts	Single-threaded only	Backward pass (sequential per stage)
Add new cut coefficients	Single-threaded only	Backward pass (sequential per stage)

```

/// LpProblem thread-safety documentation
impl LpProblem {
    /// Access pattern during forward pass:
    ///
    /// ```text
    /// Forward Pass (PARALLEL):
    ///   - LpProblem is READ-ONLY (shared reference)
    ///   - RHS updates go to thread-local buffer
    ///   - Solver reads from LpProblem + thread-local RHS
    ///
    /// Backward Pass (SEQUENTIAL per stage):
    ///   - Cuts are enabled/disabled by single thread
    ///   - New cut coefficients written by single thread
    ///   - No parallel access during modification
    /// ```
    ///
    /// This separation eliminates need for locking on LpProblem.
}

```

Relationship to Section 6.9.6 SolverPool:

Clarification: Section 6.9.6 defines `SolverPool` as an alternative pattern for solver instance management. The two approaches serve different scenarios:

Approach	Use Case	Memory	Warm-Start Efficiency
<code>ThreadSolverWorkspace</code> (5.4.9)	Production SDDP	Fixed: 1 solver/thread	Optimal (basis persists)
<code>SolverPool</code> (6.9.6)	Variable workloads	Pooled: $N <$ threads	Reduced (basis may not match)

Recommendation: Use `ThreadSolverWorkspace` for SDDP when:

Condition
Thread count is fixed for the run
Warm-starting is critical for performance
Memory for N solver instances is acceptable

Use `SolverPool` when:

Condition
Thread count varies dynamically
Memory is severely constrained

Condition
Solver instances have high initialization cost

5.4.10 HiGHS Implementation Guidelines

Context: HiGHS is the default open-source LP solver for POWE.RS. This section provides implementation guidance specific to HiGHS integration, complementing the abstract `LpSolver` trait.

HiGHS Architecture Alignment:

POWE.RS Concept	HiGHS Equivalent	Notes
<code>LpProblem</code>	<code>HighsLp</code>	Data holder, solver-agnostic
<code>LpSolver</code>	<code>Highs</code> class	Solver controller + internal state
<code>Basis</code>	<code>HighsBasis</code>	Column/row status arrays
<code>LpSolution</code>	<code>HighsSolution</code> + <code>HighsInfo</code>	Primal, dual, objective

Key Implementation Pattern:

```

/// HiGHS solver wrapper implementing LpSolver trait
pub struct HighsSolver {
    /// The HiGHS instance - persists for entire SDDP run
    ///
    /// IMPORTANT: Do NOT create/destroy per solve. The Highs object
    /// maintains internal working memory and basis that enables
    /// efficient warm-starting.
    highs: Highs,

    /// Retry configuration (internal, not exposed to SDDP)
    retry_config: HighsRetryConfig,

    /// Whether LP has been loaded (passModel called)
    model_loaded: bool,

    /// Accumulated statistics
    stats: SolverStatistics,
}

impl HighsSolver {
    /// Create new HiGHS solver instance
    ///
    /// Call once per thread at startup. The instance persists for the
    /// entire SDDP run.
    pub fn new(config: SolverConfig) -> Self {
        let mut highs = Highs::new();

        // Configure for SDDP workload
        highs.set_option("solver", "simplex").unwrap();
        highs.set_option("simplex_strategy", "4").unwrap(); // Dual
        highs.set_option("presolve", "off").unwrap(); // Disable for warm-start
        highs.set_option("parallel", "off").unwrap(); // Thread safety
    }
}

```

```

    highs.set_option("output_flag", false).unwrap(); // Quiet

    // Tolerances
    highs.set_option("primal_feasibility_tolerance", 1e-7).unwrap();
    highs.set_option("dual_feasibility_tolerance", 1e-7).unwrap();

    Self {
        highs,
        retry_config: HighsRetryConfig::default(),
        model_loaded: false,
        stats: SolverStatistics::default(),
    }
}

/// Load LP structure (call once, or when structure changes)
///
/// After loading, use update_and_solve() for RHS changes.
pub fn load_model(&mut self, problem: &LpProblem) {
    let lp = problem.to_highs_lp();
    self.highs.pass_model(lp).expect("Failed to load model");
    self.model_loaded = true;
}

impl LpSolver for HighsSolver {
    fn name(&self) -> &'static str {
        "HiGHS"
    }

    fn solve(&mut self, problem: &LpProblem) -> Result<LpSolution, SolverError> {
        if !self.model_loaded {
            self.load_model(problem);
        }

        // Solve with internal retry logic
        self.solve_with_retry()
    }

    fn solve_with_basis(
        &mut self,
        problem: &LpProblem,
        basis: &Basis,
    ) -> Result<LpSolution, SolverError> {
        if !self.model_loaded {
            self.load_model(problem);
        }

        // Set basis for warm-start
        let highs_basis = basis.to_highs_basis();
        self.highs.set_basis(highs_basis).expect("Failed to set basis");

        self.solve_with_retry()
    }

    fn update_and_solve(
        &mut self,
        updates: &ProblemUpdates,
    ) -> Result<LpSolution, SolverError> {
        self.update(updates);
        self.solve()
    }
}

```

```

) -> Result<LpSolution, SolverError> {
    // Apply RHS changes efficiently
    for &(row, value) in &updates.rhs_changes {
        self.highs.change_row_bounds(row, value, f64::INFINITY).unwrap();
    }

    // Solve (basis from previous solve is automatically used)
    self.solve_with_retry()
}

fn reset(&mut self) {
    self.highs.clear_solver();
    self.model_loaded = false;
}

fn statistics(&self) -> SolverStatistics {
    self.stats.clone()
}
}

```

Warm-Starting with Bound-Toggled Cuts:

Key Insight: When cuts are enabled/disabled via bound toggling (Section 5.4.3), the LP structure remains unchanged. This means:

1. **Basis dimensions are constant** - row/column counts don't change
2. **Basis status for new cuts** - newly enabled cuts start as non-basic (at lower bound)
3. **Warm-start is VALID** - simplex can proceed from current basis

```

impl HighsSolver {
    /// Enable cut by updating row bound (warm-start safe)
    ///
    /// The cut row already exists in the LP with bound = -∞.
    /// Enabling sets bound = (cut RHS).
    ///
    /// Basis impact:
    /// - If row was basic: remains basic (no impact)
    /// - If row was non-basic at -∞: becomes non-basic at
    /// - Either way, basis is VALID for warm-start
    pub fn enable_cut(&mut self, row: usize, rhs: f64) {
        self.highs.change_row_bounds(row, rhs, f64::INFINITY)
            .expect("Failed to change row bounds");
        // Basis remains valid - no need to clear
    }

    /// Disable cut by resetting row bound (warm-start safe)
    pub fn disable_cut(&mut self, row: usize) {
        self.highs.change_row_bounds(row, f64::NEG_INFINITY, f64::INFINITY)
            .expect("Failed to change row bounds");
        // Basis remains valid
    }
}

```

Retry Strategy for HiGHS:

```

impl HighsSolver {
    /// Internal solve with retry logic
    ///
    /// SDDP algorithm never sees retry details - only final result.
    fn solve_with_retry(&mut self) -> Result<LpSolution, SolverError> {

```

```

for (attempt, strategy) in self.retry_config.strategies.iter().enumerate() {
    // Apply strategy
    match strategy {
        HighsRetryStrategy::ClearBasis => {
            self.highs.clear_basis();
        }
        HighsRetryStrategy::DisablePresolve => {
            // Already disabled, try enabling briefly
            self.highs.set_option("presolve", "on").unwrap();
        }
        HighsRetryStrategy::SwitchToIPM => {
            self.highs.set_option("solver", "ipm").unwrap();
        }
        HighsRetryStrategy::RelaxTolerances { primal, dual } => {
            self.highs.set_option("primal_feasibility_tolerance", *primal).unwrap();
            self.highs.set_option("dual_feasibility_tolerance", *dual).unwrap();
        }
    }

    // Attempt solve
    let status = self.highs.run();

    match status {
        HighsStatus::kOk => {
            let model_status = self.highs.model_status();
            match model_status {
                HighsModelStatus::kOptimal => {
                    // Restore default settings for next solve
                    self.restore_default_settings();
                    return Ok(self.extract_solution());
                }
                HighsModelStatus::kInfeasible => {
                    return Err(SolverError::Infeasible {
                        stage: 0, // Filled by caller
                        reason: "LP infeasible".to_string(),
                        ray: self.highs.get_dual_ray().ok(),
                    });
                }
                HighsModelStatus::kUnbounded => {
                    return Err(SolverError::Unbounded {
                        stage: 0,
                        reason: "LP unbounded".to_string(),
                        direction: self.highs.get_primal_ray().ok(),
                    });
                }
                _ => {
                    // Numerical difficulty, try next strategy
                    self.stats.retries += 1;
                    continue;
                }
            }
        }
        _ => {
            // Solver error, try next strategy
            self.stats.retries += 1;
            continue;
        }
    }
}

```



```

    }
}

// All retries exhausted
Err(SolverError::NumericalDifficulty {
    stage: 0,
    reason: format!("Failed after {} attempts", self.retry_config.strategies.len()),
    partial_solution: self.try_extract_partial_solution(),
    suggestion: NumericalRecoverySuggestion::ProblemIllConditioned,
})
}

fn restore_default_settings(&mut self) {
    self.highs.set_option("solver", "simplex").unwrap();
    self.highs.set_option("presolve", "off").unwrap();
    self.highs.set_option("primal_feasibility_tolerance", 1e-7).unwrap();
    self.highs.set_option("dual_feasibility_tolerance", 1e-7).unwrap();
}

fn extract_solution(&self) -> LpSolution {
    let info = self.highs.info();
    let solution = self.highs.get_solution();
    let basis = self.highs.get_basis();

    LpSolution {
        status: SolveStatus::Optimal,
        objective_value: info.objective_function_value,
        primal: solution.col_value,
        dual: solution.row_dual,
        reduced_costs: Some(solution.col_dual),
        basis: Some(Basis::from_highs_basis(&basis)),
        simplex_iterations: info.simplex_iteration_count as u64,
    }
}
}

```

Memory Footprint per HiGHS Instance:

Component	Formula	Example (1120 states, 15K cuts)
LP matrix storage	$\text{nnz} \times 16 \text{ bytes}$	~5 MB
Working arrays	$(\text{rows} + \text{cols}) \times 3 \times 8 \text{ bytes}$	~1 MB
Basis storage	$(\text{rows} + \text{cols}) \times 1 \text{ byte}$	~50 KB
Factor storage	varies with sparsity	~5-10 MB
Total per instance		~15 MB
192 threads		~2.9 GB

This memory footprint is acceptable for production HPC nodes (256+ GB RAM).

Batch Bound Operations for Hot Path:

Performance Critical: During the forward/backward pass, constraint RHS values must be updated thousands of times per second. Single-row bound changes incur function call overhead per row. Batch operations amortize this overhead.

```

impl HighsSolver {
    /// Update multiple row bounds in a single call (hot path optimization)

```

```

///
/// This is the preferred method for RHS updates in SDDP:
/// - Forward pass: Update inflow constraints, state transfer constraints
/// - Backward pass: Update state constraints for branching scenarios
///
/// # Performance
///
/// | Operation | 500 rows | Overhead |
/// |-----|-----|-----|
/// | 500 × `change_row_bounds()` | ~0.5 ms | High (500 calls) |
/// | 1 × `change_rows_bounds_batch()` | ~0.05 ms | Low (1 call) |
///
/// # Arguments
///
/// * `row_indices` - Row indices to update (must be pre-allocated, reused)
/// * `lower_bounds` - New lower bounds (parallel array)
/// * `upper_bounds` - New upper bounds (parallel array)
///
/// # Thread Safety
///
/// Uses thread-local buffers to avoid allocation. Each thread maintains
/// its own index/value buffers sized to max constraint count.
pub fn change_rows_bounds_batch(
    &mut self,
    row_indices: &[i32],
    lower_bounds: &[f64],
    upper_bounds: &[f64],
) -> Result<(), SolverError> {
    debug_assert_eq!(row_indices.len(), lower_bounds.len());
    debug_assert_eq!(row_indices.len(), upper_bounds.len());

    // HiGHS API: changeRowsBounds(num_rows, indices, lower, upper)
    let status = unsafe {
        highs_sys::Highs_changeRowsBounds(
            self.highs.ptr(),
            row_indices.len() as i32,
            row_indices.as_ptr(),
            lower_bounds.as_ptr(),
            upper_bounds.as_ptr(),
        )
    };

    if status == highs_sys::kHighsStatusOk {
        Ok(())
    } else {
        Err(SolverError::InternalError {
            reason: "Failed to change row bounds".to_string(),
        })
    }
}

/// Update multiple column bounds in batch (for cut activation)
///
/// Used when enabling/disabling multiple cuts at once (e.g., after cut selection).
pub fn change_cols_bounds_batch(
    &mut self,
    col_indices: &[i32],

```

```

        lower_bounds: &[f64],
        upper_bounds: &[f64],
    ) -> Result<(), SolverError> {
        debug_assert_eq!(col_indices.len(), lower_bounds.len());
        debug_assert_eq!(col_indices.len(), upper_bounds.len());

        let status = unsafe {
            highs_sys::Highs_changeColsBounds(
                self.highs.ptr(),
                col_indices.len() as i32,
                col_indices.as_ptr(),
                lower_bounds.as_ptr(),
                upper_bounds.as_ptr(),
            )
        };

        if status == highs_sys::kHighsStatusOk {
            Ok(())
        } else {
            Err(SolverError::InternalError {
                reason: "Failed to change column bounds".to_string(),
            })
        }
    }
}

```

Thread-Local Batch Buffers:

```

/// Thread-local buffers for batch bound operations
///
/// These buffers are allocated once per thread and reused across all solves.
/// Eliminates allocation overhead on the hot path.
thread_local! {
    /// Row indices buffer (sized to max constraints per stage)
    static BATCH_ROW_INDICES: RefCell<Vec<i32>> = RefCell::new(Vec::with_capacity(2000));

    /// Lower bounds buffer
    static BATCH_LOWER_BOUNDS: RefCell<Vec<f64>> = RefCell::new(Vec::with_capacity(2000));

    /// Upper bounds buffer
    static BATCH_UPPER_BOUNDS: RefCell<Vec<f64>> = RefCell::new(Vec::with_capacity(2000));
}

```

```

impl ThreadSolverWorkspace {
    /// Prepare batch buffers for RHS update
    ///
    /// Populates thread-local buffers with row indices and new RHS values.
    /// Returns references to the buffers for use with batch API.
    pub fn prepare_batch_rhs_update(
        &self,
        updates: &[(usize, f64)], // (row_index, new_rhs)
    ) -> (&[i32], &[f64], &[f64]) {
        BATCH_ROW_INDICES.with(|indices| {
            BATCH_LOWER_BOUNDS.with(|lower| {
                BATCH_UPPER_BOUNDS.with(|upper| {
                    let mut indices = indices.borrow_mut();
                    let mut lower = lower.borrow_mut();
                    let mut upper = upper.borrow_mut();

```

```

indices.clear();
lower.clear();
upper.clear();

for &(row, rhs) in updates {
    indices.push(row as i32);
    lower.push(rhs);           // Lower bound = RHS for equality/
    upper.push(f64::INFINITY); // Upper bound = ∞ for constraints
}

// Return borrowed slices
// Note: In actual implementation, need to handle lifetimes properly
})
})
}
}

```

5.4.11 Uncertainty Observation Data (PAR Preprocessing)

Design Rationale: The SDDP hot path requires updating constraint RHS values millions of times. Instead of computing AR model realizations at runtime (involving lag lookups and multiplications), we precompute the deterministic and stochastic components at problem setup. This reduces the hot-path RHS update to a single multiply-add: $\text{rhs} = \text{base} + \sigma_t \cdot \dots$.

Mathematical Foundation:

For a PAR(p) model, the inflow at stage t is:

$$\left(\frac{Y_t - \mu_t}{\sigma_t} \right) = \sum_{i=1}^p \phi_i \left(\frac{Y_{t-i} - \mu_{t-i}}{\sigma_{t-i}} \right) + \eta_t$$

Where:

- Y_t = inflow at stage t
- μ_t = seasonal mean for stage t
- ϕ_i = AR coefficient for lag i
- σ_t = seasonal standard deviation (innovation scale)
- $\eta_t \sim N(0, 1)$ = independent standard normal innovation

We can rewrite and put Y_t explicitly in the LHS:

$$Y_t = \mu_t + \sigma_t \left[\sum_{i=1}^p \phi_i \left(\frac{Y_{t-i} - \mu_{t-i}}{\sigma_{t-i}} \right) + \eta_t \right]$$

Then, we can see this expression as an alternative PAR(p) model:

$$Y_t = \sum_{i=1}^P \frac{\sigma_t}{\sigma_{t-1}} \phi_i Y_{t-i} + \mu_t - \sum_{i=1}^P \frac{\sigma_t}{\sigma_{t-1}} \phi_i \mu_{t-i} + \sigma_t \eta_t$$

If we define scaled coefficients $\psi_i = \frac{\sigma_t}{\sigma_{t-1}} \phi_i$, then:

$$Y_t = \sum_{i=1}^P \psi_i Y_{t-i} + \mu_t - \sum_{i=1}^P \psi_i \mu_{t-i} + \sigma_t \eta_t$$

Rearranging for Hot-Path Computation:

$$Y_t = \underbrace{\left(\mu_t - \sum_{i=1}^p \psi_i \mu_{t-i} \right)}_{\text{deterministic_base}} + \underbrace{\sum_{i=1}^p \psi_i Y_{t-i}}_{\text{lag contribution (state)}} + \underbrace{\sigma_t}_{\text{seasonal_std}} \cdot \eta_t$$

The lag contribution $\sum \psi_i Y_{t-i}$ comes from state variables (past realized inflows), which are already known at each stage. The `deterministic_base` is a constant per stage that can be precomputed. Then, the process of updating uncertainty observation is simply fixing the lagged variables in the LP without any changes and rescaling the sampled innovation with a simple multiply-add, then fixing it in the innovation variable.

Data Structure:

```

/// Precomputed data for efficient uncertainty realization on hot path
///
/// For each uncertain quantity (inflow, load), stores the precomputed values
/// needed to realize a scenario with minimal computation:
///
/// realized_value = deterministic_base + lag_contribution + seasonal_std * innovation
///
/// Where:
/// - `deterministic_base`:  $\mu_t - \sum \psi_i \times \mu_{t-i}$  (constant per stage)
/// - `lag_contribution`:  $\sum \psi_i \times Y_{t-i}$  (from state variables, computed at runtime)
/// - `seasonal_std * innovation`:  $\sigma_t \times \eta_t$  (stochastic component)
///
/// For load models without AR terms (order = 0):
/// realized_value = mean + seasonal_std * innovation
#[derive(Debug, Clone)]
pub struct UncertaintyObservationData {
    /// Entity identifier (hydro_id for inflows, bus_id for loads)
    pub entity_id: u32,

    /// Stage identifier
    pub stage_id: u32,

    /// Constraint row index in LP for this entity's uncertainty constraint
    ///
    /// This is the row where the RHS will be set to the realized value.
    /// For inflows: the water balance constraint RHS
    /// For loads: the power balance constraint RHS
    pub constraint_row: usize,

    /// Precomputed deterministic base:  $\mu_t - \sum \psi_i \times \mu_{t-i}$ 
    ///
    /// For AR(0) models: this equals the mean ( $\mu_t$ )
    /// For AR(p) models: mean minus weighted historical means
    pub deterministic_base: f64,

    /// Seasonal standard deviation ( $\sigma_t$ ) for scaling innovations
    ///
    /// The realized stochastic contribution is: seasonal_std *
    /// where  $\eta_t \sim N(0,1)$  is the correlated innovation
    pub seasonal_std: f64,

    /// AR order (0 = independent, no lag contribution)
    pub ar_order: u8,

    /// AR coefficients [ $\psi_1, \psi_2, \dots, \psi_p$ ]
    ///

```

```

    /// Empty if ar_order = 0. Used to compute lag contribution from state.
    pub ar_coefficients: SmallVec<f64; 6>,

    /// State variable indices for lag values [Y_{t-1}, Y_{t-2}, ..., Y_{t-p}]
    ///
    /// These index into the state vector to retrieve past realizations.
    /// Empty if ar_order = 0.
    pub lag_state_indices: SmallVec<usize; 6>,
}

impl UncertaintyObservationData {
    /// Compute realized value given state and innovation (HOT PATH)
    ///
    /// This is called millions of times per SDDP run. Must be branchless
    /// and allocation-free.
    ///
    /// # Arguments
    /// * `state` - Current state vector (contains lag values Y_{t-i})
    /// * `innovation` - Correlated N(0,1) sample for this entity
    ///
    /// # Returns
    /// Realized value (e.g., inflow in m³/s, load in MW)
    #[inline(always)]
    pub fn realize(&self, state: &[f64], innovation: f64) -> f64 {
        /// Compute lag contribution:  $\sum_i \phi_i \times Y_{t-i}$ 
        let lag_contribution = self.ar_coefficients.iter()
            .zip(self.lag_state_indices.iter())
            .map(|(&phi, &idx)| phi * state[idx])
            .sum::<f64>();

        /// Final realization
        self.deterministic_base + lag_contribution + self.seasonal_std * innovation
    }

    /// Compute realized value without lag (for AR(0) models)
    ///
    /// Optimized path when ar_order = 0 (no state dependency).
    #[inline(always)]
    pub fn realize_independent(&self, innovation: f64) -> f64 {
        debug_assert_eq!(self.ar_order, 0);
        self.deterministic_base + self.seasonal_std * innovation
    }
}

```

Precomputation at Setup:

```

/// Build UncertaintyObservationData for all hydros at a given stage
///
/// Called once during problem construction, not on hot path.
pub fn build_uncertainty_data(
    stage_id: u32,
    hydros: &[Hydro],
    inflow_models: &HashMap<u32, u32>, InflowModel>, // (hydro_id, stage_id) -> model
    constraint_layout: &ConstraintLayout,
    state_layout: &StateLayout,
) -> Vec<UncertaintyObservationData> {
    hydros.iter()
        .filter(|h| h.is_active_at_stage(stage_id))

```

```

.map(|hydro| {
    let model = &inflow_models[&(hydro.id, stage_id)];

    // Compute deterministic base:  $_t - \sum_i _i \times _{t-i}$ 
    let deterministic_base = model.mean_m3s - model.ar_coefficients.iter()
        .enumerate()
        .map(|(i, &phi)| {
            let lag_stage = stage_id.saturating_sub((i + 1) as u32);
            let lag_mean = inflow_models[&(hydro.id, lag_stage)].mean_m3s;
            phi * lag_mean
        })
        .sum::<f64>();

    UncertaintyObservationData {
        entity_id: hydro.id,
        stage_id,
        constraint_row: constraint_layout.inflow_constraint_row(hydro.id),
        deterministic_base,
        seasonal_std: model.std_m3s,
        ar_order: model.ar_order as u8,
        ar_coefficients: model.ar_coefficients.iter().copied().collect(),
        lag_state_indices: (0..model.ar_order)
            .map(|lag| state_layout.inflow_lag_index(hydro.id, lag as u32 + 1))
            .collect(),
    }
})
.collect()
}

```

Hot-Path RHS Update Pattern:

```

impl StageSubproblem {
    /// Update all uncertainty constraints for a scenario realization (HOT PATH)
    ///
    /// This is the critical inner loop of forward/backward pass.
    ///
    /// # Arguments
    /// * `state` - Incoming state vector (storage levels, inflow lags)
    /// * `innovations` - Correlated  $N(0,1)$  innovations for this scenario
    /// * `workspace` - Thread-local workspace with batch buffers
    pub fn update_uncertainty_rhs(
        &self,
        state: &[f64],
        innovations: &[f64],
        workspace: &mut ThreadSolverWorkspace,
    ) {
        // Clear batch buffers
        workspace.rhs_indices.clear();
        workspace.rhs_values.clear();

        // Realize each uncertain quantity
        for (i, obs_data) in self.uncertainty_data.iter().enumerate() {
            let realized = obs_data.realize(state, innovations[i]);
            workspace.rhs_indices.push(obs_data.constraint_row as i32);
            workspace.rhs_values.push(realized);
        }

        // Batch update (single HiGHS call for all constraints)
    }
}

```

```

        workspace.solver.change_rows_bounds_batch(
            &workspace.rhs_indices,
            &workspace.rhs_values,
            &workspace.infinity_buffer[..workspace.rhs_values.len()],
        ).expect("RHS update failed");
    }
}

```

5.4.12 Backward Pass Warm-Start Pattern

Design Rationale: The backward pass computes cuts by solving the stage subproblem for multiple scenarios (branching). All scenarios share the same incoming state but differ in their uncertainty realizations. The forward pass has already solved this stage for one scenario, producing an optimal basis. This basis is often valid or near-valid for the backward branching scenarios, enabling significant warm-start benefits.

Concept:

Forward Pass at Stage t :

State $\hat{x}_{t-1} \rightarrow$ Solve $LP(0) \rightarrow$ Solution $\hat{x}_t$, Basis B_{forward}

Backward Pass at Stage t (K branching scenarios):

Same state $\hat{x}_{t-1}$, different innovations $1, 2, \dots, k$

Scenario 1: $\hat{x}_{t-1} \rightarrow$ Solve $LP(1)$ with warm-start from $B_{\text{forward}} \rightarrow \text{Cut}_1$

Scenario 2: $\hat{x}_{t-1} \rightarrow$ Solve $LP(2)$ with warm-start from $B_{\text{forward}} \rightarrow \text{Cut}_2$

...

Scenario K : $\hat{x}_{t-1} \rightarrow$ Solve $LP(k)$ with warm-start from $B_{\text{forward}} \rightarrow \text{Cut}_k$

Why Warm-Start is Effective:

Aspect	Forward $\rightarrow$ Backward Warm-Start
State variables	Identical (same $\hat{x}_{t-1}$)
Constraint structure	Identical
Uncertainty realization	Different values
RHS changes	Only uncertainty-dependent rows
Typical basis validity	High (70-90% of variables unchanged)
Simplex iterations saved	60-80% vs cold start

Implementation Pattern:

```

/// Context for backward pass warm-starting from forward pass results
///
/// Captures the forward pass state needed to efficiently solve
/// backward branching scenarios.
pub struct BackwardWarmStartContext {
    /// Basis from forward pass solve at this stage
    ///
    /// This basis is the starting point for all backward scenarios.
    /// It was optimal for the forward scenario's uncertainty realization.
    pub forward_basis: Basis,

    /// Incoming state ( $\hat{x}_{t-1}$ ) that was used in forward pass
    ///
    /// All backward scenarios use this same state.
    pub incoming_state: Vec<f64>,
}

```



```

    /// Stage identifier
    pub stage_id: u32,

    /// Forward pass scenario index (for debugging/logging)
    pub forward_scenario_idx: u32,
}

impl ThreadSolverWorkspace {
    /// Solve backward branching scenario with warm-start from forward pass
    ///
    /// # Arguments
    /// * `stage` - Stage identifier
    /// * `problem` - Stage LP problem (shared, read-only)
    /// * `context` - Warm-start context from forward pass
    /// * `innovations` - Uncertainty innovations for this backward scenario
    /// * `uncertainty_data` - Precomputed observation data
    ///
    /// # Returns
    /// Solution with duals for cut computation
    pub fn solve_backward_scenario(
        &mut self,
        stage: u32,
        problem: &LpProblem,
        context: &BackwardWarmStartContext,
        innovations: &[f64],
        uncertainty_data: &[UncertaintyObservationData],
    ) -> Result<BackwardSolution, SolverError> {
        // 1. Set incoming state (same as forward pass)
        self.update_state_constraints(&context.incoming_state);

        // 2. Update uncertainty RHS for this scenario's innovations
        self.update_uncertainty_constraints(
            &context.incoming_state,
            innovations,
            uncertainty_data
        );

        // 3. Set basis from forward pass (warm-start)
        self.solver.set_basis(&context.forward_basis)?;

        // 4. Solve (typically converges quickly due to warm-start)
        let solution = self.solver.solve(problem)?;

        // 5. Extract duals for cut computation
        Ok(BackwardSolution {
            objective: solution.objective_value,
            duals: solution.dual.clone(),
            state_duals: self.extract_state_duals(&solution),
            simplex_iterations: solution.simplex_iterations,
            warm_start_effective: solution.simplex_iterations < 50, // Heuristic
        })
    }
}

/// Solution from backward scenario solve, focused on cut computation
pub struct BackwardSolution {

```

```

    /// Optimal objective value  $Q(x, )$ 
    pub objective: f64,

    /// All constraint duals (for debugging)
    pub duals: Vec<f64>,

    /// State variable duals (cut coefficients)
    ///
    /// These are the values for state-transfer constraints.
    /// Cut coefficient  $_{j} = _j$  for state variable  $j$ .
    pub state_duals: Vec<f64>,

    /// Simplex iterations used
    pub simplex_iterations: u64,

    /// Whether warm-start was effective (low iteration count)
    pub warm_start_effective: bool,
}

```

Warm-Start Validity Considerations:

Important: The forward basis may become infeasible or suboptimal for backward scenarios due to RHS changes. The simplex method handles this gracefully:

1. **Primal infeasibility:** If the basis is primal infeasible (some basic variables violate bounds due to RHS change), dual simplex restores feasibility in few iterations.
2. **Dual infeasibility:** If the basis is dual infeasible (reduced costs have wrong sign), primal simplex restores optimality.
3. **Both:** In rare cases, Phase I may be needed, but starting from a nearby basis is still faster than cold start.

```

impl ThreadSolverWorkspace {
    /// Decide whether to use warm-start for backward scenario
    ///
    /// In most cases, warm-starting is beneficial. The solver handles
    /// infeasible bases automatically via dual simplex.
    fn should_warm_start_backward(
        &self,
        forward_context: &BackwardWarmStartContext,
        backward_scenario_idx: usize,
    ) -> bool {
        // Always attempt warm-start for backward branching
        // The solver will handle basis repair if needed
        //
        // Only skip if:
        // 1. No forward basis available (shouldn't happen)
        // 2. Previous backward scenario failed numerically (rare)

        forward_context.forward_basis.is_valid() && !self.last_solve_failed
    }
}

```

5.4.13 Hot-Path Execution Flow

Design Rationale: This section documents the critical execution path that runs millions of times during SDDP training. Every operation on this path must be:

1. **Allocation-free:** Use pre-allocated thread-local buffers
2. **Cache-friendly:** Access data in contiguous patterns
3. **Branch-minimal:** Avoid unpredictable conditionals

4. NUMA-aware: Access local memory whenever possible

Execution Flow Diagram:

SDDP HOT-PATH EXECUTION FLOW

PER-ITERATION LOOP

FORWARD PASS (parallel across scenarios)

```
for scenario in 0..num_scenarios (OpenMP parallel) {
  state = initial_state
  for stage in 0..num_stages {

    1. UPDATE RHS (batch operation)
      • State transfer:  $x_{\{t-1\}} \rightarrow$  constraint RHS
      • Uncertainty: realize(innovations[scenario])
      • Single call: change_rows_bounds_batch()

      ↓

    2. SOLVE LP
      • Warm-start from cached basis (if same/adjacent stage)
      • solver.run()  $\rightarrow$  returns status
      • ~50-200 simplex iterations (warm) vs 500+ (cold)

      ↓

    3. EXTRACT SOLUTION (into thread-local buffer)
      • Primal values  $\rightarrow$  workspace.primal_solution
      • Cache basis  $\rightarrow$  workspace.cached_basis
      • Update state: state = extract_state(primal)

  }
  store forward_result[scenario]
}
```

↓

BACKWARD PASS (sequential across stages, parallel within stage)

```
for stage in (1..num_stages).rev() {
  for scenario in forward_scenarios (OpenMP parallel) {
    forward_state = forward_result[scenario].state_at(stage-1)
    forward_basis = forward_result[scenario].basis_at(stage)

    for branch in 0..num_branches {

      1. UPDATE RHS (same state, different innovation)
        • State: same forward_state
        • Innovation: branch_innovations[branch]

      ↓
```

```

2. SOLVE LP (warm-start from forward_basis)
  • Set basis from forward pass
  • Typically very fast (basis nearly optimal)

      ↓

3. EXTRACT DUALS (for cut computation)
  • Objective value  $Q(x, \cdot)$ 
  • State duals  $\rightarrow$  cut coefficients
  • Cut RHS:  $= Q - 'x$ 

    }
    aggregate cuts from branches
  }
  // Synchronization point: gather/broadcast cuts for this stage
  MPI_Gather(local_cuts)  $\rightarrow$  MPI_Bcast(selected_cuts)
  apply_cuts_to_local_fcf(stage)
}

```

Call Sequence Pseudo-Code:

```

/// Single forward pass solve (HOT PATH - called ~10M times/run)
///
/// This is the innermost loop of SDDP training. Every microsecond counts.
#[inline(always)]
fn forward_stage_solve(
    workspace: &mut ThreadSolverWorkspace,
    stage: &StageSubproblem,
    incoming_state: &[f64],
    innovations: &[f64],
) -> Result<ForwardStageResult, SolverError> {
    ///
    /// STEP 1: UPDATE RHS (allocation-free, batch operation)
    ///
    /// Time: ~50 s for 500 constraint updates

    /// 1a. Prepare state transfer constraints ( $x_{t-1} \rightarrow$  RHS)
    workspace.prepare_state_rhs(stage, incoming_state);

    /// 1b. Realize uncertainty (precomputed base + *)
    for (i, obs) in stage.uncertainty_data.iter().enumerate() {
        let realized = obs.deterministic_base
            + obs.compute_lag_contribution(incoming_state)
            + obs.seasonal_std * innovations[i];
        workspace.rhs_values[stage.state_constraints.len() + i] = realized;
    }

    /// 1c. Batch RHS update (single HiGHS call)
    workspace.solver.change_rows_bounds_batch(
        &workspace.rhs_indices[..workspace.num_updates],
        &workspace.rhs_lower[..workspace.num_updates],
        &workspace.rhs_upper[..workspace.num_updates],
    )?;
}

```

```

//
// STEP 2: SOLVE LP (warm-start when possible)
//
// Time: ~1-5 ms warm, ~10-50 ms cold

// 2a. Determine warm-start eligibility
let use_warm_start = workspace.cached_basis.is_some()
    && workspace.last_stage.map_or(false, |s| s == stage.id || s.abs_diff(stage.id) == 1);

// 2b. Set basis if warm-starting
if use_warm_start {
    // Basis already in solver from last solve - no action needed
    // (HiGHS retains basis across solves unless explicitly cleared)
}

// 2c. Solve
workspace.solver.run()?;

//
// STEP 3: EXTRACT SOLUTION (into thread-local buffers)
//
// Time: ~20 s

// 3a. Extract primal values
workspace.solver.get_primal_values_into(&mut workspace.primal_buffer);

// 3b. Extract and cache basis
workspace.solver.get_basis_into(&mut workspace.basis_buffer);
workspace.cached_basis = Some(workspace.basis_buffer.clone());
workspace.last_stage = Some(stage.id);

// 3c. Extract outgoing state
let outgoing_state = stage.variable_layout.extract_state(&workspace.primal_buffer);

// 3d. Record objective
let objective = workspace.solver.get_objective_value();

Ok(ForwardStageResult {
    objective,
    outgoing_state,
    primal: workspace.primal_buffer.clone(), // Consider avoiding clone
})
}

```

Memory Access Patterns:

Phase	Data Accessed	Location	Access Pattern
RHS Update	incoming_state	Forward: sequential, Backward: cached	Read-only, contiguous
RHS Update	uncertainty_data	Stage-local	Read-only, contiguous
RHS Update	rhs_indices, rhs_values	Thread-local	Write, contiguous
LP Solve	Solver internal	Thread-local	Read/Write
Solution Extract	primal_buffer	Thread-local	Write, contiguous

Phase	Data Accessed	Location	Access Pattern
Basis Cache	basis_buffer	Thread-local	Write, then read

NUMA Considerations:

```

/// NUMA-aware execution pattern for forward pass
///
/// Each OpenMP thread is pinned to a NUMA node. Key memory placements:
///
/// - Thread workspace: Local NUMA node (first-touch at creation)
/// - Stage subproblem: Shared read-only (replicated per NUMA, or interleaved)
/// - Scenario data: Shared memory region (accessed by all threads)
/// - Forward results: Per-scenario storage (local to computing thread)
impl ForwardPassExecutor {
    pub fn execute_parallel(
        &self,
        scenarios: &[ScenarioData],
        stages: &[StageSubproblem],
    ) -> Vec<ForwardResult> {
        /// OpenMP parallel region with NUMA binding
        /// Each thread processes multiple scenarios, all data NUMA-local

        let results: Vec<ForwardResult> = (0..scenarios.len())
            .into_par_iter()
            .map(|scenario_idx| {
                /// Get thread-local workspace (NUMA-local)
                THREAD_WORKSPACE.with(|ws| {
                    let mut workspace = ws.borrow_mut();

                    let mut state = self.initial_state.clone();
                    let scenario = &scenarios[scenario_idx];
                    let mut stage_results = Vec::with_capacity(stages.len());

                    for stage in stages {
                        let result = forward_stage_solve(
                            &mut workspace,
                            stage,
                            &state,
                            &scenario.innovations[stage.id as usize],
                        ).expect("Forward solve failed");

                        state = result.outgoing_state.clone();
                        stage_results.push(result);
                    }

                    ForwardResult {
                        scenario_idx,
                        total_cost: stage_results.iter().map(|r| r.objective).sum(),
                        stage_results,
                    }
                })
            })
            .collect();

        results
    }
}

```

```

}

Performance Instrumentation Points:

/// Timing points for hot-path profiling
///
/// Use these markers for performance analysis. In production,
/// compile with feature `timing` disabled to eliminate overhead.
#[cfg(feature = "timing")]
mod timing {
    use std::time::Instant;

    thread_local! {
        pub static TIMINGS: RefCell<HotPathTimings> = RefCell::new(HotPathTimings::default());
    }

    #[derive(Default)]
    pub struct HotPathTimings {
        pub rhs_update_ns: u64,
        pub solve_ns: u64,
        pub extract_ns: u64,
        pub solves_count: u64,
    }

    impl HotPathTimings {
        pub fn record_rhs_update(&mut self, start: Instant) {
            self.rhs_update_ns += start.elapsed().as_nanos() as u64;
        }

        pub fn record_solve(&mut self, start: Instant) {
            self.solve_ns += start.elapsed().as_nanos() as u64;
            self.solves_count += 1;
        }

        pub fn record_extract(&mut self, start: Instant) {
            self.extract_ns += start.elapsed().as_nanos() as u64;
        }

        pub fn summary(&self) -> String {
            let avg_solve_us = self.solve_ns / self.solves_count.max(1) / 1000;
            let avg_rhs_us = self.rhs_update_ns / self.solves_count.max(1) / 1000;
            let avg_extract_us = self.extract_ns / self.solves_count.max(1) / 1000;
            format!(
                "Hot path: {} solves, avg {:.0}s solve, {:.0}s RHS, {:.0}s extract",
                self.solves_count, avg_solve_us, avg_rhs_us, avg_extract_us
            )
        }
    }
}

```

5.5 LP Scaling Specification

Purpose: Production SDDP LPs often suffer from numerical ill-conditioning due to:

Issue	Example
Variables spanning wide magnitudes	10^{-6} to 10^9 (e.g., m^3/s vs. total cost in \$)
Constraints with mixed coefficients	Mixed coefficient magnitudes

Issue	Example
Dominant variables	Future cost variable dominating other variables

LP scaling transforms the problem to improve solver numerical stability.

5.5.1 Scaling Transformation Mathematical Definition:

Given original LP: $\min c'x \quad \text{s.t.} \quad Ax \leq b, x \geq 0$

Apply scaling:

- Column scaling: $\tilde{x} = D_c^{-1} x$ (scale variables)
- Row scaling: Multiply each constraint by row scale factor

Scaled LP: $\min \tilde{c}'\tilde{x} \quad \text{s.t.} \quad \tilde{A}\tilde{x} \leq \tilde{b}$

Where:

- $\tilde{c} = D_c \times c$ (scaled objective)
- $\tilde{A} = D_r \times A \times D_c$ (scaled matrix)
- $\tilde{b} = D_r \times b$ (scaled RHS)
- $D_c = \text{diag}(\text{col_scale}), D_r = \text{diag}(\text{row_scale})$

Solution Transformation:

- Primal: $x = D_c \times \tilde{x}$ (unscale to physical units)
- Row duals: $\tilde{r} = D_r \times r$ (unscale duals)
- Reduced costs: $rc = D_c^{-1} \times r\tilde{c}$

5.5.2 Scaling Data Structures

```

/// Scaling factors for an LP problem
///
/// Stored per-stage to allow different scaling for different problem structures.
/// MUST be persisted for:
/// 1. Reverting primal/dual solutions to physical units
/// 2. Computing cut coefficients in physical variable space
/// 3. Checkpoint/resume with consistent scaling
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct LpScaling {
    /// Stage identifier
    pub stage_id: u32,

    /// Column (variable) scaling factors
    /// Physical value = scaled_value * col_scale[j]
    pub col_scale: Vec<f64>,

    /// Row (constraint) scaling factors
    /// Scaled constraint = row_scale[i] * original_constraint
    pub row_scale: Vec<f64>,

    /// Scaling method used
    pub method: ScalingMethod,

    /// Whether scaling was applied (false if well-conditioned)
    pub applied: bool,

    /// Diagnostic metrics
    pub metrics: Option<ScalingMetrics>,
}

```



```

/// Scaling method selection
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum ScalingMethod {
    /// No scaling
    None,

    /// Geometric mean scaling (recommended default)
    GeometricMean {
        max_iterations: u32,
        tolerance: f64,
    },

    /// Equilibration (scale so max |a_ij| = 1 in each row/col)
    Equilibration,

    /// Let solver choose internally
    SolverAuto,
}

/// Diagnostic metrics for scaling quality
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ScalingMetrics {
    /// Coefficient range before scaling
    pub max_coef_before: f64,
    pub min_coef_before: f64,
    pub ratio_before: f64,

    /// Coefficient range after scaling
    pub max_coef_after: f64,
    pub min_coef_after: f64,
    pub ratio_after: f64,
}

```

5.5.3 Scaling Impact on Cut Coefficients

Critical: Cut coefficients must be stored in **physical (unscaled) units** for solver-agnostic portability. Scaling transformations are applied when cuts are added to the solver.

Transformation for Cut Coefficients:

If the solver operates in scaled space and produces scaled duals $\tilde{\pi}$, the physical cut coefficients are:

$$\beta_{\text{physical}} = W' \times D_r \times \tilde{\pi}$$

Where W is the technology matrix linking state variables to constraints.

In terms of scaled state variables, the cut becomes:

$$\theta \geq \alpha + (D_c \times \beta_{\text{physical}})' \times \tilde{x}_{\text{scaled}}$$

Implementation:

```

impl LpScaling {
    /// Transform primal solution from scaled to physical space
    /// physical_x = scaled_x * col_scale
    pub fn unscale_primal(&self, scaled_x: &[f64]) -> Vec<f64> {
        scaled_x.iter()
    }
}

```

```

        .zip(&self.col_scale)
        .map(|(x, s)| x * s)
        .collect()
    }

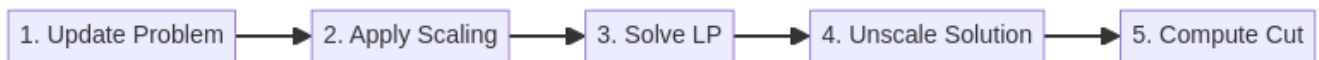
    /// Transform row duals from scaled to physical space
    /// physical_ = scaled_ * row_scale
    pub fn unscale_row_duals(&self, scaled_pi: &[f64]) -> Vec<f64> {
        scaled_pi.iter()
            .zip(&self.row_scale)
            .map(|(pi, s)| pi * s)
            .collect()
    }

    /// Transform cut coefficients from physical to scaled space
    /// (for adding cut to scaled LP)
    /// scaled_ [j] = physical_ [j] * col_scale[j]
    pub fn scale_cut_coefficients(&self, physical_beta: &[f64]) -> Vec<f64> {
        physical_beta.iter()
            .zip(&self.col_scale)
            .map(|(beta, s)| beta * s)
            .collect()
    }
}

```

5.5.4 Scaling Workflow Integration SDDP Solve with Scaling Integration:

Step	Operation	Description
1	Update Problem	Set RHS for state and scenario constraints (physical units)
2	Compute/Apply Scaling	If needed, compute and apply row/column scaling factors
3	Solve LP	Call solver; returns Optimal, Infeasible, Unbounded, or Error
4	Unscale Solution	Convert primal and dual values back to physical units
5	Compute Cut	Extract state duals () and compute intercept () in physical space



Key Point: Cuts are always stored in physical units (+ 'x). Scaling is only applied during the solve step and immediately reversed.

5.5.5 FlatBuffers Schema for Scaling Persistence

```

// File: schemas/scaling.fbs
namespace powers.solver;

/// Scaling method enum
enum ScalingMethod : byte {
    None = 0,
    GeometricMean = 1,
    Equilibration = 2,
}

```

```

    SolverAuto = 3,
}

/// Diagnostic metrics for scaling quality
table ScalingMetrics {
    max_coef_before: double;
    min_coef_before: double;
    ratio_before: double;
    max_coef_after: double;
    min_coef_after: double;
    ratio_after: double;
}

/// Scaling factors for a single stage
table StageScaling {
    stage_id: uint32;
    col_scale: [double] (required);
    row_scale: [double] (required);
    method: ScalingMethod = GeometricMean;
    applied: bool = true;
    metrics: ScalingMetrics;
}

/// Collection of scaling factors for all stages
table ScalingCollection {
    version: uint32 = 1;
    num_stages: uint32;
    stages: [StageScaling] (required);
}

root_type ScalingCollection;
file_identifier "SCAL";
file_extension "scales";

```

6. MPI Communication Structures

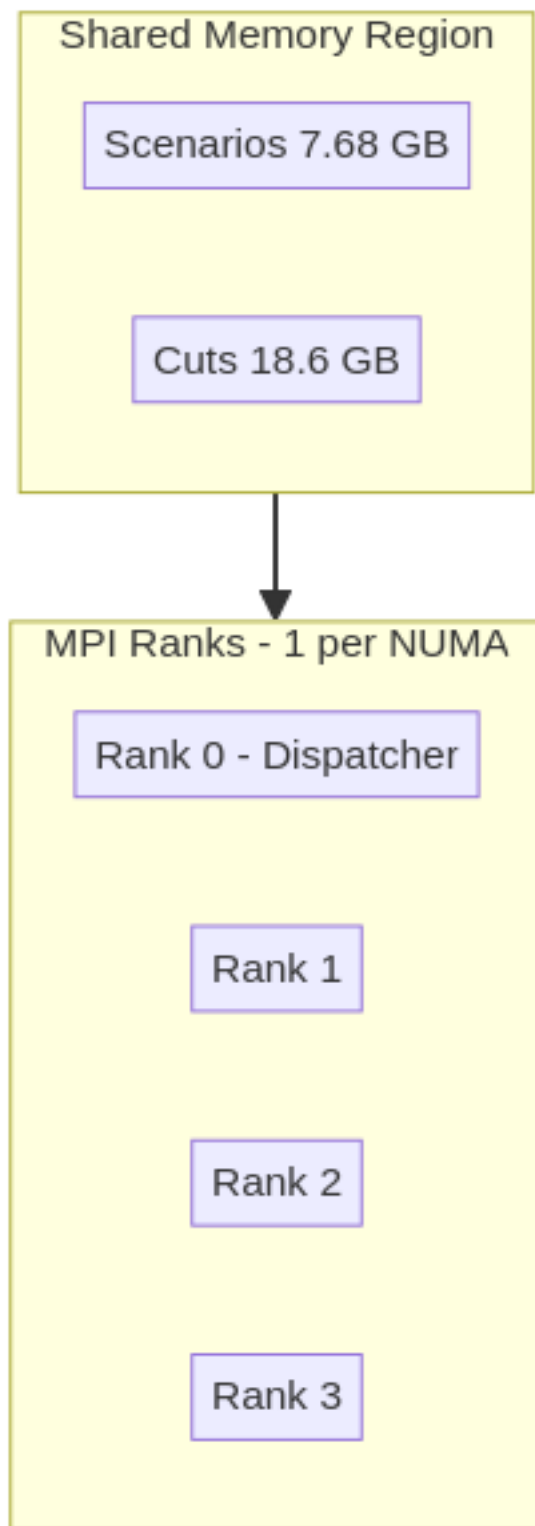
6.1 Hybrid MPI+OpenMP Architecture Overview

Design Philosophy: POWE.RS uses a hybrid MPI+OpenMP architecture to achieve both memory efficiency and optimal load balancing:

Component	Responsibility
MPI (Inter-Node/Inter-NUMA)	Distributes work across NUMA nodes, handles cut aggregation
OpenMP (Intra-NUMA)	Parallelizes forward passes within each rank, leverages shared memory
Shared Memory (MPI Windows)	Scenarios and cuts shared within node, not replicated

This hybrid approach provides the load balancing benefits of NEWAVE’s dynamic dispatch while avoiding its memory replication bottleneck.

6.1.1 Architecture Diagram Hybrid MPI+OpenMP Architecture (Single Node):



Shared Memory Contents:

Region	Size	Access Pattern
Scenario Storage	7.68 GB	1000 passes $\times$ 120 stages $\times$ 20 branches, read by any rank
Cut Storage	18.6 GB	Preallocated slots, written by rank 0, read by all

Region	Size	Access Pattern
--------	------	----------------

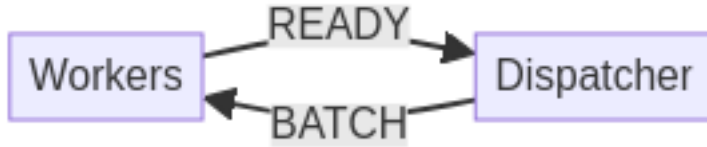
Per-Rank Resources:

- OpenMP threads: 16 per rank
- LP solver instances: 1 per thread
- Local memory: ~240 MB per rank

Memory Comparison:

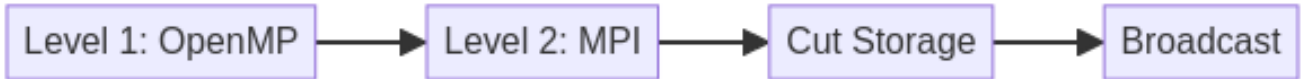
Approach	Memory Required
NEWAVE-style (replicated)	16 ranks $\times$ 26.3 GB = 294 GB
POWE.RS hybrid (shared)	26.3 GB + 4 $\times$ 240 MB = 27.3 GB
Reduction	$\sim 11\times$

6.1.2 Communication Pattern Summary Forward Pass - Dynamic dispatch from rank 0:



- Synchronization: None (async dispatch)
- Load balancing: Optimal (fast workers get more batches)

Backward Pass - Two-level reduction per stage ($t = T-1$ down to 1):



Level	Operation	Result
1 - OpenMP	Thread-local reduction within rank	local_ , local_ per rank
2 - MPI	MPI_Reduce + MPI_Bcast	All ranks have identical global_ , global_
Storage	Rank 0 writes, fence, replicated selection	Identical LP structure on all ranks

- Synchronization: MPI_Reduce + MPI_Bcast per stage (implicit barrier)
- Reproducibility: Deterministic order (reduce to rank 0, then broadcast)

6.1.3 Key Design Decisions

Aspect	Decision	Rationale
Intra-rank parallelism	OpenMP (not Rayon)	HPC cluster compatibility, HiGHS integration, mature MPI+OpenMP patterns
Work distribution	Dynamic dispatch from rank 0	Optimal load balancing like NEWAVE, but with shared memory benefit

Aspect	Decision	Rationale
Batch size	= OpenMP thread count	One forward pass per thread, full utilization
Scenario storage	MPI shared memory (MPI_Win)	No replication within node, any rank can access any pass
Cut storage	MPI shared memory (MPI_Win)	Single copy per node, replicated selection for consistency
Cut aggregation	MPI_Reduce + MPI_Bcast	Deterministic order for reproducibility
Cut formulation	Single-cut (primary)	Simpler, well-suited for MPI_Allreduce pattern

6.1.4 Deployment Configuration

Example: 2-socket AMD EPYC with 64 cores per socket, 4 NUMA nodes per socket
Total: 128 cores, 8 NUMA nodes

```
# MPI: 1 rank per NUMA node
export MPI_RANKS=8

# OpenMP: All cores in NUMA node minus 1 for dispatcher (rank 0 only)
export OMP_NUM_THREADS=16
export OMP_PROC_BIND=close      # Keep threads on same NUMA node
export OMP_PLACES=cores         # Bind to physical cores
export OMP_SCHEDULE=static      # Reproducible scheduling

# Launch command
mpirun -np 8 \
  --map-by ppr:1:numa \
  --bind-to numa \
  -x OMP_NUM_THREADS \
  -x OMP_PROC_BIND \
  -x OMP_PLACES \
  ./powers_sddp --config case/config.json
```

6.2 Message Structures

```
/// Cut data for MPI transmission
#[repr(C)]
pub struct CutMessage {
    pub stage_id: u32,
    pub iteration: u32,
    pub forward_pass_idx: u32,
    pub rank_id: u32,
    pub rhs: f64,
    pub state_dimension: u32,
    _padding: u32,
    // Followed by: coefficients[state_dimension]
    // Followed by: state_coefficients[state_dimension]
}

impl CutMessage {
    /// Size in bytes for a given state dimension
    pub fn size_bytes(state_dim: usize) -> usize {
        std::mem::size_of::<Self>() + 2 * state_dim * std::mem::size_of::<f64>()
    }
}
```

```

}

/// FCF update message from master
#[repr(C)]
pub struct FcfUpdateMessage {
    pub stage_id: u32,
    pub iteration: u32,
    pub num_new_cuts: u32,
    pub num_removed_cuts: u32,
    pub num_returned_cuts: u32,
    _padding: u32,
    // Followed by: new_cut_data (variable size)
    // Followed by: removed_cut_ids[num_removed_cuts]
    // Followed by: returned_cut_ids[num_returned_cuts]
}

/// Persistent communication handles (MPI 4.0+)
pub struct PersistentComm {
    /// Gather: all ranks → master (cut data)
    pub cut_gather: PersistentRequest,

    /// Broadcast: master → all ranks (FCF updates)
    pub fcf_broadcast: PersistentRequest,

    /// Allreduce: lower bound computation
    pub bound_allreduce: PersistentRequest,

    /// Preallocated buffers
    pub cut_send_buffer: Vec<u8>,
    pub cut_recv_buffer: Vec<u8>,
    pub fcf_buffer: Vec<u8>,
}

```

6.3 Synchronization Points

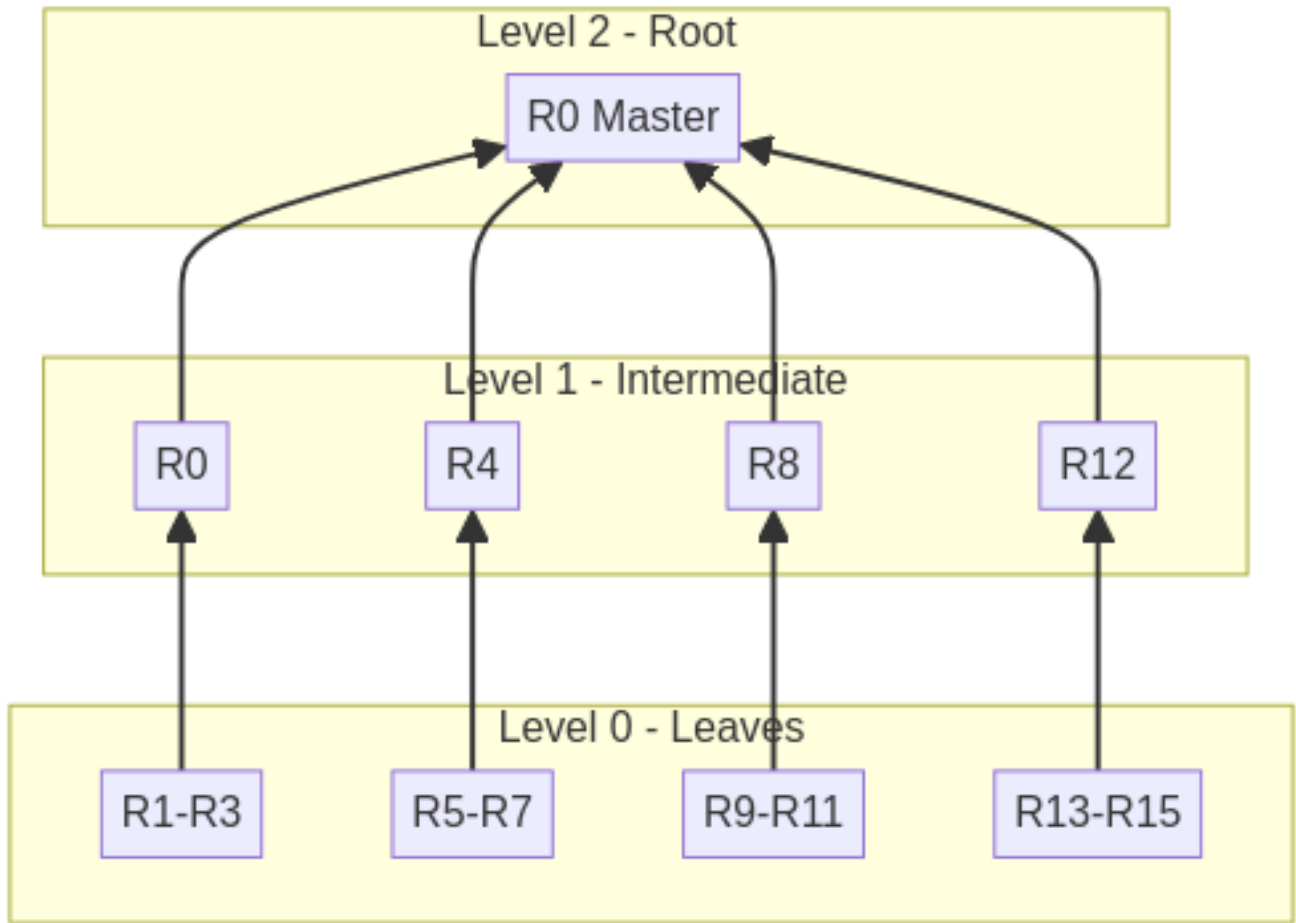
Phase	Sync Type	Data Size	Frequency
Forward pass end	None	-	Per iteration
Backward per-stage	MPI_Gather	~50KB × ranks	Per stage
FCF update	MPI_Bcast	~100KB	Per stage
Lower bound	MPI_Allreduce	8 bytes	Per iteration
Checkpointing	MPI_Barrier	-	Every N iterations

6.4 Hierarchical Cut Aggregation

Problem: With flat gather-to-master pattern, rank 0 becomes a serialization bottleneck at scale. For 128 ranks sending 50KB each, rank 0 must process 6.4MB of receives sequentially, adding ~100ms overhead per stage.

Solution: Hierarchical tree-based aggregation distributes the aggregation work across intermediate ranks.

Hierarchical Aggregation (fanout=4, 16 ranks):



Benefits:

- Reduces master receive operations from $N-1$ to $\log_{\text{fanout}}(N)$
- Distributes aggregation computation across tree
- Enables partial cut selection at intermediate levels (optional)

Implementation Protocol:

```

```rust
/// Hierarchical aggregation tree node
pub struct AggregationNode {
 pub rank: i32,
 pub parent: Option<i32>,
 pub children: Vec<i32>,
 pub level: u32,
}

impl AggregationNode {
 /// Build aggregation tree for given world size and fanout
 pub fn build_tree(world_size: i32, fanout: i32) -> Vec<AggregationNode> {
 // Level 0: all ranks are leaves
 // Level 1+: ranks at positions 0, fanout, 2*fanout, ... are aggregators
 // Continue until single root (rank 0)
 todo!()
 }
}

```



```

/// Aggregation protocol per stage
pub fn hierarchical_aggregate(
 local_cuts: &[CutMessage],
 tree: &AggregationNode,
 comm: &MpiComm,
) -> Option<Vec<CutMessage>> {
 // Step 1: Receive from children (if any)
 let mut all_cuts = local_cuts.to_vec();
 for child in &tree.children {
 let child_cuts = comm.recv::<Vec<CutMessage>>(*child);
 all_cuts.extend(child_cuts);
 }

 // Step 2: Optional local aggregation (cut selection at intermediate level)
 // This reduces data volume but may affect cut quality
 // let aggregated = local_cut_selection(&all_cuts);

 // Step 3: Send to parent (if not root)
 if let Some(parent) = tree.parent {
 comm.send(&all_cuts, parent);
 None // Non-root ranks don't return cuts
 } else {
 Some(all_cuts) // Root returns all aggregated cuts
 }
}

```

**Configuration:**

Ranks	Recommended Fanout	Tree Depth	Master Receives
16	4	2	4
64	8	2	8
128	8	3	~16
512	16	2	32
2048	16	3	~128

## 6.5 Backward Pass Computation Modes

**Design Rationale:** The SDDP backward pass can be computed in two valid modes, each with different trade-offs between convergence rate and wall-clock time. Both modes produce valid lower bounds; the difference is in cut tightness.

**6.5.1 Mathematical Foundation** At iteration  $k$ , the backward pass computes cuts for stage  $t$  by solving sub-problems at stage  $t + 1$  and extracting dual information. The key design question is: **which approximation of the future cost function should stage  $t + 1$  use?**

**Option A - Sequential (uses  $V_{t+1}^k$ ):**

$$Q_t^k(x_{t-1}, \omega_t) = \min_{x_t} \{c_t^\top x_t + Q_{t+1}^k(x_t)\}$$

The superscript  $k$  means we use the freshly-computed approximation from the current iteration. This requires that stage  $t + 1$ 's cuts have already been computed before we compute stage  $t$ .

**Option B - Pipelined (uses  $V_{t+1}^{k-1}$ ):**

$$Q_t^k(x_{t-1}, \omega_t) = \min_{x_t} \{c_t^\top x_t + Q_{t+1}^{k-1}(x_t)\}$$

The superscript  $k-1$  means we use the approximation from the previous iteration. This allows stages to be computed in parallel or with overlapped communication.

**Mathematical Validity:** Both approaches produce valid cuts that are supporting hyperplanes of the true value function. The difference is only in tightness—sequential cuts are generally tighter because they incorporate more recent information.

*“It doesn’t matter what order we visit the nodes to generate these cuts for. For example, we could compute them all in parallel, using the current approximations of  $V^K_i$ .”* — SDDP.jl Documentation

**6.5.2 Sequential Mode (Default)** The standard backward pass walks stages from  $T$  to 1. At each stage, it waits for the downstream stage’s cuts to be computed and broadcast before proceeding.

**Sequential Backward Pass Timeline:**

Time	Stage T	Stage T-1	Stage T-2
0-20	Compute	Wait	Wait
20-30	Gather	Wait	Wait
30-40	Bcast	Wait	Wait
40-60	Done	Compute	Wait
60-70	Done	Gather	Wait
70-80	Done	Bcast	Wait
80-100	Done	Done	Compute

**Key:** Barrier between each stage ensures  $V_{t+1}^k$  is available before computing  $V_t^k$

**Implementation:**

```

/// Sequential backward pass (default mode)
///
/// Produces tighter cuts by using freshly-computed V_{t+1}^k at each stage.
/// This is the recommended mode and matches SDDP.jl's default behavior.
pub fn backward_pass_sequential(
 iteration: u32,
 forward_results: &ForwardResults,
 fcf: &mut FutureCostFunction,
 comm: &MpiComm,
 num_stages: u32,
) {
 for stage in (1..num_stages).rev() {
 // Step 1: Compute cuts for current stage
 // Uses $V_{stage+1}^k$ which was just updated in the previous loop iteration
 let local_cuts = compute_stage_cuts(stage, forward_results, fcf);

 // Step 2: Gather cuts from all ranks (hierarchical aggregation)
 let all_cuts = hierarchical_gather(&local_cuts, comm);

 // Step 3: Master selects/aggregates cuts
 let selected_cuts = if comm.is_master() {
 cut_selection(all_cuts.unwrap(), &fcf.selection_config)
 } else {
 Vec::new()
 };
 }
};

```

```

// Step 4: Broadcast selected cuts to all ranks (BLOCKING)
let broadcast_cuts = comm.bcast(&selected_cuts, MASTER_RANK);

// Step 5: All ranks apply cuts to their local FCF copy
// This updates V_stage^k, ready for stage-1 to use
fcf.apply_cuts(stage, &broadcast_cuts);

// Implicit barrier: all ranks synchronized before next stage
}
}

```

#### Advantages:

- Tighter cuts (uses most recent  $V_{t+1}^k$ )
- Faster convergence (fewer iterations to optimality gap)
- Simpler implementation (no overlapping state to manage)
- Default in SDDP.jl and most production implementations

#### Disadvantages:

- Synchronization barriers at each stage
- Cannot overlap computation with communication

### 6.5.3 Pipelined Mode (Future Enhancement)

**Status:** Documented for future implementation. Not available in v2.0.

The pipelined backward pass overlaps computation with communication by using  $V_{t+1}^{k-1}$  (previous iteration's approximation) instead of  $V_{t+1}^k$ .

#### Pipelined Backward Pass Timeline:

Time	Stage T	Stage T-1	Stage T-2	Comm T	Comm T-1
0-10	Compute	-	-	-	-
10-20	Compute	Compute	-	-	-
20-25	Done	Compute	Compute	-	-
25-30	Done	Compute	Compute	Ibcast	-
30-35	Done	Irecv	Compute	Ibcast	-
35-40	Done	Irecv	Compute	Ibcast	Ibcast
40-50	Done	Done	Done	Done	Ibcast

**Key Insight:** Stage T-1 uses  $V_T^{k-1}$  (from previous iteration), **NOT**  $V_T^k$ . This allows overlapped execution but produces looser cuts.

#### Implementation (for future reference):

```

/// Pipelined backward pass (future enhancement)
///
/// Overlaps communication with computation by using V_{t+1}^{k-1}.
/// Produces looser cuts but reduces wall-clock time per iteration.
pub fn backward_pass_pipelined(
 iteration: u32,
 forward_results: &ForwardResults,
 fcf: &mut FutureCostFunction,
 comm: &PersistentComm,
 num_stages: u32,
) {
 let mut pending_broadcast: Option<(u32, MpiRequest)> = None;

```

```

for stage in (1..num_stages).rev() {
 // Step 1: Check if previous stage's broadcast completed
 if let Some((prev_stage, req)) = pending_broadcast.take() {
 req.wait(); // Ensure FCF update for prev_stage is applied
 }

 // Step 2: Compute cuts for current stage (can proceed immediately)
 // NOTE: Uses $V_{\{stage+1\}}^{k-1}$ from PREVIOUS iteration
 let local_cuts = compute_stage_cuts(stage, forward_results, fcf);

 // Step 3: Gather cuts (hierarchical)
 let all_cuts = hierarchical_aggregate(&local_cuts, &comm.tree, &comm.comm);

 // Step 4: Master aggregates and prepares broadcast
 let fcf_update = if comm.is_master() {
 let selected = cut_selection(all_cuts.unwrap());
 fcf.apply_update(stage, &selected);
 prepare_broadcast_message(stage, &selected)
 } else {
 FcfUpdateMessage::default()
 };

 // Step 5: Start non-blocking broadcast (continues in background)
 let bcast_req = comm.fcf_broadcast.ibcast(&fcf_update);
 pending_broadcast = Some((stage, bcast_req));

 // Loop continues to next stage while broadcast proceeds
}

// Final: Wait for last broadcast
if let Some((_, req)) = pending_broadcast {
 req.wait();
}
}

```

#### 6.5.4 Trade-off Analysis

Aspect	Sequential (Default)	Pipelined (Future)
Cut source	$V_{t+1}^k$ (current iteration)	$V_{t+1}^{k-1}$ (previous iteration)
Cut quality	Tighter	Looser
Iterations to converge	Fewer	More
Time per iteration	Longer (barriers)	Shorter (overlapped)
Implementation	Simple	Complex (async state)
MPI primitives	MPI_Bcast	MPI_Ibcast, MPI_Wait

#### Latency Reduction Estimate (Pipelined):

Stages	Barrier Overhead (Sequential)	Pipelined Overhead	Reduction
60	$\sim 60 \times 5\text{ms} = 300\text{ms}$	$\sim 60\text{ms}$ (overlapped)	80%
120	$\sim 120 \times 5\text{ms} = 600\text{ms}$	$\sim 100\text{ms}$	83%

Stages	Barrier Overhead (Sequential)	Pipelined Overhead	Reduction
240	$\sim 240 \times 5\text{ms} = 1.2\text{s}$	$\sim 180\text{ms}$	85%

#### When to consider pipelined mode (future):

- Very large stage counts ( $>100$  stages)
- High network latency between nodes
- When iteration count is less important than wall-clock time
- After profiling shows backward pass communication is the bottleneck

#### 6.5.5 Configuration

```
{
 "training": {
 "backward_pass": {
 "mode": "sequential"
 }
 }
}
```

Mode	Description	Status
sequential	Standard backward pass using $V_{t+1}^k$ . <b>Default.</b>	Implemented
pipelined	Overlapped backward pass using $V_{t+1}^{k-1}$ .	<b>DEFERRED</b>

**Future work:** Implement pipelined mode when production benchmarks indicate that communication latency is the dominant bottleneck. The decision should be based on profiling data showing that barrier synchronization accounts for  $>30\%$  of backward pass time.

#### References:

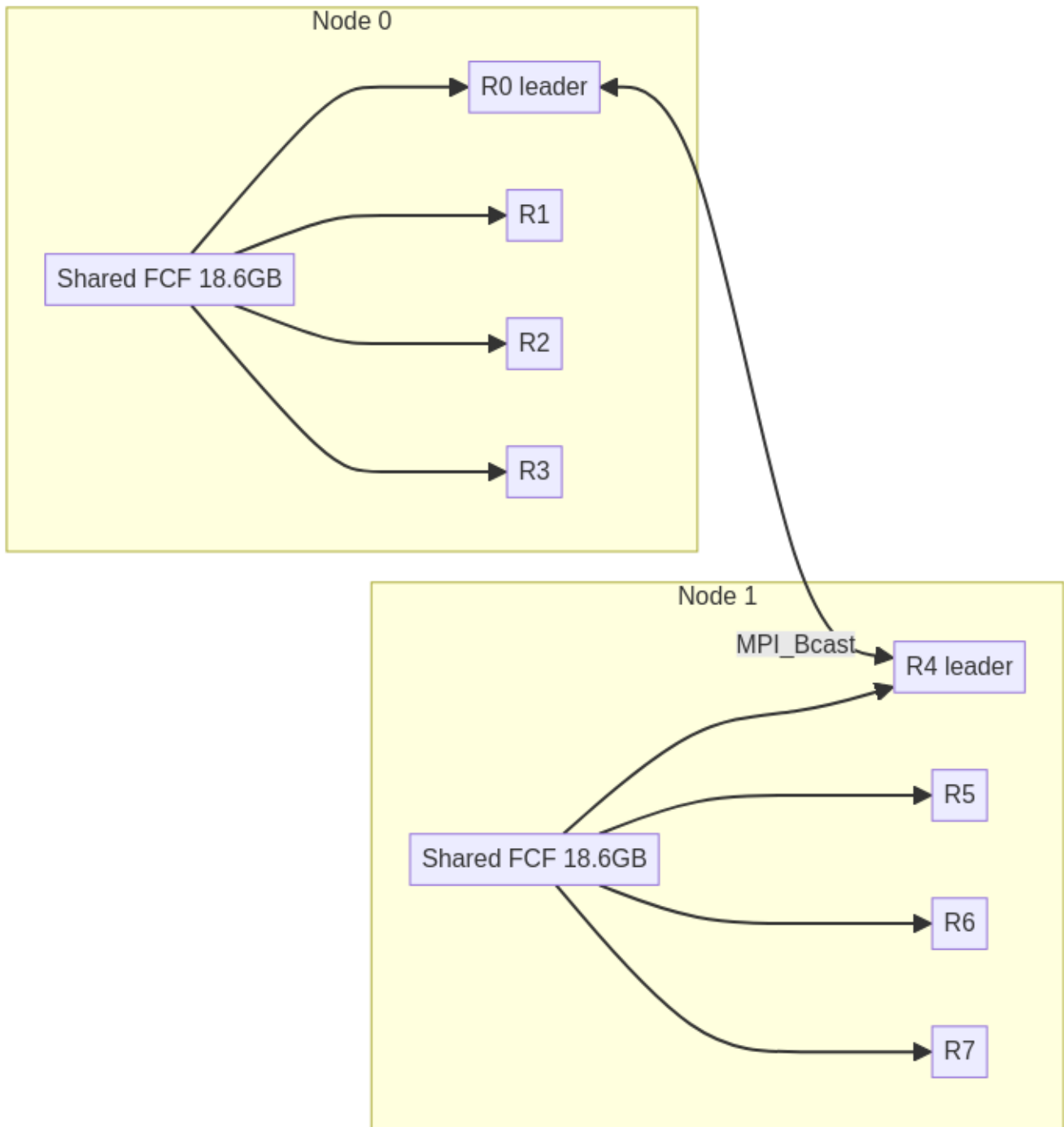
- Pereira, M.V.F. & Pinto, L.M.V.G. (1991). “Multi-stage stochastic optimization applied to energy planning.” *Mathematical Programming*, 52, 359-375.
- Dowson, O. (2020). SDDP.jl documentation: [https://sddp.dev/stable/explanation/theory\\_intro/](https://sddp.dev/stable/explanation/theory_intro/)
- Philpott, A.B., de Matos, V.L., & Finardi, E.C. (2013). “On solving multistage stochastic programs with coherent risk measures.” *Operations Research*, 61(4), 957-970.

#### 6.6 Intra-Node Shared Memory (MPI Windows)

**Problem:** Each MPI rank maintains a full FCF replica (18.6 GB at production scale). With 4 ranks per node, this requires 42.8 GB just for cuts.

**Solution:** Use MPI shared memory windows so ranks on the same node share a single FCF copy.

#### Intra-Node Shared Memory Architecture:



#### Per-Rank Access Pattern:

- Reads FCF via shared pointer (zero-copy)
- Writes to thread-local buffer
- Leader applies updates after fence

#### Implementation:

```

/// Shared memory FCF manager
pub struct SharedFcf {
 /// MPI window for shared memory
 win: MpiWin,

```

```

/// Base pointer to shared memory (accessible by all ranks on node)
base_ptr: *mut u8,

/// Total size in bytes
total_size: usize,

/// Shared memory communicator (ranks on same node)
shm_comm: MpiComm,

/// Rank within shared memory communicator
shm_rank: i32,

/// Whether this rank is the node leader (allocates memory)
is_leader: bool,
}

impl SharedFcf {
 pub fn new(world_comm: &MpiComm, fcf_size: usize) -> Self {
 /// Create shared memory communicator
 let shm_comm = world_comm.split_type(MPI_COMM_TYPE_SHARED);
 let shm_rank = shm_comm.rank();
 let is_leader = shm_rank == 0;

 /// Only leader allocates; others get size 0
 let alloc_size = if is_leader { fcf_size } else { 0 };

 /// Allocate shared memory window
 let (win, local_ptr) = MpiWin::allocate_shared(alloc_size, &shm_comm);

 /// All ranks query rank 0's pointer to get shared base
 let (base_ptr, _) = win.shared_query(0);

 Self {
 win,
 base_ptr,
 total_size: fcf_size,
 shm_comm,
 shm_rank,
 is_leader,
 }
 }

 /// Read access (all ranks)
 pub fn read_cuts(&self, stage: u32) -> &[BendersCut] {
 /// Direct pointer access - no MPI communication needed
 unsafe {
 let offset = self.stage_offset(stage);
 let ptr = self.base_ptr.add(offset) as *const BendersCut;
 std::slice::from_raw_parts(ptr, self.cuts_per_stage(stage))
 }
 }

 /// Write access (leader only, with window lock)
 pub fn apply_update(&mut self, stage: u32, update: &FcfUpdateMessage) {
 assert!(self.is_leader, "Only leader can write to shared FCF");

 /// Lock window for exclusive access

```

```

self.win.lock(MPI_LOCK_EXCLUSIVE, 0);

// Apply updates directly to shared memory
unsafe {
 let offset = self.stage_offset(stage);
 let ptr = self.base_ptr.add(offset) as *mut BendersCut;
 // ... apply update ...
}

self.win.unlock(0);

// Memory barrier ensures visibility to other ranks
self.win.sync();
}
}

```

#### Memory Savings:

Configuration	Without Sharing	With Sharing	Savings
4 ranks/node, 18.6 GB FCF	74.4 GB/node	18.6 GB/node	75%
8 ranks/node, 18.6 GB FCF	148.8 GB/node	18.6 GB/node	87.5%
16 ranks/node, 18.6 GB FCF	297.6 GB/node	18.6 GB/node	93.75%

### 6.7 NUMA-Aware Memory Management

**Background:** Modern EPYC systems (e.g., AWS c7a.48xlarge) have 8 NUMA nodes. Memory access latency varies by ~3x between local and remote NUMA nodes. For 192-thread ranks, proper NUMA placement is critical.

#### EPYC 8-NUMA TOPOLOGY

NUMA 0	NUMA 1	NUMA 2	NUMA 3
24 cores	24 cores	24 cores	24 cores
Local Memory			

#### Interconnect

24 cores	24 cores	24 cores	24 cores
		Local Memory	
NUMA 4	NUMA 5	NUMA 6	NUMA 7

Memory Latency (ns):

- Local NUMA: ~80ns
- Adjacent NUMA: ~120ns
- Remote NUMA: ~200ns



## First-Touch Initialization:

```
/// NUMA-aware array allocation with first-touch initialization
pub fn allocate_numa_aware<T: Default + Send>(size: usize) -> Vec<T> {
 /// Allocate uninitialized
 let mut vec = Vec::with_capacity(size);
 unsafe { vec.set_len(size); }

 /// Initialize in parallel - each thread touches its portion
 /// Memory pages are allocated on the NUMA node of the touching thread
 let num_threads = rayon::current_num_threads();
 let chunk_size = (size + num_threads - 1) / num_threads;

 vec.par_chunks_mut(chunk_size)
 .for_each(|chunk| {
 for elem in chunk.iter_mut() {
 *elem = T::default(); // First touch allocates on local NUMA
 }
 });

 vec
}

/// Scenario data partitioned by NUMA node
pub struct NumaPartitionedScenarios {
 /// Scenario data, partitioned so each NUMA node's threads access local data
 partitions: Vec<Vec<ScenarioData>>,

 /// Mapping from scenario_id to (numa_node, local_index)
 scenario_map: Vec<(usize, usize)>,
}

impl NumaPartitionedScenarios {
 pub fn new(scenarios: Vec<ScenarioData>, num_numa_nodes: usize) -> Self {
 let scenarios_per_node = (scenarios.len() + num_numa_nodes - 1) / num_numa_nodes;

 /// Partition scenarios across NUMA nodes
 let partitions: Vec<Vec<ScenarioData>> = (0..num_numa_nodes)
 .into_par_iter()
 .map(|numa_id| {
 let start = numa_id * scenarios_per_node;
 let end = std::cmp::min(start + scenarios_per_node, scenarios.len());

 /// Clone data on each NUMA node (first-touch allocates locally)
 scenarios[start..end].to_vec()
 })
 .collect();

 /// Build index map
 let scenario_map = (0..scenarios.len())
 .map(|s| {
 let numa = s / scenarios_per_node;
 let local = s % scenarios_per_node;
 (numa, local)
 })
 .collect();
 }
}
```

```

 Self { partitions, scenario_map }
 }

 pub fn get(&self, scenario_id: usize) -> &ScenarioData {
 let (numa, local) = self.scenario_map[scenario_id];
 &self.partitions[numa][local]
 }
}

```

## Deployment Configuration (SLURM Best Practices):

```

#!/bin/bash

#=====
POWERS SDDP Solver - SLURM Job Script Template
Optimized for hybrid MPI+OpenMP on NUMA systems
#=====

#SBATCH --job-name=powers-sddp
#SBATCH --output=powers-%j.out
#SBATCH --error=powers-%j.err

#=====
RESOURCE ALLOCATION
#=====
#SBATCH --nodes=4 # Number of compute nodes
#SBATCH --ntasks-per-node=1 # One MPI rank per node (recommended)
#SBATCH --cpus-per-task=192 # All cores for OpenMP threads
#SBATCH --mem=0 # All available memory per node
#SBATCH --exclusive # Exclusive node access
#SBATCH --time=24:00:00 # Maximum runtime

#=====
PARTITION (site-specific)
#=====
#SBATCH --partition=compute
#SBATCH --account=my_project

#=====
ENVIRONMENT SETUP
#=====

module purge
module load openmpi/4.1.5

#=====
OPENMP CONFIGURATION
CRITICAL: Use SLURM's computed value - DO NOT hardcode!
#=====

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PROC_BIND=close # Keep threads close for NUMA locality
export OMP_PLACES=cores # One thread per physical core
export OMP_STACKSIZE=64M # Stack for deep recursion

#=====
JOB INFORMATION
#=====

```

```

echo "=====
echo "POWE.RS SDDP Job Information"
echo "=====
echo "Job ID: ${SLURM_JOB_ID}"
echo "Nodes: ${SLURM_JOB_NUM_NODES}"
echo "Tasks/Node: ${SLURM_NTASKS_PER_NODE}"
echo "CPUs/Task: ${SLURM_CPUS_PER_TASK}"
echo "OMP_NUM_THREADS: ${OMP_NUM_THREADS}"
echo "Memory/Node: ${SLURM_MEM_PER_NODE:-all} MB"
echo "Node List: ${SLURM_JOB_NODELIST}"
echo "=====

#=====
RUN APPLICATION (config uses "auto" - will read SLURM vars)
#=====

CASE_DIR="${1:-./case}"

srun --cpu-bind=verbose \
 --distribution=block:block \
 ./powers train --config "${CASE_DIR}/config.json"

exit $?

PBS/Torque Equivalent:

#!/bin/bash
#PBS -N powers-sddp
#PBS -l nodes=4:ppn=48
#PBS -l mem=512gb
#PBS -l walltime=24:00:00

export OMP_NUM_THREADS=${PBS_NUM_PPN}
cd ${PBS_O_WORKDIR}
mpirun -np $(cat ${PBS_NODEFILE} | wc -l) \
 -hostfile ${PBS_NODEFILE} \
 ./powers train --config case/config.json

```

## 6.8 Performance Monitoring

The following metrics should be collected and reported in `training/timing/mpi_ranks.parquet`:

Metric	Description	Diagnostic Use
<code>computation_time_ms</code>	Time in LP solves and cut computation	Baseline work
<code>communication_time_ms</code>	Time in MPI calls	Communication overhead
<code>idle_time_ms</code>	Time waiting at barriers	Load imbalance
<code>gather_time_ms</code>	Time in cut gather phase	Aggregation bottleneck
<code>bcast_time_ms</code>	Time in FCF broadcast	Distribution overhead
<code>memory_high_water_mb</code>	Peak RSS during iteration	Memory pressure
<code>numa_local_ratio</code>	Fraction of local NUMA accesses	Memory placement quality

### Load Imbalance Detection:

```

/// Analyze per-rank timing for load imbalance
pub fn analyze_load_balance(rank_timings: &[RankTiming]) -> LoadBalanceReport {
 let compute_times: Vec<f64> = rank_timings.iter()
 .map(|r| r.computation_time_ms as f64)
 .collect();

 let mean = compute_times.iter().sum::<f64>() / compute_times.len() as f64;
 let max = compute_times.iter().cloned().fold(0.0_f64, f64::max);
 let min = compute_times.iter().cloned().fold(f64::MAX, f64::min);

 let imbalance_ratio = (max - min) / mean;
 let slowest_rank = rank_timings.iter()
 .enumerate()
 .max_by(|(_, a), (_, b)| a.computation_time_ms.cmp(&b.computation_time_ms))
 .map(|(i, _)| i)
 .unwrap();

 LoadBalanceReport {
 mean_compute_ms: mean,
 max_compute_ms: max,
 min_compute_ms: min,
 imbalance_ratio,
 slowest_rank,
 recommendation: if imbalance_ratio > 0.2 {
 "Consider dynamic scenario distribution or adaptive load balancing"
 } else {
 "Load balance acceptable"
 },
 }
}

```

## 6.9 HPC Implementation Requirements (Critical)

**HPC Review Status:** Reviewed 2026-01-19. This section documents critical implementation requirements identified during specialist review that must be addressed for correct and performant operation.

### 6.9.1 Cut Slot Management: Thread-Safety Analysis

**Resolution:** The “thread-safety bug” identified in the original review is a **non-issue** with the chosen full preallocation strategy. This section documents the analysis and confirms thread-safety.

#### Original Concern:

The initial review flagged that `CutSlotManager.allocate()` using `&mut self` with non-atomic operations would cause data races under OpenMP parallelism.

#### Why This Is Not a Problem:

With full preallocation and deterministic slot assignment (Section 5.4.3):

##### 1. Slots are COMPUTED, not allocated at runtime

```

// Deterministic slot computation - pure function, no mutation
slot = warm_start_count + iteration * forward_passes + forward_pass_idx

```

##### 2. No runtime allocation during parallel solve

- All slots are pre-allocated at LP construction (single-threaded startup)
- During forward/backward passes, threads compute their slot indices
- No calls to `allocate()` during parallel execution

##### 3. Cut addition is deterministic

- Each (iteration, forward\_pass) maps to exactly one slot
- No contention: different threads write to different slots
- LP row updates are independent (different rows)

#### Thread-Safety Guarantees:

Operation	When	Thread-Safety
LP construction	Startup (single-threaded)	N/A
Warm-start loading	Startup (single-threaded)	N/A
Slot computation	Runtime (parallel)	Safe: pure function, no state
Cut coefficient write	Runtime (parallel)	Safe: threads write different rows
Bound toggle	Runtime (parallel)	Safe: threads toggle different rows
Bitmap update	After parallel section	Safe: single-threaded or batch merge

#### Implementation Pattern:

```
// During parallel forward pass - NO ALLOCATION
#[omp_parallel_for]
for scenario_idx in 0..num_scenarios {
 let forward_pass_idx = omp_get_thread_num() as u32;

 // Deterministic slot computation (no locks, no atomics)
 let slot = cut_manager.slot_for_new_cut(iteration, forward_pass_idx);

 // Each thread writes to its own slot (no contention)
 lp.set_cut_coefficients(slot, rhs, &coefficients, &cut_manager);
}

// After parallel section (single-threaded)
for fwd_pass in 0..num_forward_passes {
 let slot = cut_manager.slot_for_new_cut(iteration, fwd_pass as u32);
 cut_manager.activate(slot);
}
```

**Removed Implementation:** The CutSlotManagerThreadSafe with crossbeam\_queue::ArrayQueue and atomic CAS loop is **not needed** and has been removed. The simple CutSlotManager from Section 5.4.3 is sufficient.

**Summary:** Full preallocation eliminates runtime allocation, making thread-safety concerns moot. The design is both simpler and faster than a dynamic allocation approach with atomic operations.

#### 6.9.2 NUMA-Aware FCF Allocation

**CRITICAL:** Standard MPI\_Win\_allocate\_shared allocates the entire 18.6GB FCF on a **single NUMA node**, causing 3x latency penalty for threads on remote NUMA nodes.

**Problem:** On 8-NUMA EPYC systems (AWS c7a.48xlarge), threads on NUMA 7 accessing FCF allocated on NUMA 0 experience ~200ns vs ~80ns latency.

#### Required Fix - NUMA-Distributed Allocation:

```
/// NUMA-aware shared FCF allocation
pub struct NumaDistributedFcf {
 /// Per-NUMA-node cut storage
 numa_partitions: Vec<NumaCutPartition>,
}
```

```

 /// Stage → NUMA node assignment (round-robin or based on access patterns)
 stage_to_numa: Vec<usize>,

 /// MPI shared window (for inter-process sharing)
 shm_win: MPI_Win,
}

impl NumaDistributedFcf {
 pub fn allocate(num_stages: usize, cuts_per_stage: usize, num_numa_nodes: usize) -> Self {
 /// Round-robin stage assignment across NUMA nodes
 let stage_to_numa: Vec<usize> = (0..num_stages)
 .map(|s| s % num_numa_nodes)
 .collect();

 /// Count stages per NUMA node
 let stages_per_numa: Vec<usize> = (0..num_numa_nodes)
 .map(|n| stage_to_numa.iter().filter(|&&s| s == n).count())
 .collect();

 /// Allocate each partition with first-touch on target NUMA node
 let numa_partitions: Vec<NumaCutPartition> = (0..num_numa_nodes)
 .map(|numa_id| {
 /// Pin allocation thread to target NUMA node
 let _guard = numa_bind_guard(numa_id);

 let num_stages = stages_per_numa[numa_id];
 let capacity = num_stages * cuts_per_stage;

 /// Allocate and first-touch initialize
 let cuts = allocate_numa_aware::<CutData>(capacity);

 NumaCutPartition {
 numa_id,
 cuts,
 stage_offsets: Vec::new(), // Computed during load
 }
 })
 .collect();

 Self {
 numa_partitions,
 stage_to_numa,
 shm_win: MPI_WIN_NULL, // Initialized separately
 }
 }

 /// Get cuts for a stage (returns reference to NUMA-local data)
 #[inline]
 pub fn get_stage_cuts(&self, stage: usize) -> &[CutData] {
 let numa = self.stage_to_numa[stage];
 &self.numa_partitions[numa].get_stage(stage)
 }
}

```

#### Deployment Requirements:

- Use libnuma bindings for NUMA node binding
- Configure SLURM with --mem-bind=local when available

- Monitor `numa_local_ratio` metric (target: >90%)

### 6.9.3 False Sharing Prevention in Cut Evaluation

**Issue:** The `domination_count` field in `BendersCut` (Section 7.2.1) causes false sharing when multiple threads evaluate cuts in parallel.

**Problem:** Adjacent cuts share cache lines. When thread T1 writes `cuts[i].domination_count` and thread T2 writes `cuts[i+1].domination_count`, they invalidate each other's cache lines even though they're accessing different cuts.

**Required Fix - Thread-Local Accumulation:**

```
/// Thread-local workspace for cut evaluation
pub struct CutEvaluationWorkspace {
 /// Local domination counts, accumulated per thread
 /// Indexed by: local_counts[thread_id][cut_index]
 local_counts: Vec<Vec<u32>>,

 /// Local cut values buffer (reused across scenarios)
 cut_values: Vec<f64>,

 /// Cache-line padded to prevent false sharing between workspaces
 _padding: [u8; 64],
}

impl CutEvaluationWorkspace {
 pub fn new(num_threads: usize, max_cuts: usize) -> Self {
 Self {
 local_counts: vec![vec![0u32; max_cuts]; num_threads],
 cut_values: vec![0.0; max_cuts],
 _padding: [0u8; 64],
 }
 }

 /// Merge thread-local counts into global domination counts
 /// Called once after batch evaluation completes
 pub fn merge_counts(&self, global_counts: &mut [AtomicU32]) {
 for thread_counts in &self.local_counts {
 for (i, &count) in thread_counts.iter().enumerate() {
 if count > 0 {
 global_counts[i].fetch_add(count, Ordering::Relaxed);
 }
 }
 }
 }
}
```

**Implementation Pattern:**

```
// Parallel cut evaluation WITHOUT false sharing
#[omp_parallel]
fn evaluate_cuts_batch(
 cuts: &[CutData],
 states: &[StateVector],
 workspace: &CutEvaluationWorkspace,
) {
 let tid = omp_get_thread_num();

 /// Each thread accumulates into its local buffer
```

```

for state in states.iter() {
 let dominating_cut = find_dominating_cut(cuts, state, &mut workspace.cut_values);
 workspace.local_counts[tid][dominating_cut] += 1;
}

#[omp_barrier]

// Single thread merges results
#[omp_single]
workspace.merge_counts(&cuts.domination_counts);
}

```

#### 6.9.4 Load Balancing Strategy

**Scope Clarification:** Work-stealing operates at TWO levels with different mechanisms:

1. **Inter-Rank (MPI):** Dynamic dispatch from rank 0 (see Section 6.10)
2. **Intra-Rank (OpenMP):** `schedule(dynamic)` or `schedule(guided)` within parallel regions

#### Why NOT Cross-Rank Work-Stealing:

Cross-rank work-stealing (where rank A steals work from rank B via MPI) was considered but rejected:

Factor	Intra-Rank (OpenMP)	Cross-Rank (MPI)
Latency	~100ns (shared memory)	~10-100 s (network)
State transfer	None (shared memory)	Must serialize LP state
Implementation	Built into OpenMP runtime	Complex custom protocol
Benefit for SDDP	Handles thread variance	~1-5% speedup at most

For SDDP with 1-10 second iterations, the overhead of MPI task migration would exceed any benefit.

#### Intra-Rank Work-Stealing via OpenMP:

Within each rank, OpenMP handles load imbalance automatically:

```

// Forward pass processing within a rank
// Each thread processes complete forward passes from the batch
#pragma omp parallel for schedule(dynamic, 1)
for (int i = 0; i < batch_size; i++) {
 int pass_idx = batch[i];
 forward_pass_sequential(pass_idx, scenarios, cuts, &solvers[omp_get_thread_num()]);
}

```

The `schedule(dynamic, 1)` clause enables OpenMP's built-in work-stealing:

- Threads that finish early steal work from the shared queue
- No explicit stealing code needed
- Optimal for variable LP solve times

#### Inter-Rank Load Balancing via Dynamic Dispatch:

See Section 6.10 for the dynamic work distribution protocol where rank 0 dispatches batches to workers as they become available. This achieves optimal load balancing across MPI ranks without explicit work-stealing.

#### Expected Improvement:

- Intra-rank (OpenMP dynamic): 15-25% better efficiency vs static scheduling
- Inter-rank (dynamic dispatch): Near-optimal utilization across all ranks



### 6.9.5 Asynchronous Checkpointing

**Issue:** Synchronous checkpoint I/O can exceed iteration time at large scale, blocking all ranks.

#### Required Implementation:

```
/// Async checkpointing with double-buffering
pub struct AsyncCheckpointer {
 /// Current buffer being filled
 write_buffer: Arc<Mutex<CheckpointBuffer>>,

 /// Background writer thread
 writer_thread: Option<JoinHandle<()>>,

 /// Channel for checkpoint requests
 checkpoint_tx: mpsc::Sender<CheckpointRequest>,
}

impl AsyncCheckpointer {
 /// Initiate async checkpoint (non-blocking)
 pub fn checkpoint_async(&self, iteration: u32, cuts: &StageCuts) {
 // Serialize to buffer (fast, in-memory)
 let buffer = {
 let mut buf = self.write_buffer.lock();
 buf.clear();
 cuts.serialize_to(&mut buf);
 buf.clone()
 };

 // Send to background writer (non-blocking)
 let _ = self.checkpoint_tx.send(CheckpointRequest {
 iteration,
 data: buffer,
 });
 }

 /// Ensure previous checkpoint is complete (call before modifying data)
 pub fn wait_for_pending(&self) {
 // Block until background writer is idle
 // Implementation: use a completion channel or atomic flag
 }
}
```

#### I/O Strategy:

- Use double-buffering: fill buffer N while writing buffer N-1
- Compress with ZSTD level 1-3 (fast compression, good ratio)
- Consider MPI-IO for parallel writes to shared filesystem

### 6.9.7 Summary: Critical vs High-Priority Issues

Issue	Severity	Section	Status
NUMA FCF allocation	<b>CRITICAL</b>	6.9.2	Must fix for EPYC systems
False sharing in cut evaluation	HIGH	6.9.3	10-20% perf impact
Work-stealing load balance	HIGH	6.9.4	15-25% perf impact

Issue	Severity	Section	Status
Async checkpointing	MEDIUM	6.9.5	Required at scale

## 6.10 Dynamic Work Distribution

**Context:** Unlike NEWAVE's single-threaded ranks with full memory replication, POWE.RS uses hybrid MPI+OpenMP with shared memory within nodes. This enables dynamic work distribution from rank 0 while benefiting from shared cut/scenario storage.

### 6.10.1 Architecture Overview Dynamic Work Distribution:



#### Rank 0 (Dispatcher + Worker):

- Work Queue: [0, 1, 2, ..., 999]
- Dispatcher Thread: Receives READY, sends batch
- Compute Threads: Process local batches via OpenMP

#### Workers (Ranks 1..N-1):

1. Send READY to rank 0
2. Receive batch assignment
3. Process batch with OpenMP parallel for

**Batch Processing:** `#pragma omp parallel for schedule(dynamic, 1)`

**Batch Size:** = `num_threads` (one forward pass per thread per batch)

### 6.10.2 Rank 0 Bottleneck Mitigation

**Issue:** Rank 0 acts as both dispatcher and worker. If dispatching blocks computation or vice versa, efficiency suffers.

#### Solution: Dedicated Dispatcher Thread with Non-Blocking MPI

```

/// Rank 0 execution model with dedicated dispatcher
pub struct Rank0Executor {
 /// Work queue (protected by mutex for thread-safe access)
 work_queue: Mutex<VecDeque<u32>>,

 /// Local batch queue for compute threads
 local_batches: ArrayQueue<Vec<u32>>,

 /// Configuration
 batch_size: usize,

 /// MPI communicator
 comm: MpiComm,
}

impl Rank0Executor {
 /// Start dispatcher thread and compute threads
 pub fn execute_forward_pass(&self, total_passes: u32) -> ForwardPassResults {
 // Fill work queue
 {
 let mut queue = self.work_queue.lock();
 queue.extend(0..total_passes);
 }
 }
}

```

```

}

// Spawn dispatcher thread (uses 1 core)
let dispatcher_handle = std::thread::spawn(|| self.dispatcher_loop());

// Main thread coordinates OpenMP compute (uses remaining cores)
let results = self.compute_loop();

// Wait for dispatcher to finish
dispatcher_handle.join().unwrap();

results
}

/// Dispatcher thread: handle MPI requests from workers
fn dispatcher_loop(&self) {
 let world_size = self.comm.size();
 let mut active_workers = world_size - 1; // Excluding rank 0

 while active_workers > 0 {
 // Non-blocking probe for worker requests
 if let Some(status) = self.comm.iprobe_any(TAG_READY) {
 let worker = status.source();
 let mut ready_signal: u32 = 0;
 self.comm.recv(&mut ready_signal, worker, TAG_READY);

 // Try to get batch from queue
 let batch = self.try_pop_batch();

 if let Some(batch) = batch {
 self.comm.send(&(batch.len() as u32), worker, TAG_BATCH_SIZE);
 self.comm.send_slice(&batch, worker, TAG_BATCH_DATA);
 } else {
 // No more work, send DONE
 self.comm.send(&0u32, worker, TAG_BATCH_SIZE);
 active_workers -= 1;
 }
 } else {
 // No pending requests, push batch to local queue for compute threads
 if let Some(batch) = self.try_pop_batch() {
 let _ = self.local_batches.push(batch);
 }

 // Brief yield to avoid busy-wait
 std::thread::yield_now();
 }
 }
}

// Signal local compute threads that no more batches will come
// (they can still process batches already in local_batches)
}

/// Compute loop: OpenMP parallel processing of batches
fn compute_loop(&self) -> ForwardPassResults {
 let mut results = ForwardPassResults::new();

 loop {

```

```

 // Try to get batch from local queue
 let batch = match self.local_batches.pop() {
 Some(b) => b,
 None => {
 // Check if more batches might come
 if self.work_queue.lock().is_empty() {
 break; // All done
 }
 std::thread::yield_now();
 continue;
 }
 };

 // Process batch with OpenMP
 let batch_results = self.process_batch_openmp(&batch);
 results.merge(batch_results);
}

results
}

fn try_pop_batch(&self) -> Option<Vec<u32>> {
 let mut queue = self.work_queue.lock();
 if queue.is_empty() {
 return None;
 }

 let count = self.batch_size.min(queue.len());
 Some(queue.drain(..count).collect())
}
}

```

#### Thread Allocation Strategy:

Node Cores	Rank 0 Threads	Worker Rank Threads	Efficiency Loss
64	1 dispatcher + 63 compute	64 compute	1.5%
128	1 dispatcher + 127 compute	128 compute	0.8%
16	1 dispatcher + 15 compute	16 compute	6%

For smaller nodes, consider:

- Option A: Accept 6% loss (simple)
- Option B: Dispatcher interleaved with compute (complex, better efficiency)

**Recommendation:** Use dedicated dispatcher thread (Option A) for simplicity. The efficiency loss is negligible at scale.

#### 6.10.3 Worker Implementation

```

/// Worker rank execution model
pub struct WorkerExecutor {
 /// Batch size (should match rank 0's batch_size)
 batch_size: usize,

 /// MPI communicator
 comm: MpiComm,
}

```

```

 /// OpenMP thread-local solver workspaces (see Section 5.4.9)
 /// Each thread owns one workspace with persistent solver instance.
 workspaces: WorkspaceManager,
}

impl WorkerExecutor {
 pub fn execute_forward_pass(&mut self) -> ForwardPassResults {
 let mut results = ForwardPassResults::new();

 loop {
 /// Request work from rank 0
 self.comm.send(&1u32, 0, TAG_READY);

 /// Receive batch size
 let mut batch_size: u32 = 0;
 self.comm.recv(&mut batch_size, 0, TAG_BATCH_SIZE);

 if batch_size == 0 {
 break; /// DONE signal
 }

 /// Receive batch data
 let mut batch = vec![0u32; batch_size as usize];
 self.comm.recv_slice(&mut batch, 0, TAG_BATCH_DATA);

 /// Process batch with OpenMP
 let batch_results = self.process_batch_openmp(&batch);
 results.merge(batch_results);
 }

 results
 }

 fn process_batch_openmp(&mut self, batch: &[u32]) -> Vec<ForwardPassResult> {
 /// #pragma omp parallel for schedule(dynamic, 1)
 /// Each thread processes one complete forward pass using its workspace
 batch.par_iter().enumerate().map(|(local_idx, &pass_idx)| {
 let thread_id = omp_get_thread_num();
 let workspace = self.workspaces.get_mut(thread_id);

 /// Sequential forward pass (stages are sequential)
 forward_pass_sequential(pass_idx, workspace)
 }).collect()
 }
}

/// Execute single forward pass (all stages, sequential)
///
/// Uses ThreadSolverWorkspace (Section 5.4.9) for thread-local solving.
fn forward_pass_sequential(
 pass_idx: u32,
 workspace: &mut ThreadSolverWorkspace,
) -> ForwardPassResult {
 let mut state = initial_state();
 let mut total_cost = 0.0;

 for stage in 0..num_stages {

```

```

// Get pregenerated scenario for this pass/stage
let scenario = scenarios.get(pass_idx, stage);

// Prepare RHS updates for this scenario
let rhs_updates = compute_rhs_updates(stage, &state, scenario);

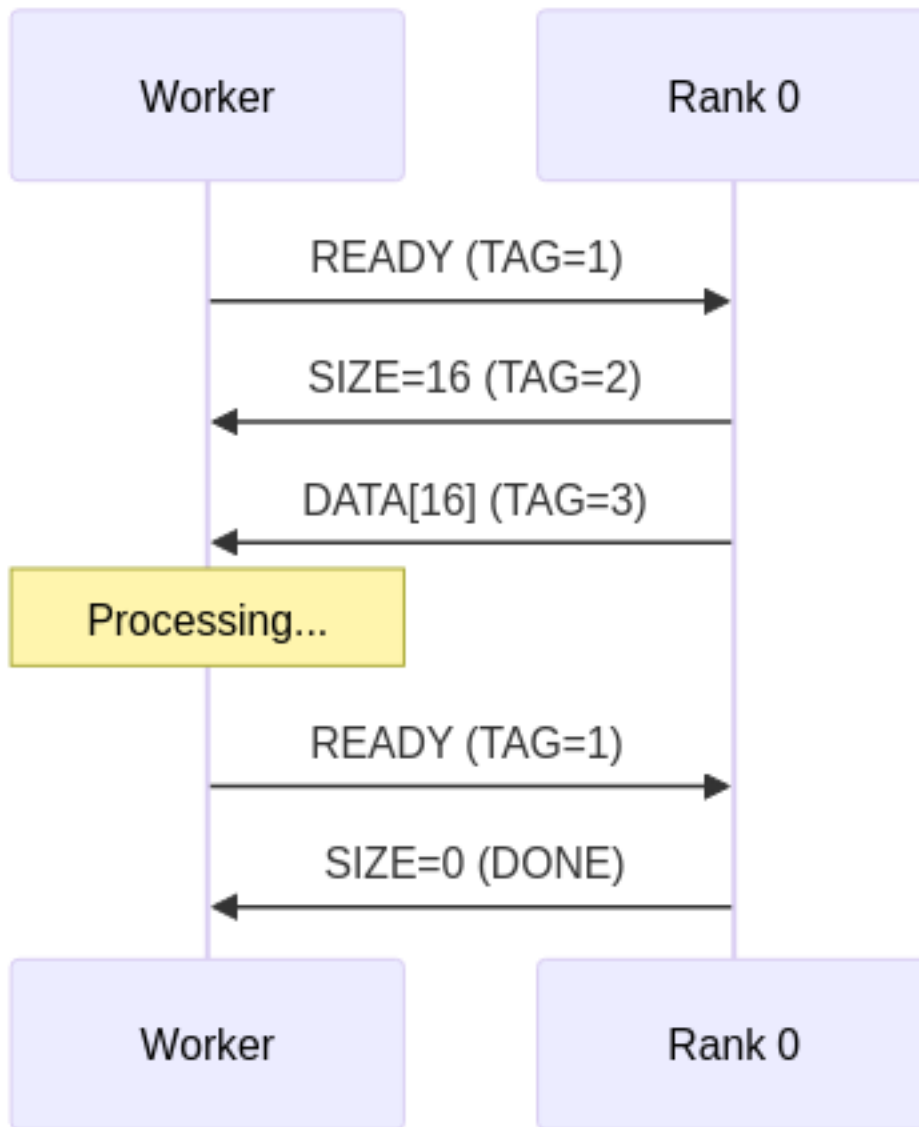
// Solve stage LP using workspace (with automatic warm-starting)
let solution = workspace.solve_stage(stage, &stage_problems[stage], &rhs_updates)
 .expect("Stage solve failed");

// Update state and cost
state = solution.next_state;
total_cost += solution.stage_cost;
}

ForwardPassResult {
 pass_idx,
 total_cost,
 states: state_history,
}
}

```

#### 6.10.4 MPI Message Protocol Forward Pass Message Protocol:



#### Message Tags:

Tag	Name	Direction	Data
1	TAG_READY	Worker → Rank 0	uint32 (ignored, triggers receive)
2	TAG_BATCH_SIZE	Rank 0 → Worker	uint32 (batch_size, or 0 for DONE)
3	TAG_BATCH_DATA	Rank 0 → Worker	uint32[batch_size] (forward pass indices)

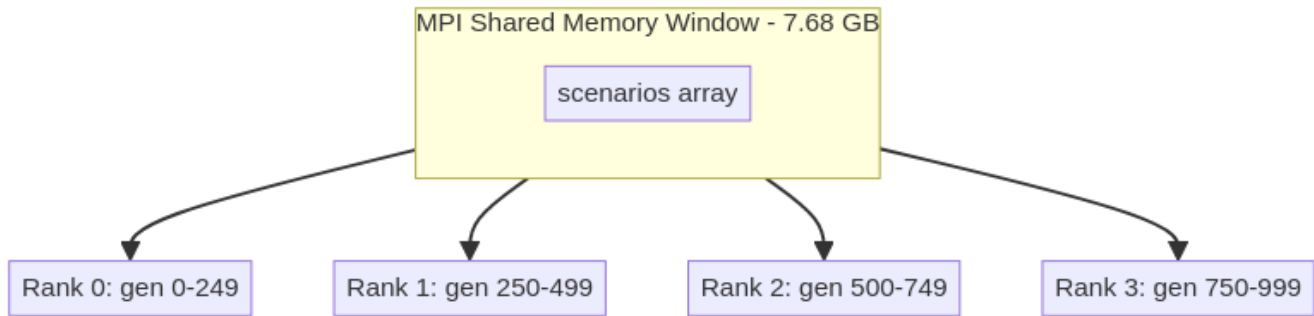
#### 6.11 Shared Memory Scenario Storage

**Context:** With dynamic work distribution, any forward pass can be assigned to any rank. Scenarios must be accessible regardless of assignment. Two approaches:

1. **Replicated:** Each rank stores all scenarios (7.68 GB × ranks = memory explosion)
2. **Shared:** Ranks on same node share scenario storage via MPI windows (7.68 GB per node)

**Decision:** Shared memory within node using `MPI_Win_allocate_shared`.

### 6.11.1 Shared Memory Architecture Shared Scenario Storage (Single Node):



**Array Layout:** `scenarios[pass_idx][stage][branch][variable]`

- $1000 \text{ passes} \times 120 \text{ stages} \times 20 \text{ branches} \times 320 \text{ vars} \times 8 \text{ bytes} = 7.68 \text{ GB}$

**Access Pattern:**

Operation	Who	When
Write	Each rank writes assigned portion	During scenario generation
Read	Any rank reads any pass	During forward pass (dynamic assignment)
Sync	<code>MPI_Win_fence</code>	After generation, before iteration start

### 6.11.2 Deterministic Scenario Seeding

**CRITICAL for Reproducibility:** Scenarios must be identical regardless of:

Must Be Independent Of
Number of MPI ranks
Number of OpenMP threads
Which rank generates which pass
Order of generation

**Solution:** Each scenario is seeded by its **identity**, not its computational assignment.

```

/// Deterministic seed computation
/// INVARIANT: Same (master_seed, pass, stage, branch) → same seed → same scenario
#[inline]
pub fn scenario_seed(master_seed: u64, pass_idx: u32, stage: u32, branch: u32) -> u64 {
 // SplitMix64-style mixing for high-quality seed derivation
 let mut x = master_seed;

 // Mix in pass index
 x = x.wrapping_add(0x9e3779b97f4a7c15);
 x ^= pass_idx as u64;
 x = x.wrapping_mul(0xbf58476d1ce4e5b9);

 // Mix in stage

```



```

x = x.wrapping_add(0x9e3779b97f4a7c15);
x ^= stage as u64;
x = x.wrapping_mul(0x94d049bb133111eb);

// Mix in branch
x = x.wrapping_add(0x9e3779b97f4a7c15);
x ^= branch as u64;
x = x.wrapping_mul(0xbf58476d1ce4e5b9);

// Final mixing
x ^= x >> 30;
x = x.wrapping_mul(0xbf58476d1ce4e5b9);
x ^= x >> 27;
x
}

/// Generate correlated noise for a single scenario point
pub fn generate_scenario(
 seed: u64,
 cholesky_factor: &[f64], // Lower triangular L where $\Sigma = L \times L^T$
 num_vars: usize,
) -> Vec<f64> {
 let mut rng = Xoshiro256PlusPlus::seed_from_u64(seed);

 // Generate standard normal vector $z \sim N(0, I)$
 let z: Vec<f64> = (0..num_vars)
 .map(|_| StandardNormal.sample(&mut rng))
 .collect();

 // Apply correlation: $= L \times z \sim N(0, \Sigma)$
 let mut epsilon = vec![0.0; num_vars];
 let mut idx = 0;
 for i in 0..num_vars {
 for j in 0..=i {
 epsilon[i] += cholesky_factor[idx] * z[j];
 idx += 1;
 }
 }

 epsilon
}

```

### 6.11.3 Distributed Generation Protocol

```

/// Distributed scenario generation using MPI shared memory
pub fn generate_scenarios_distributed(
 config: &ScenarioConfig,
 correlation: &CorrelationFactors,
 node_comm: &MpiComm, // Communicator for ranks on same node
) -> SharedScenarioStorage {
 let rank = node_comm.rank() as usize;
 let size = node_comm.size() as usize;
 let num_vars = config.num_hydros + config.num_loads;
 let num_passes = config.num_forward_passes as usize;

 // Calculate total storage size
 let total_branches: usize = config.branches_per_stage.iter()

```

```

 .map(|&b| b as usize).sum();
let floats_per_pass = total_branches * num_vars;
let total_floats = num_passes * floats_per_pass;
let total_bytes = total_floats * std::mem::size_of::<f64>();

// Allocate shared memory (only rank 0 provides size)
let (win, base_ptr) = unsafe {
 let mut ptr: *mut f64 = std::ptr::null_mut();
 let mut win = MPI_WIN_NULL;

 MPI_Win_allocate_shared(
 if rank == 0 { total_bytes as MPI_Aint } else { 0 },
 std::mem::size_of::<f64>() as i32,
 MPI_INFO_NULL,
 node_comm.as_raw(),
 &mut ptr as *mut _ as *mut c_void,
 &mut win,
);

 // All ranks query base pointer from rank 0
 if rank != 0 {
 let mut size: MPI_Aint = 0;
 let mut disp: i32 = 0;
 MPI_Win_shared_query(win, 0, &mut size, &mut disp,
 &mut ptr as *mut _ as *mut c_void);
 }

 (win, ptr)
};

// Divide generation work across ranks
let passes_per_rank = num_passes / size;
let remainder = num_passes % size;
let my_start = rank * passes_per_rank + rank.min(remainder);
let my_count = passes_per_rank + if rank < remainder { 1 } else { 0 };

// Parallel generation within rank using OpenMP
// #pragma omp parallel for schedule(static)
for local_idx in 0..my_count {
 let pass_idx = (my_start + local_idx) as u32;

 for (stage, &num_branches) in config.branches_per_stage.iter().enumerate() {
 let cholesky = &correlation.factors[stage];

 for branch in 0..num_branches {
 // Deterministic seed
 let seed = scenario_seed(config.master_seed, pass_idx, stage as u32, branch);

 // Generate correlated noise
 let epsilon = generate_scenario(seed, cholesky, num_vars);

 // Write to shared memory
 let offset = compute_offset(pass_idx as usize, stage, branch as usize,
 &config.branches_per_stage, num_vars);

 unsafe {
 std::ptr::copy_nonoverlapping(
 epsilon.as_ptr(),

```

```

 base_ptr.add(offset),
 num_vars,
);
 }
}

// Synchronization fence - ensures all writes visible to all ranks
unsafe { MPI_Win_fence(0, win); }

SharedScenarioStorage {
 win,
 base_ptr,
 num_passes,
 num_stages: config.num_stages as usize,
 branches_per_stage: config.branches_per_stage.clone(),
 num_vars,
}
}

```

#### 6.11.4 Memory Layout and Access Pattern

```

/// Shared scenario storage with efficient indexing
pub struct SharedScenarioStorage {
 win: MPI_Win,
 base_ptr: *mut f64,

 num_passes: usize,
 num_stages: usize,
 branches_per_stage: Vec<u32>,
 num_vars: usize,

 /// Precomputed offsets for O(1) access: offset[stage] = start of stage's data
 stage_offsets: Vec<usize>,
}

impl SharedScenarioStorage {
 /// Get scenario noise for (pass, stage, branch)
 /// Returns slice of correlated noise values for all random variables
 #[inline]
 pub fn get(&self, pass: usize, stage: usize, branch: usize) -> &[f64] {
 debug_assert!(pass < self.num_passes);
 debug_assert!(stage < self.num_stages);
 debug_assert!(branch < self.branches_per_stage[stage] as usize);

 let offset = self.compute_offset(pass, stage, branch);
 unsafe {
 std::slice::from_raw_parts(self.base_ptr.add(offset), self.num_vars)
 }
 }

 #[inline]
 fn compute_offset(&self, pass: usize, stage: usize, branch: usize) -> usize {
 // Layout: [pass][stage][branch][variable]
 let floats_per_pass = self.floats_per_pass();
 let pass_offset = pass * floats_per_pass;
 }
}

```

```

 let stage_offset = self.stage_offsets[stage];
 let branch_offset = branch * self.num_vars;

 pass_offset + stage_offset + branch_offset
}

fn floats_per_pass(&self) -> usize {
 self.branches_per_stage.iter()
 .map(|&b| b as usize * self.num_vars)
 .sum()
}
}

```

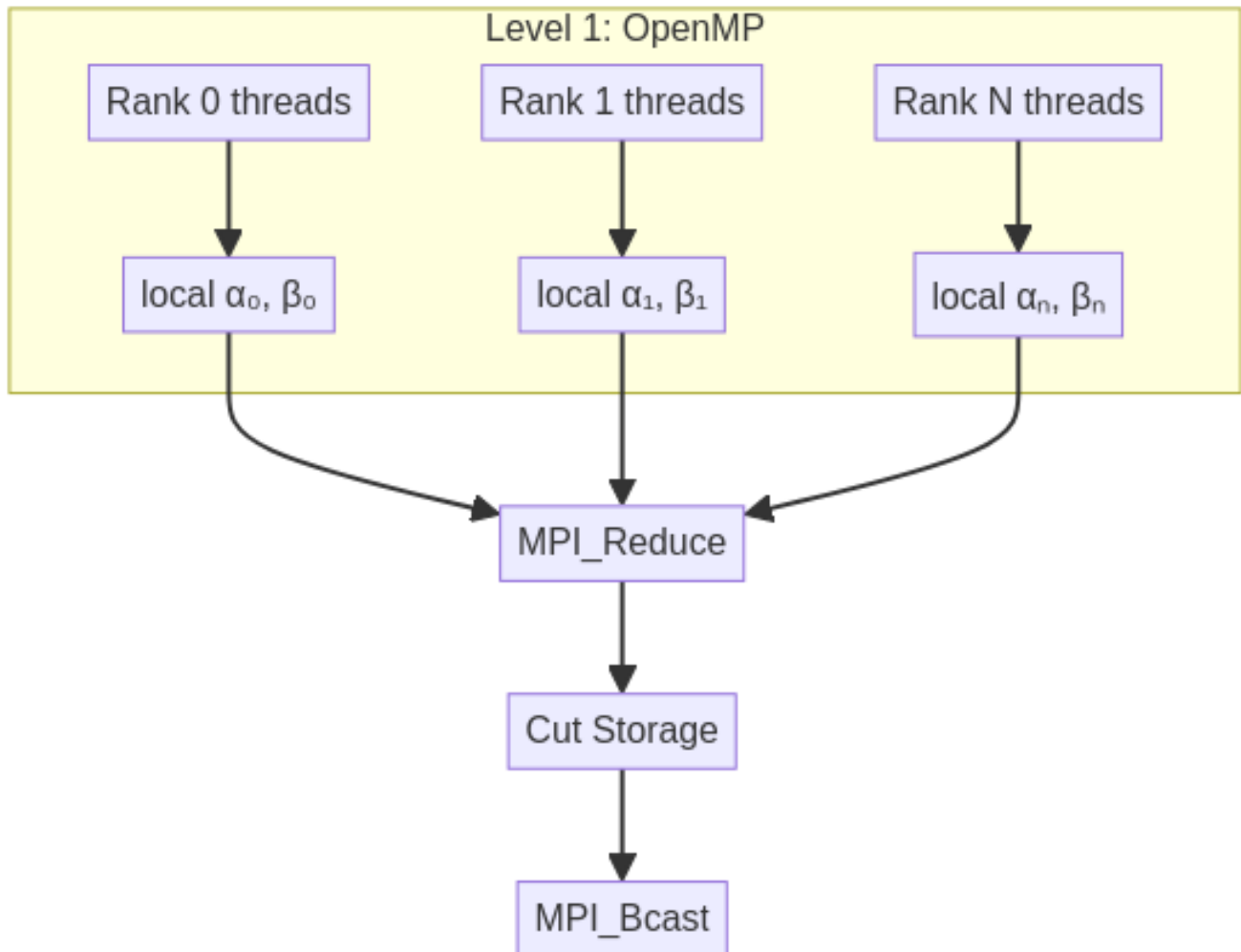
## 6.12 Two-Level Cut Aggregation

**Context:** The SDDP backward pass requires aggregating cuts from all forward passes. With hybrid MPI+OpenMP, this involves two levels of reduction:

1. **Level 1 (Intra-Rank):** OpenMP reduction across threads within each rank
2. **Level 2 (Inter-Rank):** MPI reduction across ranks

For single-cut formulation, all contributions are summed into one cut per stage.

### 6.12.1 Two-Level Reduction Architecture Two-Level Cut Aggregation (Single-Cut) - Backward Pass for Stage $t$ :



Level	Operation	Result
1 - OpenMP	Thread reduction within rank	local_ , local_ per rank
2 - MPI	MPI_Reduce(..., SUM, root=0)	global_ , global_ at rank 0
Storage	cuts[slot] = { , , active}	Rank 0 writes to shared memory
Broadcast	MPI_Bcast from root=0	All ranks have identical cut data

Slot Calculation:  $\text{slot} = \text{iteration} \times (\text{num\_stages} - 1) + (\text{stage} - 1)$

### 6.12.2 Implementation

```

/// Two-level cut aggregation for single-cut formulation
pub fn aggregate_single_cut(
 stage: usize,
 iteration: u32,
 local_results: &[BackwardSubproblemResult],
 state_dim: usize,
 comm: &MpiComm,
) -> SingleCut {
 let rank = comm.rank();

 /// =====
 /// LEVEL 1: OpenMP reduction within rank
 /// =====

 let (local_alpha, local_beta) = {
 let mut alpha_sum = 0.0f64;
 let mut beta_sum = vec![0.0f64; state_dim];

 /// OpenMP parallel for with reduction
 /// #pragma omp parallel for reduction(+:alpha_sum) reduction(+:beta_sum[:state_dim])
 for result in local_results {
 let weight = result.probability;
 alpha_sum += result.alpha * weight;
 for (i, &coef) in result.beta.iter().enumerate() {
 beta_sum[i] += coef * weight;
 }
 }

 (alpha_sum, beta_sum)
 };

 /// =====
 /// LEVEL 2: MPI reduction across ranks
 /// =====

 /// Pack into contiguous buffer for efficient MPI
 let mut local_buf = Vec::with_capacity(1 + state_dim);
 local_buf.push(local_alpha);
 local_buf.extend_from_slice(&local_beta);

 let mut global_buf = vec![0.0f64; 1 + state_dim];

 /// MPI_Reduce to rank 0 (deterministic order for reproducibility)
 comm.reduce_sum(&local_buf, &mut global_buf, 0);

```

```

// =====
// BROADCAST: All ranks receive identical cut
// =====

comm.broadcast_slice(&mut global_buf, 0);

SingleCut {
 alpha: global_buf[0],
 beta: global_buf[1..].to_vec(),
 iteration,
 stage: stage as u32,
}
}

```

### 6.12.3 Replicated Cut Selection

**Key Insight:** After MPI\_Bcast, all ranks have identical cut data. If all ranks run the **same deterministic algorithm** on this data, they will produce **identical selection decisions** without additional communication.

```

/// Replicated cut selection - runs identically on all ranks
/// CRITICAL: Must be deterministic (same inputs → same outputs)
pub fn replicated_cut_selection(
 stage: usize,
 cuts: &mut SharedCutStorage,
 config: &CutSelectionConfig,
) {
 // Get all cuts for this stage (same on all ranks after broadcast)
 let stage_cuts = cuts.get_stage_cuts(stage);
 let num_cuts = stage_cuts.len();

 // =====
 // DETERMINISTIC DOMINATION CHECK
 // =====

 // Iterate in slot order (deterministic, same on all ranks)
 let mut active_mask = vec![true; num_cuts];

 for i in 0..num_cuts {
 if !stage_cuts[i].is_active || !active_mask[i] {
 continue;
 }

 for j in 0..num_cuts {
 if i == j || !stage_cuts[j].is_active || !active_mask[j] {
 continue;
 }

 // Check domination using deterministic criteria
 if is_dominated(&stage_cuts[i], &stage_cuts[j], config) {
 active_mask[i] = false;
 break;
 }
 }
 }
}

// =====
// APPLY SELECTION

```

```

// =====

// Only rank 0 writes to shared storage (avoids races)
// But all ranks compute the same active_mask!
if cuts.comm.rank() == 0 {
 for (i, &active) in active_mask.iter().enumerate() {
 if !active && stage_cuts[i].is_active {
 cuts.deactivate_cut(stage, i);
 }
 }
}

// Fence ensures all ranks see the update
cuts.fence();
}

/// Deterministic domination check
/// Returns true if cut_a is dominated by cut_b
fn is_dominated(cut_a: &CutData, cut_b: &CutData, config: &CutSelectionConfig) -> bool {
 // Cut b dominates cut a if:
 // For all feasible states x: $_b + _b'x \leq _a + _a'x$

 // Simplified check: compare at generation points
 // (Full domination check is NP-hard, use heuristics)

 let state = &cut_a.state_at_generation;
 let value_a = cut_a.alpha + dot(&cut_a.beta, state);
 let value_b = cut_b.alpha + dot(&cut_b.beta, state);

 // Cut b dominates at cut a's generation point
 if value_b >= value_a - config.tolerance {
 // Also check at cut b's generation point for robustness
 let state_b = &cut_b.state_at_generation;
 let value_a_at_b = cut_a.alpha + dot(&cut_a.beta, state_b);
 let value_b_at_b = cut_b.alpha + dot(&cut_b.beta, state_b);

 return value_b_at_b >= value_a_at_b - config.tolerance;
 }

 false
}

```

### 6.13 Reproducibility Guarantees

**Requirement:** Given the same inputs and random seed, POWE.RS must produce **bit-for-bit identical** results regardless of:

Must Be Independent Of
Number of MPI ranks
Number of OpenMP threads per rank
Execution timing/ordering
Hardware platform (with IEEE 754 compliance)

#### 6.13.1 Reproducibility Mechanisms

Component	Mechanism	Guarantee
<b>Scenario Generation</b>	Deterministic seeding by (pass, stage, branch)	Same scenarios regardless of which rank/thread generates
<b>Forward Pass Results</b>	Results indexed by global pass ID	Assignment to ranks doesn't affect indexing
<b>Cut Aggregation</b>	MPI_Reduce with fixed order + MPI_Bcast	All ranks have identical aggregated cuts
<b>Cut Selection</b>	Replicated algorithm on identical data	All ranks make identical decisions
<b>LP Constraint Order</b>	Cuts added by slot index (iteration order)	Same LP structure across runs
<b>Random Number Generator</b>	Xoshiro256++ with explicit seeding	Portable, reproducible sequence

### 6.13.2 Potential Reproducibility Issues Issue 1: Floating-Point Associativity

Problem:  $(a + b) + c \neq a + (b + c)$  in floating-point  
MPI\_Reduce order may vary between runs

Solution: Use MPI\_Reduce to rank 0 (fixed tree order) instead of MPI\_Allreduce  
Then MPI\_Bcast from rank 0

Code:

```
// WRONG: Order may vary
MPI_Allreduce(local, global, MPI_SUM, comm);

// CORRECT: Fixed order
MPI_Reduce(local, global, MPI_SUM, 0, comm);
MPI_Bcast(global, 0, comm);
```

### Issue 2: OpenMP Reduction Order

Problem: `#pragma omp parallel for reduction(+:sum)`  
Summation order depends on thread scheduling

Solution: Use compensated summation (Kahan) for high precision  
Or accept small FP differences (document tolerance)

Recommended: Accept FP tolerance within rank, enforce exact order across ranks

### Issue 3: Dynamic Work Distribution Order

Problem: With dynamic dispatch, forward pass completion order varies  
Does this affect results?

Analysis: NO - because:

1. Each pass's scenarios are determined by global pass ID, not completion order
2. Cut aggregation sums all contributions (commutative)
3. Results are indexed by pass ID, not completion order

Key: Use `global_pass_idx` for all indexing, never `completion_order`

### 6.13.3 Verification Protocol

```
/// Reproducibility verification (run in debug/test builds)
pub struct ReproducibilityChecker {
 /// Hash of scenario data (should match across runs)
```



```

scenario_hash: u64,

/// Per-iteration cut hashes
cut_hashes: Vec<u64>,

/// Final bounds
lower_bound: f64,
upper_bound: f64,
}

impl ReproducibilityChecker {
 pub fn verify_scenarios(&mut self, scenarios: &SharedScenarioStorage) {
 let mut hasher = XxHash64::default();

 /// Hash all scenarios in deterministic order
 for pass in 0..scenarios.num_passes {
 for stage in 0..scenarios.num_stages {
 for branch in 0..scenarios.branches_per_stage[stage] as usize {
 let noise = scenarios.get(pass, stage, branch);
 for &value in noise {
 hasher.write(&value.to_le_bytes());
 }
 }
 }
 }

 self.scenario_hash = hasher.finish();
 }

 pub fn verify_iteration(&mut self, iteration: u32, cuts: &SharedCutStorage) {
 let mut hasher = XxHash64::default();

 /// Hash cuts added this iteration
 for stage in 0..cuts.num_stages {
 let slot = iteration * (cuts.num_stages as u32 - 1) + stage as u32;
 if let Some(cut) = cuts.get_cut(stage, slot as usize) {
 hasher.write(&cut.alpha.to_le_bytes());
 for &coef in &cut.beta {
 hasher.write(&coef.to_le_bytes());
 }
 }
 }

 self.cut_hashes.push(hasher.finish());
 }

 pub fn compare(&self, other: &ReproducibilityChecker) -> bool {
 self.scenario_hash == other.scenario_hash
 && self.cut_hashes == other.cut_hashes
 && (self.lower_bound - other.lower_bound).abs() < 1e-10
 }
}

```

#### 6.13.4 Configuration Requirements

```

{
 "reproducibility": {

```

```

 "enabled": true,
 "master_seed": 12345,
 "mpi_reduce_deterministic": true,
 "openmp_schedule": "static",
 "fp_tolerance": 1e-12
 }
}

```

### Environment Variables for Reproducibility:

```

Ensure deterministic OpenMP scheduling
export OMP_SCHEDULE="static"
export OMP_PROC_BIND="close"

Disable Intel MKL threading (can cause non-determinism)
export MKL_NUM_THREADS=1

Use deterministic BLAS operations
export OPENBLAS_NUM_THREADS=1

```

## 7. File Format Decisions

### 7.1 Summary Table

Data Category	Read/Write	Format	Rationale
Algorithm Config	Read	JSON	Small, editable
System Registry	Read	JSON	Structured objects
Stage/Block Def	Read	JSON	Graph structure
Uncertainty Models	Read	JSON	Complex nested
Distributions	Read	JSON	Parameters
Load Profiles	Read	Parquet	Large, indexed
Inflow History	Read	Parquet	Time series
Policy Cuts	Read/Write	FlatBuffers	Zero-copy, in-memory training
Policy States	Read/Write	FlatBuffers	Zero-copy, in-memory training
Policy Vertices	Read/Write	FlatBuffers	Zero-copy, in-memory training
Training Results	Write	Parquet	Analytics-ready
Simulation Detail	Write	Parquet	Large volume
Dictionaries	Write	CSV	Human-readable

### 7.2 FlatBuffers for Policy Data (Decision: 2026-01-19)

**Context:** Policy data (cuts, states, vertices) has a unique access pattern:

Characteristic	Description
<b>In-memory during training</b>	Entire cut pool lives in RAM, accessed every LP solve
<b>Checkpointed periodically</b>	Written only at checkpoint intervals (not every iteration)

Characteristic	Description
<b>High state dimension</b>	1120 coefficients per cut at production scale
<b>Large volume</b>	Up to 1.2M cuts totaling ~18.6 GB

**Problem with Parquet:** Using 1120 individual columns (`coefficient_0` through `coefficient_1119`) is inefficient for Parquet, which is optimized for columnar analytics, not dense fixed-size arrays.

**Decision:** Use FlatBuffers for policy files (cuts, states, vertices) due to: 1. **Zero-copy deserialization:** Load directly into memory without parsing overhead 2. **Cache-friendly layout:** Dense coefficient arrays optimal for SIMD operations 3. **Simple schema:** Flat structure maps directly to Rust structs 4. **Fast checkpoint writes:** Serialize directly from in-memory structures

**Reproducibility Requirement:** Checkpoints must enable **bit-for-bit identical** results on resume:

Requirement	Description
ALL cuts serialized	Active AND inactive to preserve LP structure
Slot indices preserved	For deterministic LP row mapping
Bound states preserved	<code>is_active</code> states maintained
Solver basis saved	Optional but recommended for exact reproducibility

### 7.2.1 FlatBuffers Schema Definitions

```
// File: schemas/policy.fbs
// POWE.RS Policy Data Schemas

namespace powers.policy;

// =====
// Benders Cut for Outer Approximation
// =====

// Standard Benders cut in form: + '(x - x̂)
// Equivalently: (- 'x̂) + 'x = intercept + 'x
table BendersCut {
 // Unique identifier within stage
 cut_id: uint64;

 // Slot index in preallocated LP (REQUIRED for reproducibility)
 // This preserves the exact LP row mapping on checkpoint/resume
 slot_index: uint32;

 // Generation metadata
 iteration: uint32;
 forward_pass_idx: uint32;
 scenario_idx: uint32;

 // Cut data in standard Benders notation:
 // intercept + Σi coefficients[i] × (state[i])
 // where intercept = - 'x̂ (pre-computed for efficiency)
 intercept: double; // - 'x̂ (right-hand side at origin)

 // Cut coefficients (dual multipliers / subgradient)
 // Length = state_dimension, stored as dense array
 coefficients: [double];
```

```

// State at which cut was generated $\hat{x}$ (for cut selection)
// Length = state_dimension
state_at_generation: [double];

// Cut management (REQUIRED for reproducibility)
// is_active determines LP bound: true $\rightarrow$ lb=intercept, false $\rightarrow$ lb=- ∞
is_active: bool = true;
domination_count: uint32 = 0;
}

// Collection of cuts for a single stage
// For reproducibility, ALL cuts (active AND inactive) are serialized
table StageCuts {
 stage_id: uint32;
 state_dimension: uint32;

 // Capacity information (for LP reconstruction)
 capacity: uint32; // Total preallocated slots
 warm_start_count: uint32; // Slots [0..warm_start_count) from loaded policy

 // All cuts indexed by slot (includes inactive cuts)
 // Length = populated_count (not capacity - empty slots at end not serialized)
 // Each cut's slot_index indicates its position in the LP
 cuts: [BendersCut];

 // Active cut indices for O(1) lookup during LP construction
 active_cut_indices: [uint32];

 // Number of currently populated slots
 populated_count: uint32;
}

// =====
// Visited State for Cut Selection
// =====

table VisitedState {
 // Unique identifier within stage
 state_id: uint64;

 // Generation metadata
 iteration: uint32;
 forward_pass_idx: uint32;
 scenario_idx: uint32;

 // State vector components
 // Length = state_dimension
 components: [double];

 // Cut selection data
 dominating_cut_id: uint64;
 dominating_objective: double;
}

// Collection of visited states for a single stage
table StageStates {

```

```

 stage_id: uint32;
 state_dimension: uint32;
 states: [VisitedState];
}

// =====
// Vertex for Inner Approximation (Upper Bound / SIDP)
// =====

table Vertex {
 // Unique identifier within stage
 vertex_id: uint64;

 // Generation metadata
 iteration: uint32;
 forward_pass_idx: uint32;
 scenario_idx: uint32;

 // State vector components at this vertex
 // Length = state_dimension
 components: [double];

 // Upper bound cost-to-go value at this state
 upper_bound_value: double;

 // Lipschitz constant used for interpolation from this vertex
 // (accumulated from deficit penalties backwards)
 lipschitz_constant: double;
}

// Collection of vertices for a single stage
table StageVertices {
 stage_id: uint32;
 state_dimension: uint32;
 vertices: [Vertex];

 // Stage-level Lipschitz constant (maximum over all vertices)
 stage_lipschitz: double;
}

// =====
// Policy Metadata
// =====

table PolicyMetadata {
 version: string;
 powers_version: string;
 created_at: string; // ISO 8601 timestamp

 // Algorithm state for resume
 completed_iterations: uint32;
 last_forward_pass: uint32;
 final_lower_bound: double;
 best_upper_bound: double;

 // Capacity parameters (for resume/warm-start)
 max_iterations: uint32; // Training parameter

```

```

forward_passes: uint32; // Training parameter
warm_start_cuts: uint32; // Cuts loaded from previous policy
// Derived: capacity = warm_start_cuts + max_iterations * forward_passes

// RNG state for exact reproducibility
rng_seed: uint64;
rng_state: [uint64]; // Full RNG state for resume

// Integrity
state_dimension: uint32;
num_stages: uint32;
config_hash: string;
system_hash: string;
}

```

```
root_type StageCuts;
```

### 7.2.2 File Structure

```

policy/
 metadata.json # Human-readable metadata (JSON for editability)
 state_dictionary.json # State variable mapping (JSON)
 cuts/
 stage_000.bin # FlatBuffers StageCuts
 stage_001.bin
 ...
 states/
 stage_000.bin # FlatBuffers StageStates
 ...
 vertices/ # Only if inner approximation enabled
 stage_000.bin # FlatBuffers StageVertices
 ...
 basis/ # Optional, solver-specific format
 ...

```

### 7.2.3 FlatBuffers Encoding Guidelines

Field Type	Encoding	Rationale
cut_id, state_id, vertex_id	uint64	Unique across all iterations
iteration, stage_id	uint32	Sufficient for practical limits
coefficients, components	[double] dense array	SIMD-friendly, no dictionary
is_active	bool	Bit-packed by FlatBuffers
Timestamps	string (ISO 8601)	Human-readable in metadata

**Compression:** FlatBuffers files are optionally compressed with Zstd for checkpoints:

- .bin - uncompressed (for fast load during resume)
- .bin.zst - Zstd-compressed (for archival/transfer)

### 7.2.4 Memory Layout Alignment

```

// Rust struct matching FlatBuffers layout for zero-copy access
#[repr(C, align(64))] // Cache-line aligned
pub struct BendersCutData {
 pub cut_id: u64,

```

```

 pub iteration: u32,
 pub forward_pass_idx: u32,
 pub scenario_idx: u32,
 pub is_active: bool,
 pub domination_count: u32,
 _padding: [u8; 3],
 pub intercept: f64,
 // coefficients and state_at_generation stored separately for SIMD
 pub coefficients_offset: usize,
 pub state_offset: usize,
}

// Coefficient storage: separate dense arrays for SIMD vectorization
pub struct CutCoefficients {
 // All cuts' coefficients packed contiguously
 // Layout: [cut0_coef0, cut0_coef1, ..., cut1_coef0, ...]
 pub data: Vec<f64>,
 pub state_dimension: usize,
 pub num_cuts: usize,
}

impl CutCoefficients {
 #[inline]
 pub fn get_cut_coefficients(&self, cut_idx: usize) -> &[f64] {
 let start = cut_idx * self.state_dimension;
 &self.data[start..start + self.state_dimension]
 }
}

```

### 7.2.5 Checkpoint Reproducibility Requirements

**Critical Requirement:** Checkpoint/resume must produce **bit-for-bit identical** results. This requires complete state serialization, not just active cuts.

#### Why Complete State Matters:

Run A: Fresh start, 50 iterations

- Generates cuts in slots [0..10000)
- Deactivates some via Level 1 selection
- LP has specific row structure
- Solver chooses specific pivots → specific duals → specific new cuts

Run B: Resume from checkpoint at iteration 25

- Must reconstruct IDENTICAL LP structure
- Same slots, same coefficients, same bounds
- Same solver warm-start basis
- Then: identical pivots → identical duals → identical cuts → identical results

#### Serialization Rules:

Data	Must Serialize	Rationale
All cuts (active + inactive)	Yes	LP row structure
Slot indices	Yes	Row mapping
is_active flags	Yes	Bound values
Coefficients	Yes	Cut geometry
RNG state	Yes	Scenario reproducibility
Solver basis	Recommended	Exact warm-start

## Checkpoint Contents:

```
/// Complete checkpoint for reproducible resume
pub struct Checkpoint {
 /// Metadata (iterations, bounds, capacity)
 pub metadata: PolicyMetadata,

 /// ALL cuts per stage (active AND inactive)
 pub stage_cuts: Vec<StageCuts>,

 /// Visited states (for cut selection)
 pub stage_states: Vec<StageStates>,

 /// Vertices (if inner approximation enabled)
 pub stage_vertices: Option<Vec<StageVertices>>,

 /// Full RNG state (not just seed)
 pub rng_state: RngState,

 /// Solver basis per stage (optional but recommended)
 pub solver_basis: Option<Vec<SolverBasis>>,
}

impl Checkpoint {
 /// Restore LP to exact state for reproducible resume
 pub fn restore_lp(&self, stage: usize, lp: &mut LpProblem, cut_manager: &CutSlotManager) {
 let stage_cuts = &self.stage_cuts[stage];

 // Reconstruct ALL cuts (not just active)
 for cut in &stage_cuts.cuts {
 let slot = cut.slot_index;

 // Set coefficients (same for active and inactive)
 lp.set_cut_coefficients(
 slot,
 cut.intercept,
 &cut.coefficients,
 cut_manager,
);

 // Set bound based on active state
 if cut.is_active {
 lp.enable_cut(slot, cut.intercept, cut_manager);
 } else {
 lp.disable_cut(slot, cut_manager);
 }
 }
 }
}
```

## Warm-Start vs Resume:

Mode	Cut Loading	RNG State	Capacity	Results
fresh	None	From config seed	max_iter × fwd_passes	Deterministic from seed
warm_start	All from policy	Fresh from config seed	loaded + new capacity	Different from original run



Mode	Cut Loading	RNG State	Capacity	Results
resume	All + exact state	Restored	Same as checkpoint	<b>Bit-for-bit identical</b>

#### Implementation Notes:

1. **Inactive cuts MUST be serialized:** They occupy LP rows and affect solver numerics
2. **Slot indices are immutable:** Once assigned, a cut keeps its slot forever
3. **RNG state is critical:** Same random scenarios → same forward pass states → same cuts
4. **Basis restoration is optional but recommended:** Without it, solver may choose different pivots

### 7.3 Parquet Configuration (for non-policy data)

```

/// Parquet writer settings for simulation and training outputs
pub struct ParquetConfig {
 /// Compression algorithm
 pub compression: Compression::ZSTD(ZstdLevel::try_new(3).unwrap()),

 /// Row group size (for parallel reading)
 pub row_group_size: 100_000,

 /// Enable statistics
 pub enable_statistics: true,

 /// Dictionary encoding threshold
 pub dictionary_enabled: true,
 pub dictionary_page_size_limit: 1_048_576,
}

```

## 8. Validation Requirements

### 8.1 Input Validation Phases

#### INPUT VALIDATION PIPELINE

Phase 0: Canonicalization (Order Invariance)

- Sort all entity collections by ID
- Sort stages by ID
- Sort blocks within stages by ID
- Sort generic constraints by ID
- Verify IDs are unique within each collection
- (See Section 1.3 for full order-invariance requirements)

Phase 1: Schema Validation

- JSON Schema validation (config, system, temporal)
- Parquet schema validation (constraints, checkpoint)
- Required field presence
- Type correctness

Phase 2: Reference Integrity

- Bus IDs exist for lines, hydros, thermals
- Downstream hydro IDs exist (cascade)
- Model IDs in uncertainty\_models exist
- Stage IDs in transitions exist
- Season IDs match distribution definitions

#### Phase 3: Business Rules

- Hydro cascade is acyclic
- min initial max (storage, generation)
- Probabilities sum to 1.0
- Correlation matrices are positive semi-definite
- AR coefficients ensure stationarity
- Block weights sum to 1.0
- Deficit segments are monotonically increasing
- stopping\_rules must contain at least one iteration\_limit rule
- GNL thermal gnl\_config.lag\_stages must be 1
- gnl\_pipeline thermal\_id must reference thermal with gnl\_config
- gnl\_pipeline committed\_mw must be within thermal bounds

#### Phase 3b: Conditional Validation (based on config modes)

- IF horizon.mode = "infinite\_periodic":
  - At least one transition must create cycle
  - Cycle transitions must have discount\_rate > 0
  - max\_horizon\_length must be specified
- IF horizon.mode = "markovian":
  - markov\_states must be defined in stages.json
  - All transitions must specify valid markov states
  - Markov transition probabilities must sum to 1.0 per source
- IF simulation.sampling\_scheme.type = "external":
  - simulation/external\_scenarios/ directory must exist
  - inflows.parquet must exist with correct schema
- IF thermal has gnl\_config:
  - gnl\_pipeline entries should cover all lag\_stages

#### Phase 4: Dimension Consistency

- Load profiles cover all (stage, block, bus) combinations
- Inflow history covers all hydros with PAR models
- Seasonal parameters have correct length (num\_seasons)
- Correlation matrix dimensions match entity count

#### Phase 5: Warm-start Compatibility

- State dimension matches current system
- Cut stage IDs exist in current stage graph
- Config hash matches (optional strict mode)

## 8.2 Validation Error Types

```
#[derive(Error, Debug)]
pub enum ValidationError {
 // Schema errors
 #[error("JSON schema validation failed for {file}: {details}")]
```

```

JsonSchema { file: String, details: String },

#[error("Parquet schema mismatch in {file}: expected {expected}, got {actual}")]
ParquetSchema { file: String, expected: String, actual: String },

// Reference errors
#[error("{entity_type} {entity_id} references non-existent {ref_type} {ref_id}")]
BrokenReference {
 entity_type: String,
 entity_id: u32,
 ref_type: String,
 ref_id: u32,
},

// Business rule errors
#[error("Hydro cascade contains cycle: {cycle:?}")]
CyclicCascade { cycle: Vec<u32> },

#[error("Value out of range: {field} = {value}, expected [{min}, {max}]")]
OutOfRange { field: String, value: f64, min: f64, max: f64 },

#[error("Correlation matrix is not positive semi-definite for block {block}")]
NotPositiveSemiDefinite { block: String },

#[error("Missing required stopping rule: iteration_limit")]
MissingIterationLimit,

#[error("Invalid GNL configuration for thermal {thermal_id}: {details}")]
InvalidGnlConfig { thermal_id: u32, details: String },

#[error("GNL pipeline references thermal {thermal_id} without gnl_config")]
GnlPipelineInvalidThermal { thermal_id: u32 },

// Conditional validation errors
#[error("Infinite periodic mode requires at least one cycle in transitions")]
NoCycleInInfiniteMode,

#[error("Cycle transitions must have discount_rate > 0 for infinite periodic mode")]
MissingDiscountInCycle,

#[error("Markovian mode requires markov_states definition in stages.json")]
MissingMarkovStates,

#[error("Markov transition probabilities from state {state} don't sum to 1.0: {sum}")]
InvalidMarkovProbabilities { state: u32, sum: f64 },

#[error("External sampling scheme requires simulation/external_scenarios/ directory")]
MissingExternalScenarios,

// Dimension errors
#[error("Missing data for ({stage}, {block}, {entity}): expected {expected} rows")]
MissingTimeSeries {
 stage: u32,
 block: u32,
 entity: String,
 expected: usize,
},

```

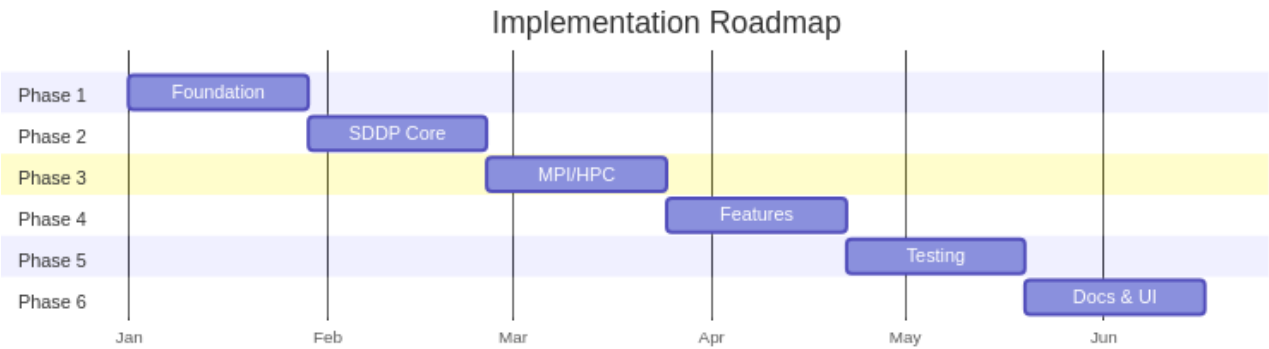
```
// Warm-start errors
#[error("Warm-start state dimension mismatch: expected {expected}, got {actual}")]
WarmstartDimensionMismatch { expected: u32, actual: u32 },
}
```

9. Next Steps

9.1 Implementation Timeline Overview

POWE.RS v2.0 Implementation Roadmap (6 months):

Phase	Weeks	Focus Areas
Phase 1	1-4	Core Foundation, Data I/O Layer
Phase 2	5-8	SDDP Algorithm Core
Phase 3	9-12	MPI/HPC Foundation, Optimization
Phase 4	13-16	Algorithm Features
Phase 5	17-20	Testing & Validation
Phase 6	21-24	Documentation, Frontend



Team Size: 3-4 senior Rust developers with HPC experience

9.2 Phase 1: Foundation (Weeks 1-4)

Goal: Working single-threaded SDDP solver with new data model

Week 1-2: Core Infrastructure

Task	Description	Deliverable
1.1	Project structure	Cargo workspace with powers-core, powers-io, powers-cli crates
1.2	JSON schema validation	Runtime validation using jsonschema crate for all config files
1.3	Parquet I/O layer	Read/write time series data using arrow2 or parquet crate
1.4	FlatBuffers code generation	Generate Rust code from .fbs schemas (Section 7.2.1)

Task	Description	Deliverable
1.5	<b>Error handling</b>	Implement <code>ValidationError</code> enum (Section 8.2) with <code>thiserror</code>
1.6	<b>Logging infrastructure</b>	Structured logging with <code>tracing</code> crate

### Week 3-4: Data Model Implementation

Task	Description	Deliverable
1.7	<b>System model parsing</b>	Load <code>system/*.json</code> (topology, hydros, thermals)
1.8	<b>Temporal data parsing</b>	Load <code>stages.json</code> , <code>initial_conditions.json</code>
1.9	<b>Stochastic model parsing</b>	Load scenarios, correlations, time series
1.10	<b>Declaration order invariance</b>	Canonical sorting by ID (Section 1.3)
1.11	<b>Full input validation pipeline</b>	Phases 0-4 from Section 8.1
1.12	<b>Test case infrastructure</b>	Small, medium, large test cases with known solutions

**Milestone M1:** Load and validate all input files for production-scale case

## 9.3 Phase 2: SDDP Algorithm Core (Weeks 5-8)

**Goal:** Complete single-rank SDDP training and simulation

### Week 5-6: LP Model Building

Task	Description	Deliverable
2.1	<b>HiGHS integration</b>	Rust bindings for HiGHS solver via <code>highs</code> crate
2.2	<b>LP model builder</b>	Build stage subproblems from system model
2.3	<b>Block mode support</b>	Parallel and chronological block handling
2.4	<b>Penalty system</b>	Deficit, curtailment, spillage penalties
2.5	<b>Generic constraints</b>	Load linear constraints from Parquet
2.6	<b>FPHA implementation</b>	Four-Point Hyperplane Approximation for efficiency curves

### Week 7-8: SDDP Training Loop

Task	Description	Deliverable
2.7	<b>Forward pass</b>	Scenario sampling, decision making
2.8	<b>Backward pass</b>	Cut generation, dual extraction
2.9	<b>Cut management</b>	Add/store cuts in FlatBuffers format
2.10	<b>Stopping rules</b>	<code>iteration_limit</code> , <code>time_limit</code> , <code>bound_stalling</code>
2.11	<b>Checkpointing</b>	Save/load policy state

Task	Description	Deliverable
2.12	<b>Training output</b>	Write training_summary.json, convergence.parquet

**Milestone M2:** Train policy on test case, verify convergence against reference

## 9.4 Phase 3: MPI/HPC Foundation (Weeks 9-12)

**Goal:** Distributed training across multiple nodes

### Week 9-10: Basic MPI

Task	Description	Deliverable
3.1	<b>ferroMPI integration</b>	Add MPI bindings as optional feature
3.2	<b>Scheduler detection</b>	Auto-detect SLURM/PBS/LSF, respect allocations
3.3	<b>Scenario distribution</b>	Round-robin forward pass parallelization
3.4	<b>Flat cut aggregation</b>	MPI_Gather for cut coefficients
3.5	<b>FCF broadcast</b>	MPI_Bcast for updated cuts
3.6	<b>Barrier synchronization</b>	Iteration barriers for consistency

### Week 11-12: Memory Optimization

Task	Description	Deliverable
3.7	<b>Intra-node FCF sharing</b>	MPI shared memory windows (Section 6.6)
3.8	<b>NUMA-aware allocation</b>	First-touch initialization pattern
3.9	<b>Memory profiling</b>	Track RSS, validate memory budget
3.10	<b>Load balance metrics</b>	Per-rank timing collection

**Milestone M3:** 4-node training with linear weak scaling

## 9.5 Phase 4: HPC Optimization (Weeks 13-16)

**Goal:** Production-scale performance

### Week 13-14: Communication Optimization

Task	Description	Deliverable
4.1	<b>Hierarchical aggregation</b>	Tree-based cut gathering (Section 6.4)
4.2	<b>Pipelined backward pass</b>	Overlap computation/communication
4.3	<b>Non-blocking collectives</b>	MPI_Ibcast, MPI_Igather
4.4	<b>Persistent collectives</b>	MPI 4.0 optimization for iterative patterns

## Week 15-16: I/O Optimization

Task	Description	Deliverable
4.5	<b>Parallel warm-start loading</b>	Distribute stage files across ranks
4.6	<b>Multi-writer checkpointing</b>	Reduce filesystem contention
4.7	<b>Compression tuning</b>	Optimal ZSTD levels for checkpoint I/O
4.8	<b>MPI timing instrumentation</b>	Full metrics from Section 6.8

**Milestone M4:** 128-rank training with <20% communication overhead

---

## 9.6 Phase 5: Algorithm Features (Weeks 17-20)

**Goal:** Complete algorithm feature set

### Week 17-18: Risk Measures & Inner Approximation

Task	Description	Deliverable
5.1	<b>CVaR risk measure</b>	Modified cut generation with risk weights
5.2	<b>Per-stage risk profiles</b>	Varying risk aversion across horizon
5.3	<b>Lipschitz computation</b>	Backward accumulation from penalties
5.4	<b>Upper bound computation</b>	Vertex-based inner approximation
5.5	<b>Inner policy simulation</b>	Conservative policy evaluation

### Week 19-20: Advanced Features

Task	Description	Deliverable
5.6	<b>Simulation stopping rule</b>	Hybrid bound+policy stability
5.7	<b>Statistical stopping rule</b>	Monte Carlo confidence intervals
5.8	<b>Cut selection</b>	Domination-based cut removal
5.9	<b>Simulation engine</b>	Full simulation output (Section 4)
5.10	<b>External scenarios</b>	Support for backtesting mode

**Milestone M5:** All algorithm features passing integration tests

---

## 9.7 Phase 6: Testing & Validation (Weeks 21-24)

**Goal:** Production-ready quality assurance

### Week 21-22: Test Coverage

Task	Description	Deliverable
6.1	<b>Unit tests</b>	>80% coverage on core algorithms
6.2	<b>Integration tests</b>	End-to-end training/simulation
6.3	<b>Benchmark regression tests</b>	Criterion.rs performance tracking
6.4	<b>Memory regression tests</b>	DHAT/Massif analysis

Task	Description	Deliverable
6.5	<b>MPI correctness tests</b>	Multi-rank determinism verification

## Week 23-24: Validation & Documentation

Task	Description	Deliverable
6.6	<b>Reference case validation</b>	Match known optimal solutions
6.7	<b>Performance benchmarks</b>	Document throughput on reference hardware
6.8	<b>User documentation</b>	CLI reference, configuration guide
6.9	<b>API documentation</b>	Rustdoc for public interfaces
6.10	<b>Deployment guides</b>	SLURM, PBS, AWS EFA, Docker

**Milestone M6:** Production-ready v2.0 release candidate

## 9.8 Frontend Development (Parallel Track, Weeks 17-24)

**Goal:** Web-based UI for case configuration and monitoring **Team:** 1-2 frontend developers (can work in parallel with backend)

### Architecture Decision

Option	Pros	Cons	Recommendation
<b>Web (React/TypeScript)</b>	Cross-platform, modern UI, easy deployment	Requires backend API	<b>Recommended</b>
Desktop (Tauri/Electron)	Native feel, offline	Platform-specific builds	Alternative
CLI-only	No extra dependencies	Limited usability for non-experts	Minimum viable

## Week 17-18: Backend API

Task	Description	Deliverable
F.1	<b>REST API design</b>	OpenAPI 3.0 specification
F.2	<b>Validation endpoints</b>	/api/validate for input checking
F.3	<b>Case management</b>	CRUD for study cases
F.4	<b>Run management</b>	Start/stop/monitor training jobs

## Week 19-20: Core UI Components

Task	Description	Deliverable
F.5	<b>System editor</b>	Hydro/thermal/bus configuration forms
F.6	<b>Cascade visualizer</b>	Interactive hydro cascade diagram
F.7	<b>Network topology</b>	Bus/line visualization with D3.js



Task	Description	Deliverable
F.8	<b>Time series import</b>	CSV/Excel import with preview

## Week 21-22: Advanced UI

Task	Description	Deliverable
F.9	<b>Stage/block editor</b>	Temporal structure configuration
F.10	<b>Stochastic model UI</b>	PAR parameters, correlations
F.11	<b>Validation feedback</b>	Real-time error highlighting
F.12	<b>Template system</b>	Start from example cases

## Week 23-24: Monitoring & Polish

Task	Description	Deliverable
F.13	<b>Training monitor</b>	Real-time convergence plots
F.14	<b>Results viewer</b>	Simulation output analysis
F.15	<b>Case comparison</b>	Diff between configurations
F.16	<b>Export/import</b>	Full case serialization

## Frontend Technology Stack:

- Framework: React 19 + TypeScript
- UI Components: Radix UI + Tailwind CSS v4
- State: React Query / SWR for API caching
- Visualization: D3.js for network diagrams, Recharts for time series
- API: Axum (Rust) or FastAPI (Python) backend
- Build: Vite

**Milestone F1:** Fully functional web UI for case configuration

## 9.9 Validation Milestones Summary

ID	Milestone	Week	Acceptance Criteria
M1	Input Loading	4	Load production-scale case in <5 seconds
M2	Algorithm Correctness	8	Match reference solution within 0.1%
M3	Basic MPI	12	Linear scaling to 4 nodes
M4	HPC Optimization	16	80% parallel efficiency at 128 ranks
M5	Feature Complete	20	All algorithm features tested
M6	Production Ready	24	Release candidate quality
F1	Frontend	24	Full case configuration UI

## 9.10 Risk Register

Risk	Probability	Impact	Mitigation
HiGHS numerical issues	Medium	High	Early integration testing, fallback solver support
MPI deadlocks	Medium	Medium	Extensive multi-rank testing, deterministic replay
Memory exhaustion	Low	High	Continuous profiling, memory budgeting
Performance regression	Medium	Medium	Criterion.rs benchmarks in CI
Scope creep	Medium	Medium	Strict phase gates, prioritized backlog

---

*This specification is a living document. Update as decisions are finalized.*